

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP.HCM

KHOA CÔNG NGHỆ THÔNG TIN

BỘ MÔN AN TOÀN THÔNG TIN



LÊ ANH KHOA – 22162016

NGUYỄN VĂN TRƯỜNG – 22162052

KHÓA LUẬN TỐT NGHIỆP

ĐỀ TÀI
PHÁT HIỆN VÀ PHÂN LOẠI MÃ ĐỘC ỨNG DỤNG VISION
TRANSFORMER VÀ ENSEMBLE LEARNING

GIẢNG VIÊN HƯỚNG DẪN
ThS. NGUYỄN THỊ THANH VÂN

KHÓA 2022-2026

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP.HCM

KHOA CÔNG NGHỆ THÔNG TIN

BỘ MÔN CÔNG AN TOÀN THÔNG TIN



LÊ ANH KHOA - 22162016

NGUYỄN VĂN TRƯỜNG - 22162052

KHÓA LUẬN TỐT NGHIỆP

ĐỀ TÀI
PHÁT HIỆN VÀ PHÂN LOẠI MÃ ĐỘC ỨNG DỤNG VISION
TRANSFORMER VÀ ENSEMBLE LEARNING

GIẢNG VIÊN HƯỚNG DẪN
ThS. NGUYỄN THỊ THANH VÂN

KHÓA 2022 - 2026

ĐH SƯ PHẠM KỸ THUẬT
TP.HCM
KHOA CNTT

XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập – Tự do – Hạnh Phúc

PHIẾU NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

Họ và tên Sinh viên 1: Lê Anh Khoa

MSSV 1: 22162016

Họ và tên Sinh viên 2: Nguyễn Văn Trường

MSSV 2: 22162052

Ngành: An toàn thông tin

Tên đề tài: Phát hiện và phân loại mã độc ứng dụng Vision Transformer và Ensemble Learning

Họ và tên Giáo viên hướng dẫn: ThS. Nguyễn Thị Thanh Vân

NHẬN XÉT

- Về nội dung đề tài & khối lượng thực hiện:
- Ưu điểm:
- Khuyết điểm:
- Đề nghị cho bảo vệ hay không?
- Đánh giá loại:
- Điểm:

Tp. Hồ Chí Minh, ngày tháng năm 2026
Giáo viên hướng dẫn
(Ký & ghi rõ họ tên)

ĐH SƯ PHẠM KỸ THUẬT
TP.HCM
KHOA CNTT

XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập – Tự do – Hạnh Phúc

PHIẾU NHẬN XÉT CỦA GIÁO VIÊN PHẢN BIỆN

Họ và tên Sinh viên 1: Lê Anh

MSSV 1: 22162016

Họ và tên Sinh viên 2: Nguyễn Văn Trường

MSSV 2: 22162052

Ngành: An toàn thông tin

Tên đề tài: Phát hiện và phân loại mã độc ứng dụng Vision Transformer và Ensemble Learning

Họ và tên Giáo viên phản biện: TS. Lê Vĩnh Thịnh

NHẬN XÉT

- Về nội dung đề tài & khối lượng thực hiện:
- Ưu điểm:
- Khuyết điểm:
- Đề nghị cho bảo vệ hay không?
- Đánh giá loại:
- Điểm:

Tp. Hồ Chí Minh, ngày tháng năm 2026

Giáo viên phản biện
(Ký & ghi rõ họ tên)

LỜI CẢM ƠN

Hoàn thành khóa luận tốt nghiệp là một quá trình đòi hỏi sự kiên trì, nỗ lực học hỏi và khả năng vận dụng kiến thức vào thực tiễn. Việc triển khai đề tài “Phát hiện và phân loại mã độc ứng dụng Vision Transformer và Ensemble Learning” là kết quả của quá trình nghiên cứu, tìm hiểu và thực nghiệm nghiêm túc, đồng thời nhận được sự hỗ trợ quý báu từ nhiều cá nhân và tập thể.

Trước hết, em xin bày tỏ lòng biết ơn sâu sắc đến Cô Nguyễn Thị Thanh Vân, giảng viên hướng dẫn đã tận tình đồng hành trong suốt quá trình thực hiện đề tài. Cô không chỉ định hướng về mặt chuyên môn mà còn luôn dành thời gian góp ý, chỉnh sửa và hỗ trợ em trong quá trình nghiên cứu. Những kiến thức, kinh nghiệm và sự động viên của Cô đã giúp em vượt qua nhiều khó khăn trong quá trình xây dựng mô hình học máy, học sâu, triển khai các phương pháp Ensemble Learning cũng như phát triển hệ thống phát hiện và phân loại mã độc.

Tiếp theo, em xin chân thành cảm ơn quý Thầy, Cô thuộc Trường Đại học Công Nghệ Kỹ thuật Thành phố Hồ Chí Minh, đặc biệt là quý Thầy, Cô Khoa Công nghệ Thông tin và Bộ môn Mạng và An ninh mạng đã tận tâm giảng dạy và truyền đạt những kiến thức nền tảng quý báu trong suốt quá trình học tập. Những kiến thức về an toàn thông tin, học máy, học sâu và phát triển phần mềm chính là cơ sở quan trọng để em thực hiện và hoàn thành đề tài này.

Em cũng xin gửi lời cảm ơn đến các tổ chức, cộng đồng nghiên cứu và các nguồn dữ liệu mở đã hỗ trợ cung cấp tài liệu tham khảo, bộ dữ liệu và các công cụ phục vụ cho quá trình nghiên cứu. Những nguồn tài nguyên này đã tạo điều kiện thuận lợi cho việc xây dựng, huấn luyện và đánh giá các mô hình phát hiện, phân loại mã độc trong đề tài.

Mặc dù đã dành nhiều thời gian và nỗ lực cho quá trình nghiên cứu, do hạn chế về thời gian, nguồn lực và kinh nghiệm thực tiễn nên báo cáo khó tránh khỏi những thiếu sót nhất định. Em rất mong nhận được những ý kiến đóng góp và nhận xét từ quý Thầy, Cô cùng các chuyên gia để đề tài được hoàn thiện hơn và có khả năng ứng dụng cao hơn trong tương lai.

MỤC LỤC

PHẦN MỘT: MỞ ĐẦU	1
1. Lý do chọn đề tài	1
2. Mục tiêu nghiên cứu	1
3. Phương pháp nghiên cứu	3
PHẦN HAI: NỘI DUNG NGHIÊN CỨU	4
CHƯƠNG 1: TỔNG QUAN VỀ PHÁT HIỆN VÀ PHÂN LOẠI MÃ ĐỘC	4
1.1. Giới thiệu về mã độc	4
1.1.1. Phân loại mã độc	4
1.1.2. Các kỹ thuật né tránh của mã độc	8
1.2. Giới thiệu về tệp Portable Executable (PE)	10
1.3. Các phương pháp phân tích	13
1.3.1. Phân tích tĩnh	13
1.3.2. Phân tích động	14
1.4. Các kỹ thuật phát hiện mã độc	15
1.4.1. Phát hiện dựa trên chữ ký	15
1.4.2. Phát hiện dựa trên hành vi	16
1.4.3. Phát hiện dựa trên bộ nhớ	17
1.4.4. Phát hiện dựa trên lưu lượng mạng	17
CHƯƠNG 2: ỨNG DỤNG MACHINE LEARNING, DEEP LEARNING TRONG PHÁT HIỆN MÃ ĐỘC	19
2.1. Machine Learning	19
2.1.1. Random Forest	19
2.1.2. Naive Bayes	21
2.1.3. Support vector machine	24
2.2. Deep Learning	26
2.2.1. ResNet50	26
2.2.2. Vision Transformers	29
2.3. Ensemble Learning	32
CHƯƠNG 3: THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG PHÁT HIỆN VÀ PHÂN LOẠI MÃ ĐỘC ỨNG DỤNG VISION TRANSFORMER VÀ ENSEMBLE LEARNING	35
3.1. Bộ dữ liệu Microsoft Malware Classification Challenge Dataset 2015 (BIG 2015)	35

3.1.1. Tổng quan bộ dữ liệu	35
3.1.2. Tiền xử lý dữ liệu	36
3.2. Thiết kế mô hình	38
3.2.1. Thiết kế các mô hình Machine Learning	38
3.2.2. Thiết kế các mô hình Deep Learning	43
3.2.3. Thiết kế kỹ thuật Ensemble Learning	50
3.3. Xây dựng hệ thống phát hiện và phân loại mã độc	59
3.3.1. Sơ đồ chức năng hệ thống	59
3.3.2. Triển khai	61
CHƯƠNG 4: THỰC NGHIỆM VÀ ĐÁNH GIÁ	72
4.1. Môi trường thực nghiệm	72
4.2. Kết quả	73
4.3. Đánh giá	76
PHẦN BA: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	78
1. Kết quả nghiên cứu của đề tài	78
2. Hạn chế của sản phẩm	79
3. Hướng phát triển	80
TÀI LIỆU THAM KHẢO	82

DANH MỤC HÌNH ẢNH

Hình 1: Cấu trúc tệp PE [1]	11
Hình 2: Thuật toán Radom Forest	19
Hình 3: Mô hình ResNet-50 [20]	27
Hình 4: Kiến trúc Vision Transformer [26]	30
Hình 5: Quy trình tiền xử lý dữ liệu	36
Hình 6: Kết quả của mô hình ResNet-50 sau 50 epoch	44
Hình 7: Sơ đồ chức năng hệ thống phát hiện mã độc.....	59
Hình 8: Cấu trúc thư mục backend hệ thống.....	62
Hình 9: Cấu trúc thư mục frontend hệ thống.....	66
Hình 10: Giao diện website để tải và phân tích tệp.....	67
Hình 11: Giao diện website khi đang phân tích tệp	67
Hình 12: Giao diện website khi phát hiện mã độc	68
Hình 13: Giao diện website thống kê cá nhân.....	69
Hình 14: Giao diện lịch sử quét.....	70
Hình 15: Giao diện dashboard.....	70
Hình 16: Kết quả của các mô hình	74
Hình 17: Thời gian huấn luyện của các mô hình.....	75

DANH MỤC BẢNG BIỂU

Bảng 1: Các nghiên cứu sử dụng Random Forest trong phát hiện mã độc	20
Bảng 2: Các nghiên cứu sử dụng Navie Bayes trong phát hiện mã độc	23
Bảng 3: Các nghiên cứu sử dụng SVM trong phát hiện mã độc	25
Bảng 4: Các nghiên cứu sử dụng ResNet-50 trong phát hiện mã độc.....	28
Bảng 5: Các nghiên cứu sử dụng Vision Transformer trong phát hiện mã độc	31
Bảng 6: Các nghiên cứu sử dụng Ensemble Learning trong phát hiện mã độc	34
Bảng 7: Bảng thống kê kết quả của các bộ siêu tham số	38
Bảng 8: Kết quả của Random Forest.....	39
Bảng 9: Kết quả của Support Vector Machine.....	41
Bảng 10: Kết quả của Naive Bayes Multinomial.....	42
Bảng 11: Kết quả cuối cùng của ResNet-50.....	44
Bảng 12: Kết quả thử nghiệm các bộ siêu tham số của ViT	46
Bảng 13: Kết quả của Vision Transformer.....	47
Bảng 14: Kết quả của Pyramid Vision Transformer	49
Bảng 15: Kết quả của Sum Rule	51
Bảng 16: Kết quả của Weighted Sum Rule.....	52
Bảng 17: Kết quả của Product Rule	53
Bảng 18: Kết quả của Max Rule	54
Bảng 19: Kết quả của Min Rule	56
Bảng 20: Kết quả của Median Rule.....	57
Bảng 21: Kết quả của Mean Rule.....	58
Bảng 22: Các API chính của backend	64
Bảng 23: Cấu hình môi trường huấn luyện	72
Bảng 24: Cấu hình môi trường triển khai.....	73
Bảng 25: Các nghiên cứu sử dụng tập dữ liệu BIG 2015.....	75

DANH MỤC TỪ NGỮ VIẾT TẮT

STT	Từ viết tắt	Tiếng Anh
1	AI	Artificial Intelligence
2	API	Application Programming Interface
3	CNN	Convolutional Neural Network
4	CPU	Central Processing Unit
5	CSV	Comma-Separated Values
6	DL	Deep Learning
7	F1-score	F1 Score
8	FN	False Negative
9	FP	False Positive
10	GPU	Graphics Processing Unit
11	JSON	JavaScript Object Notation
12	ML	Machine Learning
13	ORM	Object Relational Mapping
14	PE	Portable Executable
15	PVT	Pyramid Vision Transformer
16	RAM	Random Access Memory
17	REST	Representational State Transfer
18	RF	Random Forest
19	SIEM	Security Information and Event Management
20	SQL	Structured Query Language
21	SVM	Support Vector Machine
22	TN	True Negative
23	TP	True Positive
24	UI	User Interface
25	URL	Uniform Resource Locator
26	ViT	Vision Transformer
27	XAI	Explainable Artificial Intelligence
28	XGB	Extreme Gradient Boosting

PHẦN MỘT: MỞ ĐẦU

1. Lý do chọn đề tài

Trong bối cảnh an ninh mạng ngày càng phức tạp, số lượng và mức độ tinh vi của các loại mã độc liên tục gia tăng, khiến các phương pháp phát hiện truyền thống dựa trên chữ ký dần bộc lộ nhiều hạn chế, đặc biệt khi đối mặt với các biến thể mới hoặc mã độc sử dụng kỹ thuật che giấu như obfuscation và polymorphism. Điều này đặt ra yêu cầu cần có các phương pháp thông minh hơn, có khả năng học và tổng quát hóa từ dữ liệu. Sự phát triển mạnh mẽ của học sâu đã mở ra nhiều hướng tiếp cận mới, trong đó Vision Transformer (ViT) nổi bật nhờ khả năng khai thác quan hệ toàn cục trong dữ liệu. Khi chuyển đổi mã độc thành dạng ảnh từ dữ liệu byte, ViT có thể học được các đặc trưng không gian phức tạp mà các mô hình truyền thống khó nắm bắt, từ đó nâng cao hiệu quả phát hiện và phân loại.

Bên cạnh đó, việc áp dụng Ensemble Learning giúp cải thiện độ chính xác và tính ổn định của hệ thống bằng cách kết hợp nhiều mô hình khác nhau, đặc biệt hữu ích trong bài toán dữ liệu không cân bằng và nhiễu nhiễu như malware. Kết hợp Vision Transformer với Ensemble Learning vì vậy trở thành một hướng nghiên cứu tiềm năng, tận dụng được ưu điểm của cả hai phương pháp. Đồng thời, các bộ dữ liệu chuẩn như Microsoft Malware Classification Challenge Dataset (2015) cung cấp nền tảng phù hợp để huấn luyện và đánh giá mô hình. Từ đó, đề tài được lựa chọn không chỉ nhằm nâng cao hiệu quả phát hiện mã độc mà còn góp phần ứng dụng các kỹ thuật học sâu hiện đại vào lĩnh vực an ninh mạng.

2. Mục tiêu nghiên cứu

Mục tiêu tổng quát của đề tài là nghiên cứu, xây dựng và triển khai một hệ thống phát hiện và phân loại mã độc dựa trên sự kết hợp giữa các kỹ thuật học sâu, học máy và Ensemble Learning nhằm nâng cao độ chính xác, khả năng tổng quát hóa và tính ứng dụng thực tế. Đề tài tập trung khai thác các đặc trưng từ dữ liệu mã độc dưới nhiều dạng biểu diễn khác nhau, bao gồm dữ liệu byte và ảnh được tạo từ tệp thực thi, từ đó tận dụng ưu điểm của các mô hình học sâu hiện đại như Vision Transformer (ViT), Pyramid Vision Transformer (PVT) cùng với các mô hình học máy truyền thống như Random Forest. Thông qua việc kết hợp nhiều mô hình và chiến lược ensemble, nghiên cứu

hướng đến việc xây dựng một hệ thống có khả năng nhận diện và phân loại mã độc hiệu quả trong môi trường thực tế.

Để đạt được mục tiêu trên, đề tài thực hiện các nhiệm vụ cụ thể sau:

- Hệ thống hóa cơ sở lý thuyết về mã độc, các phương pháp phân tích mã độc và các kỹ thuật phát hiện mã độc hiện nay.
- Nghiên cứu nguyên lý hoạt động của các mô hình học máy và học sâu được sử dụng trong bài toán phân loại mã độc, bao gồm Naive Bayes, SVM, Random Forest, ResNet-50, Vision Transformer (ViT) và Pyramid Vision Transformer (PVT).
- Xây dựng quy trình tiền xử lý dữ liệu từ bộ Microsoft Malware Classification Challenge Dataset (2015), bao gồm trích xuất đặc trưng byte và chuyển đổi dữ liệu mã độc sang dạng ảnh phục vụ cho các mô hình học sâu.
- Nghiên cứu và triển khai các phương pháp Ensemble Learning như Sum Rule, Weighted Sum Rule, Product Rule, Max Rule, Min Rule, Mean Rule và Median Rule nhằm kết hợp kết quả từ nhiều mô hình phân loại.
- Thực nghiệm, đánh giá và so sánh hiệu quả của các mô hình dựa trên các chỉ số Accuracy, Precision, Recall và F1-score để lựa chọn phương pháp phù hợp nhất.
- Mở rộng bộ dữ liệu phục vụ triển khai thực tế bằng cách thu thập thêm 25.464 tệp lành tính từ hệ điều hành Windows và 65.536 mẫu mã độc từ VirusShare để xây dựng bộ phân loại nhị phân giữa tệp lành tính và mã độc.
- Thiết kế và triển khai hệ thống phát hiện, phân loại mã độc dưới dạng ứng dụng web, cho phép người dùng tải lên tệp thực thi và nhận kết quả phân tích trực tuyến.

Đối tượng nghiên cứu của đề tài là các mẫu mã độc và các phương pháp phát hiện, phân loại mã độc dựa trên kỹ thuật học máy và học sâu. Cụ thể, nghiên cứu tập trung vào việc khai thác đặc trưng từ dữ liệu mã độc ở dạng tĩnh, bao gồm dữ liệu nhị phân và mã assembly, từ đó xây dựng các mô hình có khả năng nhận diện và phân loại các họ malware khác nhau. Ngoài ra, đề tài cũng xem xét các kiến trúc học sâu hiện đại như Vision Transformer, cùng với các phương pháp học máy truyền thống và kỹ thuật kết hợp mô hình (Ensemble Learning) nhằm nâng cao hiệu quả phân loại.

3. Phương pháp nghiên cứu

Đề tài được thực hiện theo phương pháp nghiên cứu kết hợp giữa lý thuyết và thực nghiệm nhằm xây dựng hệ thống phát hiện và phân loại mã độc dựa trên học máy, học sâu và Ensemble Learning. Trước hết, nghiên cứu tiến hành khảo sát, tổng hợp và phân tích các tài liệu liên quan đến mã độc, kỹ thuật phân tích mã độc, cấu trúc tệp thực thi PE (Portable Executable), các phương pháp phát hiện mã độc hiện nay cũng như các mô hình học máy và học sâu phổ biến. Đồng thời, đề tài tập trung tìm hiểu nguyên lý hoạt động của các mô hình Naive Bayes, SVM, Random Forest, ResNet-50, Vision Transformer (ViT), Pyramid Vision Transformer (PVT) và các kỹ thuật Ensemble Learning để làm cơ sở cho việc xây dựng mô hình đề xuất.

Về mặt thực nghiệm, nghiên cứu sử dụng bộ dữ liệu Microsoft Malware Classification Challenge Dataset (2015) để xây dựng bài toán phân loại đa lớp gồm 9 họ mã độc. Dữ liệu được tiền xử lý theo nhiều hướng khác nhau, bao gồm trích xuất đặc trưng byte phục vụ cho các mô hình học máy và chuyển đổi dữ liệu nhị phân thành ảnh để phục vụ các mô hình học sâu. Bên cạnh đó, đề tài còn mở rộng tập dữ liệu bằng cách thu thập thêm 25.464 tệp lành tính từ hệ điều hành Windows và 65.536 mẫu mã độc từ VirusShare nhằm xây dựng bộ phân loại nhị phân giữa tệp lành tính và mã độc, phục vụ cho việc triển khai thực tế.

Sau khi hoàn thành quá trình tiền xử lý, các mô hình được xây dựng, huấn luyện và đánh giá trên tập dữ liệu tương ứng. Nghiên cứu đồng thời khảo sát nhiều phương pháp Ensemble Learning như Sum Rule, Weighted Sum Rule, Product Rule, Max Rule, Min Rule, Mean Rule và Median Rule để kết hợp kết quả từ các mô hình thành viên nhằm nâng cao hiệu quả phân loại. Hiệu suất của các mô hình được đánh giá thông qua các chỉ số Accuracy, Precision, Recall và F1-score, đồng thời so sánh với các nghiên cứu liên quan trên cùng bộ dữ liệu.

Cuối cùng, mô hình được lựa chọn sẽ được tích hợp vào một hệ thống phát hiện và phân loại mã độc dưới dạng ứng dụng web. Hệ thống cho phép người dùng tải lên tệp thực thi, thực hiện phân tích tự động và trả về kết quả dự đoán theo thời gian thực. Thông qua việc kết hợp giữa nghiên cứu lý thuyết, thực nghiệm và triển khai ứng dụng, đề tài không chỉ đánh giá hiệu quả của các phương pháp đề xuất mà còn kiểm chứng khả năng ứng dụng trong môi trường thực tế.

PHẦN HAI: NỘI DUNG NGHIÊN CỨU

CHƯƠNG 1: TỔNG QUAN VỀ PHÁT HIỆN VÀ PHÂN LOẠI MÃ ĐỘC

1.1. Giới thiệu về mã độc

Phần mềm độc hại là một thuật ngữ chung, bao trùm các chương trình hoặc đoạn mã được thiết kế với ý đồ xấu, nhằm xâm nhập bất hợp pháp, gây tổn hại hoặc thực hiện các hành vi không mong muốn trên hệ thống máy tính, mạng và dữ liệu [1]. Khái niệm này dựa trên mục đích sử dụng thay vì một kỹ thuật cụ thể nào đó. Do đó, một chương trình được gọi là "malware" không phải vì nó là một virus hay một con sâu máy tính, mà vì nó được tạo ra với mục đích gây hại. Ban đầu, các chương trình này có thể chỉ được tạo ra để thử nghiệm hoặc gây phiền toái, nhưng ngày nay, chúng thường có mục đích gây hại tài chính, chính trị, hoặc làm gián điệp.

1.1.1. Phân loại mã độc

Về mặt phân loại, malware bao gồm nhiều họ và dạng thức khác nhau tùy theo cơ chế hoạt động và mục tiêu. Có thể phân loại thành:

a) Virus

Virus máy tính là một loại chương trình độc hại có khả năng tự sao chép và đính kèm vào các tệp hoặc chương trình hợp pháp khác [2]. Điểm đặc trưng của virus là nó không thể tự lây nhiễm một cách độc lập; nó cần sự tương tác của người dùng để kích hoạt, chẳng hạn như mở một tệp đính kèm trong email hoặc chạy một chương trình bị nhiễm.

Các con đường lây lan của virus rất đa dạng. Trước đây, nó chủ yếu lây nhiễm qua các thiết bị lưu trữ vật lý như đĩa mềm và USB. Hiện nay, các phương thức phổ biến hơn bao gồm lây lan qua Internet (tệp tải xuống từ các trang web giả mạo), và qua email (các tệp đính kèm hoặc liên kết độc hại). Một số tác hại phổ biến của virus bao gồm làm chậm tốc độ xử lý của máy tính, làm hỏng hoặc xóa dữ liệu, và ảnh hưởng đến hoạt động của các phần mềm.

b) Trojan Horse

Trojan Horse là một loại mã độc được ngụy trang tinh vi dưới vỏ bọc của các chương trình, ứng dụng hoặc tệp tin hợp pháp [3]. Khác với virus, Trojan không có khả năng tự sao chép hoặc tự lây lan. Thay vào đó, nó dựa hoàn toàn vào sự lừa dối để dụ dỗ người dùng tự nguyện tải về và cài đặt. Đây là một chiến thuật tấn công phi kỹ thuật (social

engineering), dựa trên sự thiếu cảnh giác của nạn nhân. Trojan thường được phát tán qua email hoặc các trang web bị nhiễm độc.

Mục đích tấn công của Trojan rất đa dạng, dẫn đến việc phân loại thành nhiều biến thể:

- Remote Access Trojan (RAT): cho phép kẻ tấn công kiểm soát máy tính của nạn nhân từ xa.
- Trojan Banker: chuyên đánh cắp thông tin tài chính như số thẻ tín dụng, số tài khoản ngân hàng và mật khẩu.
- Trojan Rootkit: được thiết kế để che giấu sự hiện diện của các mã độc khác khỏi phần mềm diệt virus và tường lửa.
- Destructive Trojan: có khả năng phá hủy dữ liệu quan trọng và làm hỏng hệ điều hành.
- Trojan Downloader: tự động tải về và cài đặt các loại malware khác, bao gồm cả adware và các Trojan mới.

Dấu hiệu nhận biết máy tính bị nhiễm Trojan có thể rất rõ ràng, chẳng hạn như CPU luôn ở mức cao bất thường, màn hình xuất hiện cửa sổ console khi khởi động sau đó nhanh chóng ẩn đi, hoặc các cài đặt hệ thống tự động thay đổi như việc ổ đĩa CD-ROM tự động đóng mở.

c) Worm

Khác biệt cốt lõi của Worm so với Virus là khả năng tự sao chép và lây lan một cách độc lập mà không cần sự tương tác của người dùng hay một tệp tin đính kèm. Worm thường lây lan bằng cách khai thác các lỗ hổng bảo mật trong hệ điều hành hoặc các ứng dụng để xâm nhập vào mạng máy tính (LAN, Internet). Ngay sau khi một thiết bị trong mạng bị lây nhiễm, worm sẽ tự động tìm kiếm các máy tính khác có cùng lỗ hổng và nhân bản sang chúng.

Hậu quả của một cuộc tấn công worm không chỉ giới hạn ở một máy tính mà mang tính hệ thống. Quá trình tự nhân bản không ngừng nghỉ của worm tiêu thụ một lượng lớn tài nguyên mạng và băng thông, dẫn đến tình trạng tắc nghẽn mạng nghiêm trọng, làm chậm tốc độ Internet và thậm chí gây sập toàn bộ hệ thống mạng. Điều này làm cho worm trở thành một mối đe dọa mang tính hệ thống trên diện rộng. Một ví dụ điển hình về sự nguy hiểm của worm là cuộc tấn công WannaCry năm 2017, khi mã độc này đã

tận dụng một lỗ hổng trong giao thức SMB của hệ điều hành Microsoft Windows để tự động lây lan và tấn công hơn 300 tổ chức tại 150 quốc gia, gây thiệt hại ước tính lên đến hàng tỷ đô la.

d) Ransomware

Ransomware là một loại phần mềm độc hại có khả năng mã hóa các tệp tin của người dùng và làm tê liệt toàn bộ hệ thống, sau đó yêu cầu một khoản tiền chuộc để giải mã [4]. Sau khi lây nhiễm, ransomware sẽ nhanh chóng quét toàn bộ hệ thống và sử dụng các thuật toán mã hóa mạnh mẽ để khóa các tệp quan trọng như tài liệu, hình ảnh và cơ sở dữ liệu. Sau đó, một thông báo đòi tiền chuộc sẽ xuất hiện trên màn hình, thường hướng dẫn nạn nhân thanh toán bằng các loại tiền điện tử như Bitcoin để lấy lại quyền truy cập.

Ransomware được phân loại thành hai dạng chính:

- Encrypting Ransomware: dạng tấn công phổ biến nhất, trực tiếp mã hóa dữ liệu và đổi tên đuôi file.
- Locker Ransomware: thay vì mã hóa, loại này chỉ khóa hoàn toàn quyền truy cập của người dùng vào thiết bị, chỉ hiển thị một màn hình yêu cầu trả tiền chuộc.

Các vụ tấn công ransomware đã chuyển từ việc nhắm mục tiêu cá nhân sang tấn công các tổ chức và doanh nghiệp lớn. Điều này nhằm mục đích làm tê liệt hoạt động kinh doanh, gây thiệt hại nghiêm trọng về kinh tế và uy tín. Một số chiến dịch tấn công hiện đại thậm chí còn sử dụng chiến thuật "tống tiền gấp đôi" (double extortion), vừa mã hóa dữ liệu vừa đe dọa công bố dữ liệu nhạy cảm đã đánh cắp nếu nạn nhân không trả tiền. Các vụ tấn công nổi tiếng như WannaCry, NotPetya và Bad Rabbit là những minh chứng điển hình cho mức độ nguy hiểm toàn cầu của loại mã độc này.

e) Spyware

Spyware là một loại phần mềm độc hại chạy ngầm, được thiết kế để theo dõi và thu thập thông tin cá nhân của người dùng mà không có sự cho phép [5]. Các thông tin bị đánh cắp bao gồm mật khẩu, thông tin tài khoản ngân hàng, lịch sử duyệt web, và các thông tin liên lạc khác.

- Spyware có nhiều dạng hoạt động khác nhau:
- Keylogger: ghi lại mọi thao tác gõ phím của người dùng.
- Screenlogger: chụp và ghi lại hình ảnh trên màn hình thiết bị.

- Clipboard Monitor: thay đổi hoặc theo dõi thông tin lưu trữ trong bộ nhớ tạm.
- Adware: mặc dù mục đích chính của adware là hiển thị quảng cáo không mong muốn, nó cũng thường thu thập dữ liệu người dùng để phục vụ mục đích quảng cáo được nhắm mục tiêu.

Có một sự chồng chéo đáng kể giữa Spyware và Adware, Adware có thể được xem là một "biến thể" của phần mềm quảng cáo. Mặc dù Adware có thể được coi là ít nguy hiểm hơn vì không có tính phá hoại, nó vẫn là một vấn đề nghiêm trọng. Adware thường đi kèm trong các phần mềm miễn phí, và chính hành vi thu thập dữ liệu của nó đã biến nó thành một dạng spyware. Hơn nữa, các quảng cáo pop-up lạ mà Adware hiển thị có thể chứa các mã độc khác, làm cho thiết bị dễ bị lây nhiễm thêm.

f) Rootkit

Rootkit là một bộ công cụ phần mềm độc hại được thiết kế để che giấu sự hiện diện của chính nó và các mã độc khác trong hệ thống, tránh bị phát hiện bởi các chương trình diệt virus truyền thống. Rootkit không có khả năng tự lây lan, nhưng nó là một công cụ cực kỳ nguy hiểm, cho phép kẻ tấn công duy trì quyền truy cập từ xa vào thiết bị.

Rootkit được phân loại dựa trên cấp độ hoạt động:

- Kernel-mode Rootkit: hoạt động ở cấp độ nhân hệ điều hành, cho phép kiểm soát toàn bộ hệ thống và các chức năng cốt lõi.
- User-mode Rootkit: hoạt động ở cấp độ ứng dụng, thay đổi hành vi của các chương trình thông thường như Notepad hoặc Word.
- Bootkit: lây nhiễm vào Master Boot Record (MBR) của ổ cứng, cho phép nó khởi động cùng hệ điều hành, khiến việc phát hiện và gỡ bỏ trở nên cực kỳ khó khăn.
- Firmware Rootkit: lây nhiễm vào firmware của thiết bị như BIOS, có thể chặn quyền truy cập vào dữ liệu được ghi trên ổ cứng.

Đặc điểm nguy hiểm nhất của Rootkit là khả năng chiếm đóng và duy trì sự kiểm soát của nó. Bằng cách cấy vào các lớp sâu nhất của hệ thống, nó có thể sửa đổi các chức năng cốt lõi, can thiệp vào cách các công cụ bảo mật hoạt động và che giấu các tiến trình độc hại khác. Điều này tạo ra một vòng lặp khó khăn: Rootkit ẩn mình, phần mềm diệt virus không thể tìm thấy nó, và do đó nó có thể tồn tại lâu dài để gây ra thiệt

hại đáng kể. Đôi khi, cách duy nhất để loại bỏ một Rootkit tinh vi là cài đặt lại toàn bộ hệ điều hành

1.1.2. Các kỹ thuật né tránh của mã độc

Để duy trì khả năng tồn tại và tránh bị phát hiện, các mã độc hiện đại thường được trang bị nhiều kỹ thuật né tránh (evasion techniques) [6]. Mục tiêu của các kỹ thuật này là che giấu đặc điểm nhận dạng, làm sai lệch kết quả phân tích hoặc ngăn cản các chuyên gia bảo mật quan sát hành vi thực sự của mã độc. Tùy theo đối tượng cần đối phó, các kỹ thuật né tránh có thể được chia thành ba nhóm chính: né tránh phân tích tĩnh, né tránh phân tích động và né tránh các hệ thống phân tích nâng cao.

a) Kỹ thuật né tránh phân tích tĩnh

Phân tích tĩnh là phương pháp kiểm tra mã độc mà không cần thực thi chương trình. Các công cụ phân tích tĩnh thường dựa trên việc tìm kiếm chữ ký, chuỗi ký tự, cấu trúc PE hoặc các mẫu mã độc đã biết. Để vượt qua cơ chế này, mã độc thường thay đổi các đặc điểm cú pháp của tệp thực thi nhưng vẫn giữ nguyên hành vi khi chạy.

Một kỹ thuật phổ biến là làm rối chuỗi (String Obfuscation) [6]. Thay vì lưu trực tiếp các chuỗi quan trọng như địa chỉ máy chủ điều khiển (C&C), khóa mã hóa hoặc thông điệp độc hại, mã độc sẽ mã hóa chúng bằng các thuật toán đơn giản như XOR hoặc Base64. Các chuỗi này chỉ được giải mã trong quá trình thực thi.

Ngoài ra, mã độc có thể sử dụng ký tự rộng (Wide Characters) hoặc ngắt chuỗi (Broken Strings) để tránh các công cụ quét dựa trên chuỗi ký tự [6]. Ví dụ, một chuỗi độc hại có thể được chia thành nhiều đoạn nhỏ hoặc lưu dưới định dạng Unicode thay vì ASCII nhằm khiến các quy tắc nhận dạng không thể phát hiện chính xác.

Một kỹ thuật quan trọng khác là Packing. Trong phương pháp này, mã thực thi được nén hoặc mã hóa và chỉ được giải nén khi chương trình bắt đầu chạy. Các công cụ đóng gói như UPX thường được sử dụng để che giấu mã độc khỏi các phần mềm diệt virus dựa trên chữ ký [6]. Bên cạnh đó, nhiều mã độc còn sử dụng kỹ thuật đa hình (Polymorphism) và biến hình (Metamorphism) để liên tục thay đổi cấu trúc mã nguồn hoặc mã máy giữa các lần lây nhiễm, khiến việc nhận diện bằng chữ ký trở nên khó khăn hơn.

b) Kỹ thuật né tránh phân tích động

Phân tích động là phương pháp quan sát hành vi của mã độc trong quá trình thực thi thông qua sandbox, máy ảo hoặc trình gỡ lỗi. Do đây là phương pháp được sử dụng rộng rãi trong nghiên cứu mã độc, nhiều kỹ thuật đã được phát triển để phát hiện và vô hiệu hóa môi trường phân tích.

Nhóm kỹ thuật đầu tiên là Anti-Debug. Mã độc sẽ kiểm tra xem tiến trình có đang bị theo dõi bởi debugger hay không bằng cách sử dụng các API như `IsDebuggerPresent()` hoặc `CheckRemoteDebuggerPresent()`. Ngoài ra, chúng có thể kiểm tra các trường trong Process Environment Block (PEB), sử dụng các lệnh ngắt như INT 3, INT 2D hoặc lợi dụng cơ chế xử lý ngoại lệ của hệ điều hành để phát hiện sự hiện diện của trình gỡ lỗi [7]. Khi phát hiện bị theo dõi, mã độc thường dừng hoạt động hoặc thực hiện các hành vi giả nhằm đánh lừa nhà phân tích.

Một kỹ thuật khác là Anti-Dump, được sử dụng để ngăn cản việc trích xuất mã độc từ bộ nhớ. Mã độc có thể xóa PE Header hoặc sửa đổi các trường quan trọng trong cấu trúc PE sau khi được nạp vào bộ nhớ nhằm gây khó khăn cho các công cụ điều tra số và phân tích bộ nhớ.

Bên cạnh đó, Code Injection là kỹ thuật cho phép mã độc thực thi bên trong không gian bộ nhớ của một tiến trình hợp pháp. Thay vì chạy trực tiếp, mã độc sẽ tiêm shellcode hoặc DLL độc hại vào các tiến trình đáng tin cậy như `explorer.exe` hoặc `svchost.exe` nhằm che giấu hoạt động khỏi phần mềm bảo mật.

Để né tránh sandbox, mã độc còn thực hiện Resource Profiling và VM Checks [7]. Chúng kiểm tra dung lượng RAM, số lượng CPU, dung lượng ổ đĩa, danh sách tiến trình đang chạy hoặc các dấu vết đặc trưng của môi trường ảo hóa như VirtualBox, VMware hay Hyper-V. Nếu phát hiện môi trường thực thi có dấu hiệu của sandbox hoặc máy ảo, mã độc sẽ không kích hoạt hành vi độc hại.

Ngoài ra, nhiều mẫu mã độc còn sử dụng Timing Attacks [7]. Chúng trì hoãn hoạt động bằng các hàm như `Sleep()` hoặc đo thời gian thực thi bằng lệnh `RDTSC`. Nếu phát hiện sự chậm trễ bất thường do cơ chế giám sát của sandbox gây ra, mã độc sẽ thay đổi hành vi hoặc dừng hoạt động hoàn toàn.

c) Kỹ thuật né tránh các hệ thống phân tích nâng cao

Sự phát triển của các kỹ thuật phân tích tự động như Symbolic Execution và Concolic Execution đã giúp các nhà nghiên cứu khám phá nhiều nhánh thực thi khác nhau của

mã độc mà không cần kích hoạt thủ công [6]. Tuy nhiên, các hệ thống này vẫn tồn tại những hạn chế về tài nguyên tính toán và khả năng xử lý.

Để đối phó, mã độc có thể sử dụng kỹ thuật Symbolic Explosion bằng cách tạo ra số lượng lớn các nhánh điều kiện. Khi đó, hệ thống phân tích phải mô phỏng hàng nghìn hoặc hàng triệu đường thực thi khác nhau, dẫn đến tình trạng cạn kiệt bộ nhớ hoặc thời gian xử lý.

Một kỹ thuật khác là sử dụng Opaque Predicates. Đây là các biểu thức logic có cấu trúc rất phức tạp nhưng kết quả thực tế luôn cố định [6]. Mặc dù chương trình chỉ thực thi một nhánh duy nhất, bộ giải ràng buộc (SMT Solver) vẫn phải tiêu tốn lượng lớn tài nguyên để chứng minh tính đúng đắn của điều kiện đó.

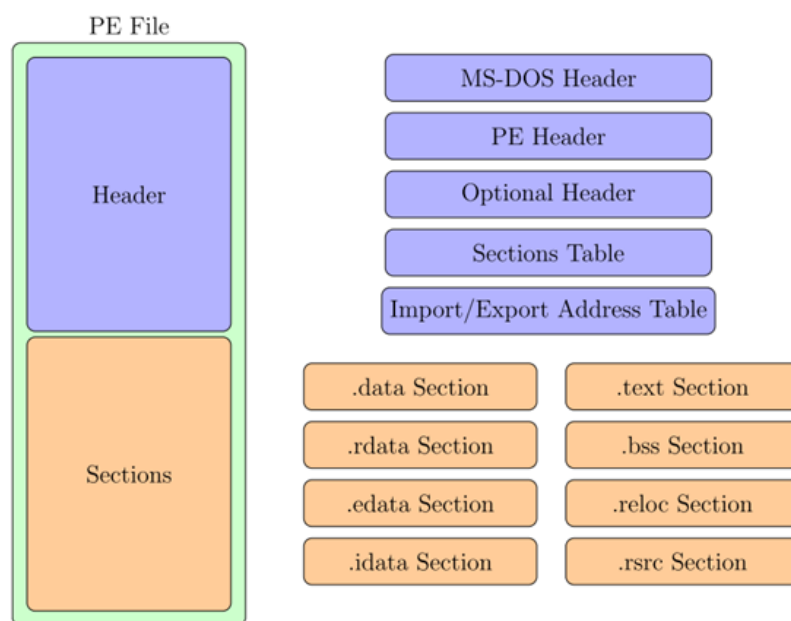
Ngoài ra, một số mã độc áp dụng cơ chế Just-In-Time Compilation (JIT) hoặc tự sửa đổi mã (Self-Modifying Code). Các lệnh thực thi chỉ được tạo ra ngay trước thời điểm chạy, khiến hệ thống phân tích không thể dự đoán hoặc mô phỏng chính xác luồng điều khiển của chương trình. Điều này làm giảm đáng kể hiệu quả của các công cụ phân tích dựa trên thực thi biểu tượng.

Nhìn chung, các kỹ thuật né tránh ngày càng trở nên tinh vi và được thiết kế để đối phó trực tiếp với các công nghệ phân tích hiện đại. Điều này đặt ra yêu cầu phát triển các phương pháp phát hiện tiên tiến hơn, đặc biệt là các giải pháp ứng dụng học máy và học sâu nhằm nâng cao khả năng nhận diện các mẫu mã độc sử dụng kỹ thuật né tránh phức tạp.

1.2. Giới thiệu về tệp Portable Executable (PE)

PE hay Portable Executable là một định dạng chung được sử dụng cho các tệp thực thi, thư viện liên kết động và các đối tượng trên Windows 32-bit và 64-bit [1], định dạng này được thiết kế dựa trên định dạng COFF (Common Object File Format) để đảm bảo tính tương thích giữa các nền tảng Windows khác nhau. PE cho phép hệ thống tải và thực thi mã một cách hiệu quả, bao gồm thông tin về cấu trúc bộ nhớ, địa chỉ tải và các nguyên liên quan. Trong phân tích mã độc, hầu hết phần mềm độc hại trên Windows được đóng gói dưới dạng này, do các đoạn mã độc hại dễ dàng được ẩn dấu trong các phần khác nhau của tệp.

Cấu trúc của tệp PE được tổ chức tuyến tính như một dòng dữ liệu, bắt đầu từ phần MS-DOS Header để đảm bảo tính tương thích với hệ thống cũ, theo sau là PE Header và Section Table.



Hình 1: Cấu trúc tệp PE [1]

a) MS-DOS Header

Mỗi tệp PE đều bắt đầu bằng một chương trình thực thi MS-DOS nhỏ, đây là một chương trình được thực thi thường dùng để thông báo “This program cannot be run in DOS mode.”, mục đích là thông báo lỗi khi chạy trên hệ thống không hỗ trợ PE như MS-DOS cũ. Bắt đầu chương trình là một DOS Header với hai trường quan trọng là `e_magic` và `e_lfanew` trong header và theo sau header là đoạn mã thực thi được gọi là DOS STUB.

Trường `e_magic` còn được gọi là “magic number”, giá trị của trường luôn luôn bằng `0x5A4D` (little-endian), tương ứng với hai ký tự trong ASCII là ‘MZ’, dùng để xác định rằng tệp có định dạng hợp lệ theo chuẩn MS-DOS.

Trường `e_lfanew` là một trường ở cuối DOS header chứa giá trị offset của tệp PE, bản thân trường `e_lfanew` nằm ở vị trí offset `0x3C` của header. Trường đóng vai trò quan trọng đối với Windows trong việc đọc đúng PE Header và thực thi đúng tệp PE.

b) PE Header

PE Header bắt đầu ngay sau DOS STUB, với signature ‘PE\0\0’ tại vị trí do `e_lfanew` chỉ định. Header bao gồm File Header (chứa các trường như Machine,

NumberOfSections, TimeDateStamp) và Optional Header (chứa các trường như SizeOfCode, CheckSum, LoaderFlags) [5].

File Header bao gồm 20 byte tiếp theo, chứa thông tin về sơ đồ bố trí vật lý và các đặc tính chung của tệp. Các trường quan trọng bao gồm:

- Machine: chỉ định kiến trúc CPU mà tệp được biên dịch cho (i386 cho kiến trúc 32-bit).
- NumberOfSections: trường này cho biết số lượng section mà tệp chứa. Giá trị này rất quan trọng trong phân tích mã độc, vì kẻ tấn công thường thay đổi nó để thêm các section ẩn hoặc bất thường.
- Characteristics: một tập hợp các cờ bit xác định các thuộc tính của tệp, chẳng hạn như đây là một chương trình thực thi (.exe) hay một thư viện (.dll)

Mặc dù có tên là "Optional," đây là thành phần bắt buộc đối với các tệp thực thi và chứa các chi tiết quan trọng cho quá trình tải [1]. Các trường đáng chú ý bao gồm:

- Magic: giá trị này xác định liệu tệp là 32-bit hay 64-bit [8].
- AddressOfEntryPoint: chứa RVA của câu lệnh đầu tiên sẽ được thực thi khi chương trình được nạp vào bộ nhớ. Đây là một mục tiêu chính của mã độc, nơi chúng sửa đổi để chuyển hướng luồng thực thi.
- SizeOfImage: kích thước tổng cộng của tệp khi được nạp vào bộ nhớ, được làm tròn đến bội số của SectionAlignment.
- Data Directories (mảng 16 mục, trỏ đến các bảng như Import Table, Export Table), phần này rất quan trọng trong phân tích vì nó tiết lộ các thư viện động mà file phụ thuộc, có thể là dấu hiệu của mã độc.

c) Section Table

Ngay sau Optional Header là Section Table. Đây là một mảng các cấu trúc Section Header, mỗi cấu trúc cung cấp thông tin chi tiết về một section. Các trường quan trọng nhất trong cấu trúc này là: Name, VirtualSize, VirtualAddress, SizeOfRawData, PointerToRawData và Characteristics (các cờ bit xác định đặc tính của section, như khả năng thực thi, ghi, hoặc đọc) [8].

d) Các section phổ biến

Các tệp PE thông thường bao gồm các section với mục đích cụ thể và tên được chuẩn hóa. Một số section phổ biến nhất bao gồm [1]:

- `.text`: chứa mã thực thi của chương trình.
- `.data` và `.rdata`: chứa dữ liệu đã được khởi tạo và dữ liệu chỉ đọc của chương trình.
- `.rsrc`: chứa các tài nguyên được sử dụng bởi ứng dụng, như biểu tượng, hình ảnh hoặc các menu giao diện người dùng.
- `.idata` và `.edata`: chứa thông tin về các hàm được nhập (import) và xuất (export).

Sự linh hoạt của cấu trúc PE thường bị các kẻ tấn công lợi dụng để che giấu và lẫn tránh sự phát hiện của các công cụ bảo mật truyền thống. Một kỹ thuật phổ biến là đóng gói (hay packing), trong đó mã thực thi gốc của mã độc bị nén hoặc mã hóa để làm cho nó không thể phân tích được bằng các phương pháp tĩnh. Một tệp PE đã được đóng gói sẽ có một số đặc điểm bất thường [9]:

- Một hoặc nhiều section mới với tên gọi "rác" hoặc tên bất thường dùng để chứa mã thực thi được nén hoặc mã hóa.
- `AddressOfEntryPoint` của tệp được sửa đổi để trỏ đến một đoạn mã giải nén (stub) nhỏ, thay vì mã thực thi gốc. Đoạn stub này sẽ thực hiện việc giải nén mã độc thực sự vào bộ nhớ khi chương trình chạy.
- Kích thước `SizeOfRawData` sẽ nhỏ hơn nhiều so với kích thước thực trong `VirtualSize` sau khi được giải nén, một dấu hiệu rõ ràng của việc nén.
- Các tệp được đóng gói thường chỉ nhập một vài hàm API cơ bản như `LoadLibrary` hoặc `GetProcAddress`, thay vì toàn bộ danh sách các hàm cần thiết, khiến cho phân tích tĩnh khó phát hiện phần mềm độc hại hơn.

Những đặc điểm bất thường này, từ việc thay đổi các trường trong PE Header đến việc tạo ra các section mới, chính là những đặc trưng tĩnh vô giá có thể được trích xuất và sử dụng để huấn luyện các kỹ thuật học máy, cho phép nhận diện các tệp mã độc đã được che giấu một cách hiệu quả.

1.3. Các phương pháp phân tích

1.3.1. Phân tích tĩnh

Phân tích tĩnh là quá trình phân tích mã độc mà không cần thực thi nó. Phương pháp này an toàn và nhanh chóng, nhưng chỉ cung cấp một cái nhìn ban đầu về cấu trúc và các chức năng tiềm năng của tệp [9].

Quy trình phân tích tĩnh bao gồm:

- Xác định loại tệp và mã băm: bước đầu tiên là xác định loại tệp (như tệp PE, PDF, v.v.) dựa trên chữ ký của nó. Sau đó, mã băm (hash) của tệp (MD5, SHA-256) được tính toán và tra cứu trên các cơ sở dữ liệu trực tuyến như VirusTotal để xem liệu mã độc này đã được cộng đồng phát hiện và phân tích trước đó hay chưa.
- Phân tích chuỗi: kỹ thuật này trích xuất các chuỗi ký tự có thể đọc được từ tệp, bao gồm tên hàm, URL, thông báo lỗi hoặc các khóa registry. Các chuỗi này thường cung cấp manh mối về chức năng của mã độc.
- Phân tích PE Header: các công cụ như PE Studio hoặc CFF Explorer được sử dụng để kiểm tra các trường trong PE Header, tìm kiếm các giá trị bất thường hoặc mâu thuẫn, chẳng hạn như NumberOfSections hoặc SizeOfImage, có thể là dấu hiệu của việc mã độc đã được đóng gói.

Trong quá trình phân tích tĩnh một mẫu tệp nghi ngờ, giả sử mẫu có tên invoice_update.exe, nhà phân tích đầu tiên tiến hành xác định loại tệp và tính mã băm SHA-256. Khi tra cứu trên VirusTotal, kết quả cho thấy mẫu này đã bị 58/70 công cụ phát hiện là mã độc thuộc họ Emotet, gợi ý khả năng phát tán qua email. Tiếp theo, phân tích chuỗi ký tự cho thấy tệp chứa các URL như "http://update-service[.]net/download.dll" và API "WinExec", "URLDownloadToFile", thể hiện hành vi tải và thực thi mã độc từ xa. Cuối cùng, khi phân tích PE Header bằng công cụ PE Studio, phát hiện Import Table bị làm trống và trường NumberOfSections chỉ có một, cho thấy mã độc có khả năng bị đóng gói bằng UPX để che giấu mã gốc.

1.3.2. Phân tích động

Phân tích động là quá trình thực thi mã độc trong một môi trường được kiểm soát để quan sát hành vi thực sự của nó [10]. Kỹ thuật này đặc biệt hữu ích để vượt qua các lớp bảo vệ của mã độc như packing và obfuscation [10].

Công cụ sử dụng trong phân tích bao gồm:

- Môi trường sandbox: đây là một môi trường ảo, cô lập - nơi mã độc được chạy mà không gây hại cho hệ thống thật. Các công cụ sandbox phổ biến bao gồm Cuckoo Sandbox và Joe Sandbox, được thiết kế để mô phỏng một môi trường thực tế, bao gồm cả kết nối mạng.

- Giám sát hệ thống và mạng: khi mã độc chạy trong sandbox, các công cụ giám sát sẽ theo dõi mọi hành động của nó, bao gồm tạo/sửa/xóa tệp, thay đổi registry, và các kết nối mạng. Process Monitor và Wireshark là hai công cụ tiêu biểu, cung cấp dữ liệu chi tiết về tương tác của mã độc với hệ thống và mạng.

Sự kết hợp giữa phân tích tĩnh và phân tích động là một quá trình kết hợp và đem lại hiệu quả cao. Phân tích tĩnh có thể bị đánh lừa bởi các kỹ thuật che giấu như packing, mã hóa chuỗi hoặc thay đổi PE Header. Ngược lại, phân tích động có thể không kích hoạt tất cả các chức năng độc hại nếu không có các điều kiện hoặc môi trường cụ thể. Do đó, các kết quả từ phân tích tĩnh sẽ cung cấp đầu vào quan trọng cho phân tích động, giúp nhà phân tích biết cần phải chú ý đến những hành vi nào khi mã độc được thực thi. Sự phụ thuộc lẫn nhau này làm nổi bật tầm quan trọng của việc thu thập và sử dụng cả hai loại dữ liệu để xây dựng một bức tranh toàn diện về mã độc.

Giả sử một mẫu mã độc có tên `invoice_update.exe` được phát hiện trong hệ thống, nhà phân tích tiến hành thực thi mẫu trong môi trường sandbox như Cuckoo Sandbox để quan sát hành vi. Ngay sau khi khởi chạy, mã độc tạo ra tiến trình con `cmd.exe` và `powershell.exe`, đồng thời ghi dữ liệu vào thư mục tạm `C:\Users\Public\Temp\update.dll`. Công cụ Process Monitor ghi nhận thao tác ghi file và chỉnh sửa khóa registry `HKCU\Software\Microsoft\Windows\CurrentVersion\Run`, cho thấy khả năng tự khởi động cùng hệ thống. Song song đó, Wireshark ghi nhận lưu lượng HTTP bất thường tới địa chỉ `http://malicious-server[.]com/c2`, thể hiện hành vi kết nối đến máy chủ điều khiển C2 (Command & Control).

1.4. Các kỹ thuật phát hiện mã độc

1.4.1. Phát hiện dựa trên chữ ký

Phát hiện dựa trên chữ ký (signature-based detection) là một trong những kỹ thuật truyền thống và phổ biến nhất trong lĩnh vực phòng chống phần mềm độc hại. Cơ chế này hoạt động bằng cách so khớp mẫu mã độc cần kiểm tra với một cơ sở dữ liệu chữ ký đã được định nghĩa trước. Chữ ký có thể là chuỗi byte, mã băm (hash), cấu trúc file, hoặc các mẫu hành vi đặc trưng của mã độc. Với ưu điểm tốc độ xử lý nhanh, tỷ lệ dương tính giả thấp và chi phí tính toán thấp, phát hiện dựa trên chữ ký đã được ứng dụng rộng rãi trong các phần mềm antivirus, IDS/IPS và các hệ thống giám sát an ninh.

Ưu điểm của phương pháp là độ chính xác rất cao và tốc độ nhanh đối với các mã độc đã được xác định. Nhưng bên cạnh đó, cách phát hiện này hoàn toàn vô dụng với các mối đe dọa mới (zero-day) hoặc các mã độc đa hình (polymorphic malware) có khả năng thay đổi chữ ký liên tục để tránh bị phát hiện [11].

Trong thực tế triển khai, phát hiện chữ ký thường bao gồm nhiều dạng khác nhau như so khớp hash toàn file (SHA-256), chuỗi byte (byte pattern), chữ ký cấu trúc (header, section, import), fuzzy hashing (ssdeep, sdhash) và các chỉ báo xâm nhập (IOC) về hành vi như địa chỉ C2, tên miền độc hại. Điển hình là hệ thống ClamAV – một công cụ mã nguồn mở nổi tiếng sử dụng cơ sở dữ liệu hàng triệu chữ ký byte để phát hiện các biến thể mã độc. Tương tự, VirusTotal – nền tảng phân tích mã độc trực tuyến – cũng dựa trên sự kết hợp của hàng chục công cụ chữ ký để nhận diện các mẫu đã biết.

1.4.2. Phát hiện dựa trên hành vi

Phát hiện dựa trên hành vi (behavior-based detection) là kỹ thuật tập trung vào việc quan sát và phân tích các hành động thực tế của chương trình thay vì dựa trên mẫu chữ ký hay đặc trưng tĩnh. Phương pháp này coi mọi tiến trình là tiềm ẩn nguy cơ, chỉ khi phần mềm biểu hiện các hành vi bất thường như tạo hoặc chỉnh sửa tệp hệ thống quan trọng, ghi vào registry nhạy cảm, tiêm mã vào tiến trình khác, tự động duy trì cơ chế khởi động lại, thiết lập kết nối đến máy chủ điều khiển (C2), hoặc thực hiện hành vi leo thang đặc quyền thì mới bị gắn cờ là đáng ngờ.

Trong thực tế, nhiều hệ thống bảo mật hiện đại như Endpoint Detection and Response (EDR), Host-based Intrusion Detection Systems (HIDS) hoặc sandbox phân tích mã độc đều tích hợp cơ chế theo dõi hành vi. Các hệ thống này có thể ghi nhận log hoạt động của tiến trình, xây dựng hồ sơ hành vi (behavioral profile), và đối chiếu với mô hình bất thường (anomaly model) hoặc tập luật định nghĩa trước (rule-based policy) để phát hiện hành vi đáng ngờ.

Ưu điểm của phát hiện hành vi là khả năng phát hiện các biến thể mới, mã độc không file (fileless malware), hoặc những hình thức tấn công zero-day vốn không có chữ ký định trước. Tuy nhiên, nhược điểm của kỹ thuật này là dễ phát sinh dương tính giả do một số phần mềm hợp pháp cũng có hành vi tương tự mã độc, đồng thời việc giám sát liên tục gây tốn tài nguyên và khó triển khai ở môi trường lớn.

1.4.3. Phát hiện dựa trên bộ nhớ

Phát hiện dựa trên bộ nhớ (memory-based detection) là kỹ thuật tập trung vào việc phân tích dữ liệu trong RAM để nhận diện sự hiện diện và hoạt động của phần mềm độc hại. Thay vì chỉ dựa vào file trên đĩa, phương pháp này khai thác các hiện vật sống (live artefacts) như tiến trình, DLL được nạp, vùng nhớ ẩn, hook API, hoặc các đoạn shellcode được tiêm vào bộ nhớ. Ưu điểm quan trọng của kỹ thuật này là khả năng phát hiện các dạng mã độc không để lại dấu vết trên ổ đĩa (fileless malware), các hoạt động injection, rootkit, hay hành vi che giấu tiến trình. Công cụ phổ biến trong phân tích bộ nhớ có thể kể đến như: Volatility, Rekall, MemProcFS, với khả năng trích xuất thông tin chi tiết về tiến trình, kết nối mạng, registry ảo, hay thậm chí reconstruct lại file từ không gian bộ nhớ.

Trong bối cảnh thực tế, phát hiện dựa trên bộ nhớ thường được sử dụng trong điều tra sự cố (incident response) và pháp y số (digital forensics), nơi việc phân tích dump RAM có thể tiết lộ các hoạt động độc hại ngay cả khi kẻ tấn công đã xóa hoặc mã hóa log hệ thống.

Tuy nhiên, kỹ thuật này cũng có những hạn chế như yêu cầu công cụ chuyên dụng, đòi hỏi kiến thức chuyên sâu để phân tích dữ liệu phức tạp, và khó triển khai ở quy mô lớn trong môi trường giám sát thời gian thực. Dù vậy, với sự gia tăng của tấn công fileless và kỹ thuật né tránh hiện đại, phát hiện dựa trên bộ nhớ ngày càng trở thành một thành phần không thể thiếu trong hệ sinh thái phòng thủ, bổ sung hiệu quả cho các phương pháp dựa trên chữ ký, tĩnh và động.

1.4.4. Phát hiện dựa trên lưu lượng mạng

Phát hiện dựa trên lưu lượng mạng (network-based detection) là kỹ thuật tập trung vào việc giám sát, phân tích và phát hiện các hành vi bất thường hoặc dấu hiệu tấn công thông qua dữ liệu truyền trên hệ thống mạng. Thay vì kiểm tra trực tiếp các file hay tiến trình, phương pháp này quan sát các gói tin, kết nối, luồng dữ liệu và mô hình giao tiếp giữa các thực thể nhằm phát hiện hành vi mã độc như kết nối đến máy chủ điều khiển (C2), tải xuống payload độc hại, quét cổng, hoặc truyền dữ liệu bất thường ra ngoài.

Ưu điểm lớn của kỹ thuật này là khả năng phát hiện sớm các hoạt động tấn công trong khi chưa xâm nhập sâu vào hệ thống, đồng thời có thể bao quát trên diện rộng và cung cấp IOC liên quan đến hạ tầng của kẻ tấn công. Tuy nhiên, phát hiện dựa trên lưu

lượng mạng cũng gặp nhiều thách thức, đặc biệt là khi mã độc sử dụng mã hóa TLS/SSL, kỹ thuật ẩn lưu lượng trong giao thức hợp pháp (DNS tunneling, HTTPS), hay tận dụng dịch vụ phổ biến như CDN và cloud để che giấu hoạt động.

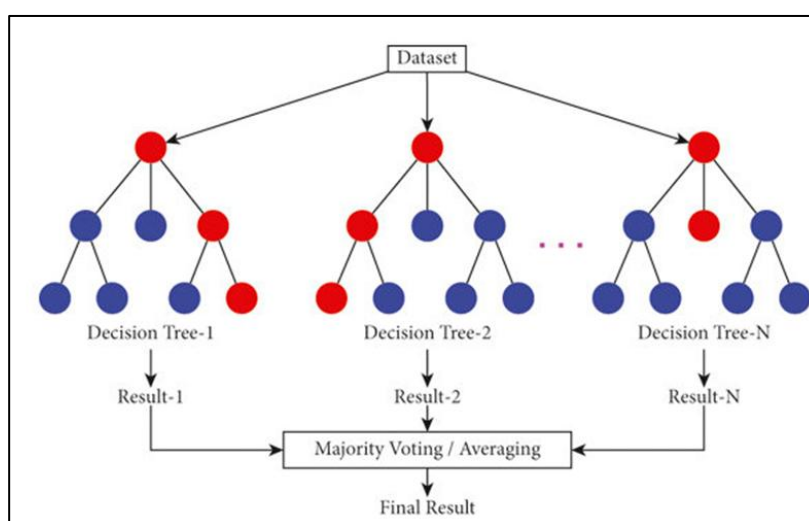
Kỹ thuật này thường được triển khai trong các hệ thống giám sát mạng, firewall thế hệ mới, SIEM, và giải pháp phân tích lưu lượng chuyên sâu (NDR). Dù còn hạn chế bởi vấn đề mã hóa và khối lượng dữ liệu khổng lồ, phát hiện dựa trên lưu lượng mạng vẫn là một tuyến phòng thủ quan trọng, cung cấp góc nhìn toàn cảnh và hỗ trợ mạnh mẽ cho điều tra sự cố cũng như truy vết tấn công.

CHƯƠNG 2: ỨNG DỤNG MACHINE LEARNING TRONG PHÁT HIỆN MÃ ĐỘC

2.1. Machine Learning

2.1.1. Random Forest

Random Forest là một thuật toán học máy thuộc nhóm học có giám sát (Supervised Learning), được sử dụng phổ biến trong các bài toán phân loại và hồi quy. Thuật toán này được đề xuất bởi Leo Breiman và hoạt động dựa trên ý tưởng kết hợp nhiều cây quyết định (Decision Tree) để tạo thành một mô hình mạnh hơn. Thay vì chỉ sử dụng một cây quyết định đơn lẻ, Random Forest xây dựng một “rừng” gồm nhiều cây khác nhau và đưa ra kết quả cuối cùng dựa trên nguyên tắc bỏ phiếu đa số đối với bài toán phân loại hoặc lấy giá trị trung bình đối với bài toán hồi quy [12].



Hình 2: Thuật toán Radom Forest

Về mặt lý thuyết, Random Forest sử dụng kỹ thuật Bagging (Bootstrap Aggregating) nhằm giảm hiện tượng overfitting thường gặp ở cây quyết định. Trong quá trình huấn luyện, thuật toán sẽ tạo ra nhiều tập dữ liệu con bằng phương pháp lấy mẫu ngẫu nhiên có hoàn lại từ tập dữ liệu ban đầu. Mỗi tập dữ liệu con sẽ được sử dụng để huấn luyện một cây quyết định riêng biệt. Đồng thời, tại mỗi nút của cây, thuật toán chỉ chọn ngẫu nhiên một tập con các đặc trưng thay vì sử dụng toàn bộ đặc trưng để tìm điểm phân chia tốt nhất [12]. Điều này giúp tăng tính đa dạng giữa các cây và cải thiện khả năng tổng quát hóa của mô hình.

Giả sử mô hình gồm n cây quyết định và mỗi cây đưa ra một kết quả dự đoán $h_i(x)$. Đối với bài toán phân loại, kết quả cuối cùng của Random Forest được xác định theo nguyên tắc bỏ phiếu đa số:

$$H(x) = \arg \max \sum_{i=1}^n I(h_i(x) = y)$$

Trong đó:

- $H(x)$ là kết quả dự đoán cuối cùng
- $h_i(x)$ là kết quả dự đoán của cây thứ i
- I là hàm chỉ thị
- y là nhãn lớp cần dự đoán

Thuật toán Random Forest hoạt động qua hai giai đoạn chính. Trong giai đoạn huấn luyện, nhiều cây quyết định được xây dựng độc lập dựa trên các tập dữ liệu và tập đặc trưng ngẫu nhiên. Trong giai đoạn dự đoán, dữ liệu đầu vào sẽ được đưa qua tất cả các cây và kết quả cuối cùng được xác định bằng cơ chế voting. Nhờ cơ chế này, Random Forest có khả năng giảm nhiễu và tăng độ ổn định so với việc sử dụng một cây quyết định đơn lẻ.

Trong việc phát hiện phần mềm độc hại, đặc trưng này đặc biệt hữu ích do dữ liệu trích xuất từ tệp thực thi (PE file) thường có tính đa dạng và dễ bị kẻ tấn công biến đổi nhằm qua mặt hệ thống. Cách tiếp cận dựa trên sự đồng thuận của nhiều cây quyết định giúp giảm thiểu rủi ro sai sót từ những biến đổi nhỏ, qua đó nâng cao độ tin cậy trong phát hiện như các công trình:

Bảng 1: Các nghiên cứu sử dụng Random Forest trong phát hiện mã độc

Tác giả	Tiền xử lý dữ liệu	Cấu hình sử dụng	Bộ dữ liệu	Kết quả đạt được
Felan Carlo C. Garcia [13]	Nhóm các chuỗi nhị phân của malware thành vector 8-bit, chuyển đổi chúng thành ma trận hình ảnh ($n \times n$, với $n=32$) và kéo phẳng thành mảng có chiều dài 1024 để	Lập trình trên ngôn ngữ R sử dụng các thư viện randomForest và caret [13].	Bộ dữ liệu Malimg	Accuracy ~ 95.26%

	làm đầu vào cho mô hình phân loại.	Đánh giá mô hình trên tập huấn luyện bằng phương pháp 10-fold Cross-Validation		
S. Abijah Roseline [14]	Chuyển đổi mã độc thành các mảng 2D và trực quan hóa dưới dạng ảnh xám (grayscale images). Sau đó, ảnh được xử lý qua hai giai đoạn: quét cửa sổ trượt (sliding window scanning) và phân lớp xếp tầng (cascade layering).	Mixed forest model kết hợp các thuật toán: Random Forest (RF), Completely Random Forests (CRF) và XGBoost (XGB Forest).	Bộ dữ liệu Maling, Microsoft BIG 2015, MaleVis, Malicia	98,65% đối với tập Maling, 97,2% đối với BIG 2015
Suyeon Yoo [15]	Trích xuất cả đặc trưng tĩnh và đặc trưng động của tệp, sau đó chấm điểm thông qua 4 mô hình phân loại: <i>Rfstatic</i> , <i>Rfdynamic</i> , <i>MLPstatic</i> và <i>MLPdynamic</i> .	Kỹ thuật bagging với 100 vòng lặp và cấu hình mặc định.	Bộ dữ liệu VirusChaser và KISA	Accuracy ~ 85.1% trên bộ dữ liệu KISA

Bên cạnh những ưu điểm, Random Forest cũng tồn tại một số hạn chế. Khi số lượng cây lớn, mô hình có thể tiêu tốn nhiều bộ nhớ và thời gian xử lý. Ngoài ra, kết quả dự đoán của Random Forest khó giải thích hơn so với cây quyết định đơn lẻ do mô hình được tạo thành từ nhiều cây khác nhau. Tuy nhiên, nhờ độ chính xác cao và khả năng chống overfitting tốt, Random Forest vẫn là một trong những thuật toán học máy hiệu quả và được ứng dụng phổ biến trong các nghiên cứu về phát hiện và phân loại mã độc.

2.1.2. Naive Bayes

Naive Bayes là một thuật toán học máy thuộc nhóm học có giám sát (Supervised Learning), được sử dụng phổ biến trong các bài toán phân loại. Thuật toán này được xây dựng dựa trên định lý Bayes trong xác suất thống kê và hoạt động theo nguyên lý dự đoán xác suất một mẫu dữ liệu thuộc về một lớp cụ thể dựa trên các đặc trưng đầu vào [16]. Điểm đặc trưng của Naive Bayes là giả định rằng các đặc trưng là độc lập với nhau khi đã biết lớp kết quả, vì vậy thuật toán được gọi là “Naive” (ngây thơ). Mặc dù giả định này khá đơn giản và không hoàn toàn đúng trong thực tế, Naive Bayes vẫn đạt hiệu quả tốt trong nhiều bài toán thực tiễn nhờ khả năng xử lý dữ liệu nhanh và ổn định. Về mặt lý thuyết, Naive Bayes dựa trên định lý Bayes trong xác suất:

$$P(C|X) = \frac{P(X|C)P(C)}{P(X)}$$

Trong đó:

- $P(C|X)$: xác suất mẫu dữ liệu X thuộc lớp C
- $P(X|C)$: xác suất xuất hiện dữ liệu X khi biết trước lớp C
- $P(C)$: xác suất tiên nghiệm của lớp C
- $P(X)$: xác suất xuất hiện của dữ liệu X

Trong bài toán phân loại, giá trị $P(X)$ là giống nhau đối với mọi lớp nên có thể bỏ qua khi so sánh. Thuật toán sẽ lựa chọn lớp có xác suất hậu nghiệm lớn nhất:

$$C = \arg \max P(X|C) P(C)$$

Do giả định các đặc trưng độc lập với nhau, xác suất $P(X|C)$ được tính bằng tích xác suất của từng đặc trưng riêng lẻ:

$$P(X|C) = \prod_{i=1}^n P(x_i|C)$$

Từ đó, công thức phân loại của Naive Bayes trở thành:

$$C = \arg \max P(C) \prod_{i=1}^n P(x_i|C)$$

Thuật toán Naive Bayes hoạt động qua hai giai đoạn chính là huấn luyện và dự đoán. Trong giai đoạn huấn luyện, mô hình sẽ tính toán xác suất tiên nghiệm của từng lớp và xác suất xuất hiện của các đặc trưng trong từng lớp dựa trên dữ liệu huấn luyện. Sang giai đoạn dự đoán, khi nhận một mẫu dữ liệu mới, thuật toán sẽ tính xác suất hậu nghiệm cho từng lớp và chọn lớp có xác suất lớn nhất làm kết quả phân loại.

Trong lĩnh vực phát hiện và phân loại mã độc, Naive Bayes thường được sử dụng để xử lý các đặc trưng được trích xuất từ file PE, opcode, API call hoặc byte sequence. Ví dụ, tần suất xuất hiện của các opcode như mov, push, call hoặc các byte n-gram có thể được sử dụng làm đầu vào cho mô hình. Với ưu điểm tốc độ huấn luyện nhanh, khả năng xử lý dữ liệu nhiều chiều tốt và yêu cầu tài nguyên thấp, Naive Bayes phù hợp với các hệ thống cần xử lý lượng lớn mẫu malware. Các công trình có thể kể đến như:

Bảng 2: Các nghiên cứu sử dụng Navie Bayes trong phát hiện mã độc

Tác giả	Tiền xử lý dữ liệu	Cấu hình sử dụng	Bộ dữ liệu	Kết quả đạt được
Marzieh Esnaashari [16]	Với đặc trưng phân loại: Nếu dưới 5 danh mục thì dùng One-Hot Encoding; từ 5-20 danh mục thì dùng Factorizing; trên 20 danh mục thì mã hóa theo Tần suất kết hợp chuẩn hóa Min-Max. Dữ liệu cuối cùng gồm 97 đặc trưng đã được mã hóa	Sử dụng mô hình Gaussian Naive Bayes, giả định rằng phân phối xác suất (likelihood) của các đặc trưng tuân theo phân phối chuẩn (Gaussian) và các đặc trưng là độc lập có điều kiện.	Microsoft Malware Prediction	Accuracy ~ 57.52%
Usukhbayar Baldangombo [17]	Sử dụng thuật toán Information Gain và PCA để chọn lọc, biến đổi và giảm chiều dữ liệu của các đặc trưng thô được trích xuất từ tệp (như thông tin PE header, danh sách DLL và các hàm API) trước khi đưa vào huấn luyện	Thuật toán Naive Bayes (NB) được lấy từ bộ công cụ học máy mã nguồn mở Weka dựa trên ngôn ngữ Java. Mô hình được huấn luyện và kiểm thử bằng phương pháp 10-fold cross-validation	Tác giả đã xây dựng và sử dụng một bộ dữ liệu quy mô lớn gồm tổng cộng 247.348 tệp chương trình Windows định dạng PE	Accuracy ~ 96%
Manish Goyal [18]	Phân tích hành vi động của tệp bằng cách trích xuất API calls.	Áp dụng 2 biến thể là: Gaussian Naive Bayes và Multi Naive Bayes.	Bộ dữ liệu 42.797 mẫu mã độc và chỉ có 1.079 mẫu phần mềm sạch	Accuracy trên Gaussian Naive Bayes ~ 73.72%

Tuy nhiên, thuật toán này cũng tồn tại hạn chế do giả định độc lập giữa các đặc trưng thường không phản ánh chính xác mối quan hệ thực tế trong dữ liệu mã độc. Trong malware, nhiều đặc trưng có liên quan chặt chẽ với nhau, chẳng hạn chuỗi opcode hoặc hành vi API call. Điều này có thể làm giảm độ chính xác của mô hình so với các phương pháp học sâu hiện đại. Mặc dù vậy, Naive Bayes vẫn được xem là một phương pháp nền tảng quan trọng nhờ tính đơn giản, dễ triển khai và thường được sử dụng làm mô hình cơ sở để so sánh trong các nghiên cứu về phân loại mã độc.

2.1.3. Support vector machine

Support Vector Machine (SVM) là một thuật toán học máy có giám sát được sử dụng phổ biến trong các bài toán phân loại và hồi quy. Thuật toán này được phát triển dựa trên ý tưởng tìm ra một siêu phẳng (hyperplane) tối ưu nhằm phân tách dữ liệu của các lớp khác nhau trong không gian đặc trưng. Mục tiêu của SVM là tìm siêu phẳng có khoảng cách lớn nhất đến các điểm dữ liệu gần nhất của mỗi lớp, từ đó giúp tăng khả năng phân loại chính xác và cải thiện khả năng tổng quát hóa của mô hình.

Về mặt lý thuyết, giả sử tập dữ liệu huấn luyện gồm các điểm dữ liệu (x_i, y_i) , trong đó x_i là vector đặc trưng và $y_i \in -1, 1$ là nhãn lớp [19]. SVM tìm siêu phẳng có dạng:

$$w \cdot x + b = 0$$

Trong đó:

- w là vector trọng số xác định hướng của siêu phẳng
- b là hệ số bias
- x là vector đặc trưng của dữ liệu

Khoảng cách từ các điểm dữ liệu gần nhất đến siêu phẳng được gọi là margin. SVM hướng tới việc tối đa hóa margin này nhằm giảm nguy cơ phân loại sai. Điều kiện phân loại dữ liệu được biểu diễn như sau [19]:

$$y_i(w \cdot x_i + b) \geq 1$$

Bài toán tối ưu của SVM là tìm giá trị w và b sao cho margin đạt giá trị lớn nhất, tương đương với việc tối thiểu hóa:

$$\min \frac{1}{2} ||w||^2$$

Trong thực tế, dữ liệu thường không thể phân tách tuyến tính hoàn toàn. Để giải quyết vấn đề này, SVM sử dụng kỹ thuật Kernel nhằm ánh xạ dữ liệu sang không gian chiều cao hơn, nơi các lớp có thể được phân tách dễ dàng hơn. Một số hàm Kernel phổ biến bao gồm:

- Linear Kernel
- Polynomial Kernel
- Radial Basis Function (RBF)
- Sigmoid Kernel

Trong đó, RBF Kernel được sử dụng rộng rãi nhờ khả năng xử lý tốt dữ liệu phi tuyến. Hàm RBF được biểu diễn như sau:

$$K(x_i, x_j) = \exp\left(-\gamma \|x_i - x_j\|^2\right)$$

Thuật toán SVM hoạt động thông qua việc xác định các điểm dữ liệu quan trọng nhất gọi là Support Vectors. Đây là các điểm nằm gần biên phân tách và có ảnh hưởng trực tiếp đến vị trí của siêu phẳng. Nhờ chỉ phụ thuộc vào một số điểm dữ liệu quan trọng, SVM thường đạt hiệu quả cao trên các tập dữ liệu có số chiều lớn.

Trong lĩnh vực phát hiện và phân loại mã độc, SVM được ứng dụng rộng rãi nhờ khả năng xử lý tốt dữ liệu nhiều chiều và dữ liệu không tuyến tính. Các đặc trưng như byte n-gram, opcode frequency, API call hoặc TF-IDF từ file assembly có thể được sử dụng làm đầu vào cho mô hình. SVM đặc biệt phù hợp với các bài toán phân loại malware khi số lượng đặc trưng lớn hơn rất nhiều so với số lượng mẫu dữ liệu.

Tuy nhiên, SVM cũng tồn tại một số hạn chế như thời gian huấn luyện cao trên tập dữ liệu lớn và việc lựa chọn Kernel phù hợp có ảnh hưởng đáng kể đến hiệu năng mô hình. Ngoài ra, thuật toán này cũng khó mở rộng đối với các hệ thống dữ liệu cực lớn so với các mô hình học sâu hiện đại. Mặc dù vậy, nhờ khả năng phân loại mạnh mẽ và độ chính xác cao, SVM vẫn là một trong những thuật toán nền tảng quan trọng trong nghiên cứu phát hiện và phân loại mã độc. Các nghiên cứu có thể kể đến như:

Bảng 3: Các nghiên cứu sử dụng SVM trong phát hiện mã độc

Tác giả	Tiền xử lý dữ liệu	Cấu hình sử dụng	Bộ dữ liệu	Kết quả đạt được
Abhijit Yewale [20]	Mã độc được giải nén bằng công cụ UPX Unpacker và dịch ngược (disassemble) bằng phần mềm IDA pro. Trích xuất đặc trưng chọn ra tần suất xuất hiện của 20 Opcode phổ biến nhất để tạo thành các véc-tơ đặc trưng đầu vào cho mô hình	Tham số mặc định	100 mẫu mã độc được tải xuống từ một trang web chia sẻ mã độc và các phần mềm an toàn lấy từ thư mục system32 của hệ điều hành Windows	Accuracy ~ 93%

Michał Kruczkowski [19]	Mỗi mẫu được gán bốn thuộc tính: thời gian (time), định dạng (format), tên miền (domain) và địa chỉ IP (address). Sau đó, các giá trị này được thu phóng về khoảng 0 - 1	Sử dụng mô hình phân loại tuyến tính và phi tuyến với hàm nhân đa thức (polynomial kernel). Các tham số được thiết lập là $\gamma=0.5$, hằng số $r=1$, và bậc $d=3$	Bộ dữ liệu nhỏ gọn bao gồm tổng cộng 398 mẫu	Accuracy ~ 93.98%
W. K. WONG [21]	SVM kết hợp với kỹ thuật mã hóa sửa lỗi đầu ra ECOC (Error Correction Output Coding) làm lớp phân loại cuối cùng.	Sử dụng mạng tập hợp Error Correction Output Coding kết hợp với thuật toán học máy SVM (ECOC-SVM) cho nhiệm vụ phân loại đa lớp	Malimg, MaleVis, Virus-MNIST, Dumpware10	Accuracy cao nhất trên Malimg đạt 99.14%

2.2. Deep Learning

2.2.1. ResNet50

ResNet50 là một mô hình học sâu thuộc họ mạng nơ-ron tích chập (Convolutional Neural Network – CNN), được phát triển bởi Microsoft Research và giới thiệu trong cuộc thi ImageNet năm 2015 [22]. ResNet50 là phiên bản gồm 50 lớp của kiến trúc Residual Network (ResNet), được thiết kế nhằm giải quyết hiện tượng suy giảm hiệu năng (degradation problem) khi số lượng lớp của mạng ngày càng sâu. Đây là một trong những kiến trúc CNN nổi bật và được sử dụng rộng rãi trong các bài toán phân loại ảnh, nhận dạng đối tượng và phân tích dữ liệu hình ảnh.

Điểm nổi bật của ResNet50 là việc sử dụng cơ chế Residual Learning thông qua các kết nối tắt (skip connection hoặc shortcut connection) [22]. Trong các mạng CNN truyền thống, khi số lớp tăng lên quá lớn, mô hình thường gặp khó khăn trong quá trình lan truyền gradient, dẫn đến hiện tượng vanishing gradient và làm giảm độ chính xác. ResNet khắc phục vấn đề này bằng cách cho phép dữ liệu đầu vào của một lớp được truyền trực tiếp tới các lớp phía sau thông qua kết nối tắt. Thay vì học trực tiếp hàm ánh xạ mong muốn $H(x)$, mô hình sẽ học phần dư (residual) $F(x)$:

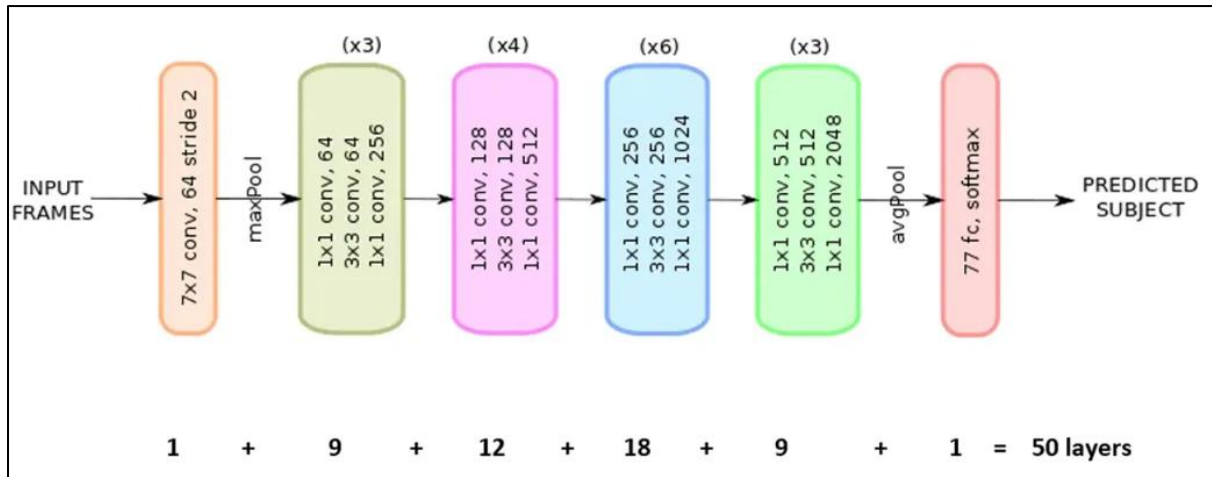
$$H(x) = F(x) + x$$

Trong đó:

- x là đầu vào của khối residual
- $F(x)$ là hàm residual được học bởi các lớp convolution

- $H(x)$ là đầu ra cuối cùng của khối

Cơ chế này giúp gradient được lan truyền hiệu quả hơn trong quá trình huấn luyện, từ đó cho phép xây dựng các mạng rất sâu mà vẫn duy trì hiệu năng cao. Kiến trúc ResNet50 gồm nhiều residual block được xếp chồng lên nhau, kết hợp với các lớp convolution, batch normalization, ReLU activation và pooling để trích xuất đặc trưng từ dữ liệu ảnh.



Hình 3: Mô hình ResNet-50 [20]

ResNet50 bao gồm tổng cộng 50 lớp học được, được tổ chức thành nhiều residual block theo cấu trúc bottleneck [23]. Mỗi bottleneck block bao gồm ba lớp convolution liên tiếp:

- Lớp convolution 1×1 dùng để giảm số chiều đặc trưng
- Lớp convolution 3×3 dùng để trích xuất đặc trưng không gian
- Lớp convolution 1×1 dùng để khôi phục chiều dữ liệu

Kiến trúc bottleneck giúp giảm số lượng tham số và chi phí tính toán mà vẫn duy trì khả năng học biểu diễn mạnh mẽ. Sau mỗi lớp convolution, mô hình sử dụng Batch Normalization kết hợp với hàm kích hoạt ReLU nhằm tăng tốc độ hội tụ và cải thiện khả năng ổn định trong quá trình huấn luyện. Bên cạnh đó, các lớp pooling được sử dụng để giảm kích thước đặc trưng và tăng khả năng trích xuất thông tin ở mức trừu tượng cao hơn.

Trong lĩnh vực phát hiện và phân loại mã độc, ResNet50 thường được ứng dụng thông qua việc chuyển đổi dữ liệu malware sang dạng hình ảnh grayscale từ chuỗi byte của file thực thi. Khi đó, mô hình CNN có khả năng học được các mẫu cấu trúc trực

quan đặc trưng của từng họ malware. Các lớp convolution trong ResNet50 giúp trích xuất đặc trưng cục bộ như texture và pattern, trong khi độ sâu lớn của mạng cho phép học các biểu diễn trừu tượng hơn liên quan đến cấu trúc tổng thể của mã độc. Điều này giúp ResNet50 đạt hiệu quả cao trong các bài toán phân loại malware dựa trên hình ảnh. Một ưu điểm quan trọng của ResNet50 là khả năng tận dụng kỹ thuật transfer learning. Mô hình có thể sử dụng trọng số được huấn luyện trước trên các tập dữ liệu lớn như ImageNet, sau đó fine-tuning trên dữ liệu malware để cải thiện hiệu năng và giảm thời gian huấn luyện. Ngoài ra, ResNet50 có khả năng tổng quát hóa tốt, độ chính xác cao và hoạt động hiệu quả trên các bài toán phân loại đa lớp với dữ liệu phức tạp.

Tuy nhiên, ResNet50 cũng tồn tại một số hạn chế nhất định. Do có độ sâu lớn và số lượng tham số cao, mô hình yêu cầu tài nguyên tính toán đáng kể, đặc biệt khi huấn luyện trên tập dữ liệu lớn. Bên cạnh đó, mặc dù CNN có khả năng trích xuất đặc trưng cục bộ rất tốt, kiến trúc này vẫn gặp hạn chế trong việc mô hình hóa quan hệ toàn cục giữa các vùng dữ liệu so với các kiến trúc hiện đại như Vision Transformer.

Bảng 4: Các nghiên cứu sử dụng ResNet-50 trong phát hiện mã độc

Tác giả	Tiền xử lý dữ liệu	Cấu hình sử dụng	Bộ dữ liệu	Kết quả đạt được
Sanjeev Kumar [24]	Các chương trình thực thi mã nhị phân thô được chuyển đổi thành một ma trận 2D gồm các số thập phân. Ma trận này sau đó được biến đổi thành các hình ảnh thang độ xám và chuyển về kích thước 224x224x3	Loại bỏ các lớp top layers của mô hình ResNet-50 và làm cho lớp fully connected cuối cùng non-trainable. Thêm một lớp fully connected dense layer gồm 128 nơ-ron để giảm chiều feature map xuống kích thước N x 128 (với N là số lượng file mã độc)	MalImg, Microsoft BIG 2015	Trên tập MalImg accuracy đạt 98,50%. Microsoft BIG 2015 đạt 94.85%
Riaz Ullah Khan [25]	Các tệp thực thi (.exe) được giải nén, dịch ngược thành các chuỗi mã lệnh và sau đó được chuyển đổi, tái tạo thành các gray-scale images.	Triển khai mô hình ResNet-50 thông qua thư viện TensorFlow	Microsoft Malware Classification Challenge 2015	Accuracy ~ 80.95%
Rajasekhar Chaganti [23]	Biểu diễn ở dạng hình ảnh thông qua 3 phương pháp khác nhau để đối chiếu: M1	Sử dụng trọng số pre-trained trên ImageNet thông qua học chuyển	Microsoft Malware Classification	Accuracy ~ 85%

	(hình ảnh có chiều rộng cố định 512 byte), M2 (chiều rộng hình ảnh dựa trên kích thước của tệp), và M3 (chiều rộng được tính dựa trên độ dài chuỗi byte nén).	giao. Lớp cuối cùng của mô hình ResNet-50 được gỡ bỏ. Batch size: 64, Learning rate: 0.01, Optimizer: Adam, Epochs: 10. Tổng số tham số của mô hình là 23.606.153	Challenge 2015	
--	---	---	----------------	--

2.2.2. Vision Transformers

Vision Transformer (ViT) là một kiến trúc học sâu được phát triển dựa trên mô hình Transformer trong lĩnh vực xử lý ngôn ngữ tự nhiên (Natural Language Processing – NLP). Mô hình này được đề xuất bởi Google Research vào năm 2020 trong công trình “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale” [26]. Khác với các mạng CNN truyền thống sử dụng phép tích chập để trích xuất đặc trưng cục bộ, Vision Transformer áp dụng cơ chế self-attention để học mối quan hệ toàn cục giữa các vùng khác nhau trong ảnh. Điều này giúp mô hình có khả năng biểu diễn dữ liệu linh hoạt và hiệu quả hơn trong nhiều bài toán thị giác máy tính.

Ý tưởng cốt lõi của Vision Transformer là xem ảnh như một chuỗi các “token” tương tự như từ trong NLP. Đầu tiên, ảnh đầu vào được chia thành nhiều patch kích thước cố định, ví dụ 16×16 . Mỗi patch sau đó được làm phẳng và ánh xạ sang vector đặc trưng thông qua phép embedding [26]. Giả sử ảnh đầu vào có kích thước $H \times W \times C$, số lượng patch được tạo ra là:

$$N = \frac{H \times W}{P^2}$$

Trong đó:

- H, W là chiều cao và chiều rộng của ảnh
- C là số kênh màu
- P là kích thước của mỗi patch
- N là số lượng patch

Sau khi embedding, các patch sẽ được bổ sung thông tin vị trí (Positional Encoding) để mô hình có thể nhận biết thứ tự và vị trí không gian của từng patch trong ảnh. Chuỗi vector đặc trưng này sau đó được đưa vào kiến trúc Transformer Encoder.

Thành phần quan trọng nhất của Vision Transformer là cơ chế Multi-Head Self-Attention. Cơ chế này cho phép mô hình học mối quan hệ giữa mọi patch trong ảnh thay

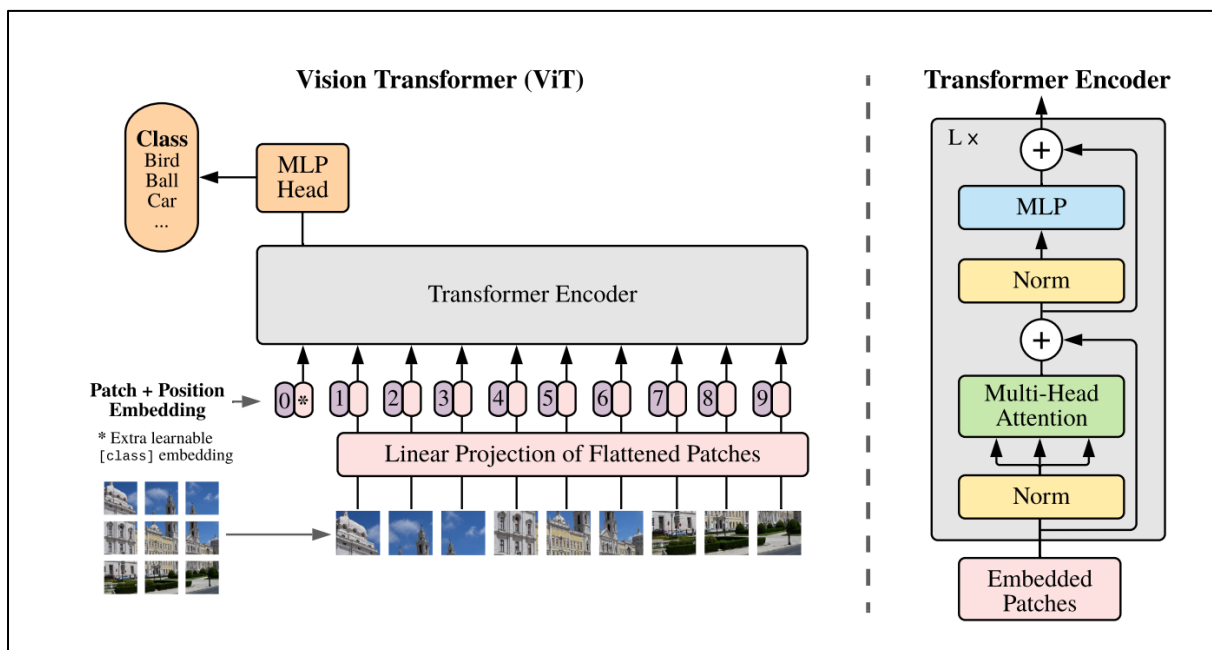
vì chỉ tập trung vào vùng lân cận như CNN. Attention được tính toán dựa trên ba ma trận Query (Q), Key (K) và Value (V):

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Trong đó:

- Q : ma trận truy vấn (Query)
- K : ma trận khóa (Key)
- V : ma trận giá trị (Value)
- d_k : số chiều của vector Key

Thông qua cơ chế attention, Vision Transformer có khả năng xác định mức độ liên quan giữa các vùng khác nhau trong ảnh, từ đó học được các đặc trưng toàn cục hiệu quả hơn. Multi-Head Attention tiếp tục mở rộng khả năng này bằng cách cho phép mô hình học nhiều mối quan hệ khác nhau đồng thời.



Hình 4: Kiến trúc Vision Transformer [26]

Kiến trúc Vision Transformer bao gồm nhiều Transformer Encoder block được xếp chồng lên nhau. Mỗi block thường gồm:

- Multi-Head Self-Attention
- Layer Normalization
- Multi-Layer Perceptron (MLP)

- Residual Connection

Residual connection giúp giảm hiện tượng mất mát gradient và cải thiện quá trình huấn luyện mạng sâu. Đồng thời, Layer Normalization giúp ổn định phân phối dữ liệu trong quá trình học.

Ưu điểm quan trọng của Vision Transformer là khả năng mở rộng tốt trên các tập dữ liệu lớn và khả năng học biểu diễn toàn cục mạnh mẽ. Ngoài ra, ViT có thể kết hợp hiệu quả với transfer learning để cải thiện hiệu năng trong các bài toán chuyên biệt như phân loại malware. Tuy nhiên, mô hình cũng tồn tại một số hạn chế như yêu cầu lượng dữ liệu lớn để huấn luyện hiệu quả và chi phí tính toán cao hơn so với CNN truyền thống.

Bảng 5: Các nghiên cứu sử dụng Vision Transformer trong phát hiện mã độc

Tác giả	Tiền xử lý dữ liệu	Cấu hình sử dụng	Bộ dữ liệu	Kết quả đạt được
Sachith Seneviratne [27]	Trích xuất tệp bytecode (DEX) từ tệp APK, chuyển đổi thành mảng 1D 8-bit rồi đưa về ảnh thang độ xám 2D và mã hóa thông tin ngữ nghĩa bằng cách gán màu sắc (RGB) cho từng byte dựa trên vị trí của nó trong cấu trúc tệp DEX.	ViT-Base (ViT-B/16) với 86 triệu tham số. Mô hình tiền huấn luyện bằng kỹ thuật Masked Auto Encoder (tự ý che đi 75% dữ liệu ảnh và yêu cầu tái tạo lại)	MalNet	Accuracy ~ 97%
In-Woong Jeong [28]	Mã nhị phân được nhóm thành các bộ 8 bit, chuyển đổi thành vector số nguyên và được bố trí vào mảng 2D với chiều rộng phụ thuộc kích thước tệp ban đầu.	Sử dụng Cross-Scale Embedding Layer để trích xuất đặc trưng từ các patch ảnh có kích thước khác nhau, giúp mô hình chống lại việc obfuscation. Áp dụng Long-Short Distance Attention để nắm bắt đồng thời các đặc trưng chi tiết cục bộ và cấu trúc toàn cục	Microsoft Malware Classification Challenge, Maling, MalNet-Tiny, MaleVis	Trên tập Microsoft đạt accuracy ~ 97.33%
Shrey Bavishi [29]	Chuyển đổi các file nhị phân PE thành ảnh RGB. Các byte được đọc theo nhóm	Giai đoạn 1: Sử dụng pre-trained trên ImageNet để phân loại nhị phân ban	MaleVis	Accuracy ~ 96,6%

	3 byte và sắp xếp vào không gian vector 2 chiều	đầu nhằm tách biệt ảnh bình thường và ảnh chứa mã độc. Giai đoạn 2: Những ảnh được phân loại là mã độc sẽ đi qua mô hình LeViT-256 để phân loại chi tiết họ mã độc.		
--	---	--	--	--

2.3. Ensemble Learning

Ensemble Learning là một phương pháp học máy trong đó nhiều mô hình học được kết hợp nhằm tạo ra một mô hình mạnh hơn và có độ chính xác cao hơn so với từng mô hình riêng lẻ [30]. Ý tưởng cơ bản của phương pháp này xuất phát từ việc mỗi mô hình có những ưu điểm và hạn chế riêng; do đó, việc kết hợp nhiều mô hình sẽ giúp giảm sai số, tăng khả năng tổng quát hóa và cải thiện tính ổn định trong quá trình dự đoán. Ensemble Learning được ứng dụng rộng rãi trong các bài toán phân loại, hồi quy và đặc biệt hiệu quả trong các lĩnh vực có dữ liệu phức tạp như an ninh mạng và phát hiện mã độc.

Về mặt lý thuyết, Ensemble Learning hoạt động dựa trên nguyên tắc kết hợp kết quả từ nhiều mô hình cơ sở (base learners) để đưa ra dự đoán cuối cùng. Các mô hình cơ sở có thể là các thuật toán giống nhau hoặc khác nhau, ví dụ như Decision Tree, Random Forest, SVM, Naive Bayes hoặc các mô hình học sâu như CNN và Vision Transformer. Đối với bài toán phân loại, kết quả cuối cùng thường được xác định thông qua cơ chế bỏ phiếu đa số (majority voting):

$$H(x) = \arg \max \sum_{i=1}^n h_i(x)$$

Trong đó:

- $H(x)$ là kết quả dự đoán cuối cùng
- $h_i(x)$ là kết quả dự đoán của mô hình thứ i
- n là số lượng mô hình trong ensemble

Trong một số trường hợp, mỗi mô hình có thể được gán trọng số khác nhau dựa trên độ chính xác hoặc mức độ tin cậy. Khi đó, phương pháp Weighted Voting được sử dụng:

$$H(x) = \arg \max \sum_{i=1}^n w_i h_i(x)$$

Trong đó:

- w_i là trọng số của mô hình thứ i

Có nhiều kỹ thuật Ensemble Learning khác nhau, trong đó phổ biến nhất là Bagging, Boosting và Stacking:

- Bagging (Bootstrap Aggregating): Phương pháp này huấn luyện nhiều mô hình độc lập trên các tập dữ liệu con được lấy mẫu ngẫu nhiên từ dữ liệu ban đầu. Kết quả cuối cùng được tổng hợp bằng voting hoặc averaging. Random Forest là một ví dụ điển hình của Bagging.
- Boosting: Boosting xây dựng các mô hình tuần tự, trong đó mỗi mô hình mới tập trung vào các mẫu dữ liệu mà mô hình trước dự đoán sai. Các thuật toán phổ biến thuộc nhóm này bao gồm AdaBoost và XGBoost.
- Stacking: Đây là phương pháp kết hợp nhiều mô hình cơ sở khác nhau, sau đó sử dụng một mô hình meta-learner để học cách kết hợp kết quả từ các mô hình đó nhằm đưa ra dự đoán cuối cùng.

Trong lĩnh vực phát hiện và phân loại mã độc, Ensemble Learning được sử dụng nhằm tận dụng ưu điểm của nhiều mô hình khác nhau. Các mô hình học máy truyền thống như SVM hoặc Random Forest có khả năng xử lý tốt dữ liệu đặc trưng thống kê, trong khi các mô hình học sâu như ResNet50 hoặc Vision Transformer lại mạnh trong việc học biểu diễn từ dữ liệu hình ảnh malware. Việc kết hợp các mô hình này giúp cải thiện khả năng nhận diện malware, giảm hiện tượng overfitting và nâng cao độ ổn định của hệ thống. Ngoài ra, Ensemble Learning còn đặc biệt hữu ích đối với dữ liệu malware do dữ liệu thường có tính đa dạng cao, không cân bằng giữa các lớp và chứa nhiều nhiễu. Các mô hình đơn lẻ có thể hoạt động tốt trên một số nhóm malware nhất định nhưng kém hiệu quả với các nhóm khác. Ensemble Learning giúp cân bằng các điểm mạnh và điểm yếu này, từ đó nâng cao hiệu suất tổng thể. Một số nghiên cứu có thể kể đến như:

Bảng 6: Các nghiên cứu sử dụng Ensemble Learning trong phát hiện mã độc

Tác giả	Tiền xử lý dữ liệu	Cấu hình sử dụng	Bộ dữ liệu	Kết quả đạt được
Jiaqi Chen [31]	Phân tích trực tiếp các tệp tin EXE bằng công cụ LIEF để trích xuất 7 nhóm đặc trưng cấu trúc: biểu đồ byte, biểu đồ entropy-byte, thông tin chuỗi,... Các đặc trưng liên tục (như biểu đồ byte) được giữ nguyên, trong khi các đặc trưng rời rạc được vector hóa bằng kỹ thuật "feature hashing"	Dùng phương pháp Stacking, gồm 7 mô hình cơ sở (sử dụng Multilayer Perceptron - MLP cho đặc trưng liên tục và Random Forest - RF cho đặc trưng rời rạc)	Nhóm nghiên cứu xây dựng (3.041 mẫu phần mềm an toàn, 9.726 mẫu mã độc và 3.032 mẫu đối kháng)	Accuracy cao nhất ~ 97.11%
Badis Hammi [32]	Các chuỗi gọi API (API call sequences) được trích xuất động cẩn thận từ các phần tử 'calls' có trong các báo cáo của công cụ Cuckoo Sandbox.	Đánh giá cấu trúc Voting Classifier kết hợp dự đoán từ các mô hình cơ sở (gồm Logistic Regression, Random Forest, Naive Bayes, và KNN)	Bộ dữ liệu do De Oliveira và các cộng sự cung cấp	Accuracy ~ 98.4%
Yunan Zhang [33]	Để xử lý sự mất cân bằng, hệ thống áp dụng kỹ thuật Undersampling và Oversampling. Kết hợp đặc trưng thông qua thuật toán "forward stepwise selection".	Lớp 1: Chạy mô hình XGBoost song song trên từng nhóm đặc trưng độc lập hoặc kết hợp Lớp 2: Lấy đầu ra của Lớp 1 làm đầu vào, sử dụng 2 mô hình học tập tập hợp là XGBoost và ExtraTreeClassifier kết hợp cùng trọng số khác nhau để ra quyết định cuối	Microsoft Malware Classification Challenge	Accuracy ~ 99.72%

Tuy nhiên, phương pháp Ensemble Learning cũng tồn tại một số hạn chế như chi phí tính toán cao, thời gian huấn luyện lâu và độ phức tạp trong quá trình triển khai. Khi số lượng mô hình tăng lên, yêu cầu về tài nguyên bộ nhớ và năng lực xử lý cũng tăng đáng kể. Bên cạnh đó, việc lựa chọn và tối ưu cách kết hợp giữa các mô hình là một thách thức quan trọng ảnh hưởng trực tiếp đến hiệu quả của hệ thống.

CHƯƠNG 3: THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG PHÁT HIỆN VÀ PHÂN LOẠI MÃ ĐỘC ỨNG DỤNG VISION TRANSFORMER VÀ ENSEMBLE LEARNING

3.1. Bộ dữ liệu Microsoft Malware Classification Challenge Dataset 2015 (BIG 2015)

3.1.1. Tổng quan bộ dữ liệu

Bộ dữ liệu Microsoft Malware Classification Challenge Dataset (2015) là một trong những nguồn dữ liệu tiêu biểu và được sử dụng rộng rãi trong nghiên cứu về phân loại mã độc bằng học máy [24]. Dataset này được công bố bởi Microsoft trong khuôn khổ một cuộc thi trên nền tảng Kaggle, với mục tiêu thúc đẩy việc phát triển các phương pháp tự động nhận diện và phân loại malware dựa trên đặc trưng tĩnh. Sự ra đời của bộ dữ liệu này xuất phát từ nhu cầu cấp thiết trong bối cảnh số lượng mã độc gia tăng nhanh chóng, khiến các phương pháp phân tích thủ công trở nên không còn hiệu quả.

Về cấu trúc, mỗi mẫu trong dataset được biểu diễn dưới dạng hai loại tệp chính là .bytes và .asm. Tệp .bytes chứa dữ liệu hex dump của file thực thi (PE), cho phép nghiên cứu các đặc trưng ở mức byte như byte n-gram hoặc entropy. Trong khi đó, tệp .asm là kết quả của quá trình disassembly, cung cấp mã assembly bao gồm các lệnh (opcode), lời gọi API và thông tin về các section trong chương trình. Hai dạng dữ liệu này bổ sung cho nhau, giúp người nghiên cứu có thể tiếp cận malware ở nhiều mức độ trừu tượng khác nhau, từ thấp (byte-level) đến cao hơn (instruction-level).

Dataset bao gồm khoảng 10.000 mẫu mã độc, được phân thành 9 họ khác nhau như Ramnit, Lollipop, Vundo hay Gatak [33]. Đây là một bài toán phân loại đa lớp (multi-class classification), trong đó mục tiêu là xây dựng mô hình có khả năng xác định chính xác họ của một mẫu malware dựa trên các đặc trưng đầu vào. Tuy nhiên, dữ liệu trong dataset không hoàn toàn cân bằng giữa các lớp, do đó đặt ra thêm thách thức trong quá trình huấn luyện mô hình, yêu cầu áp dụng các kỹ thuật xử lý như resampling hoặc điều chỉnh trọng số lớp.

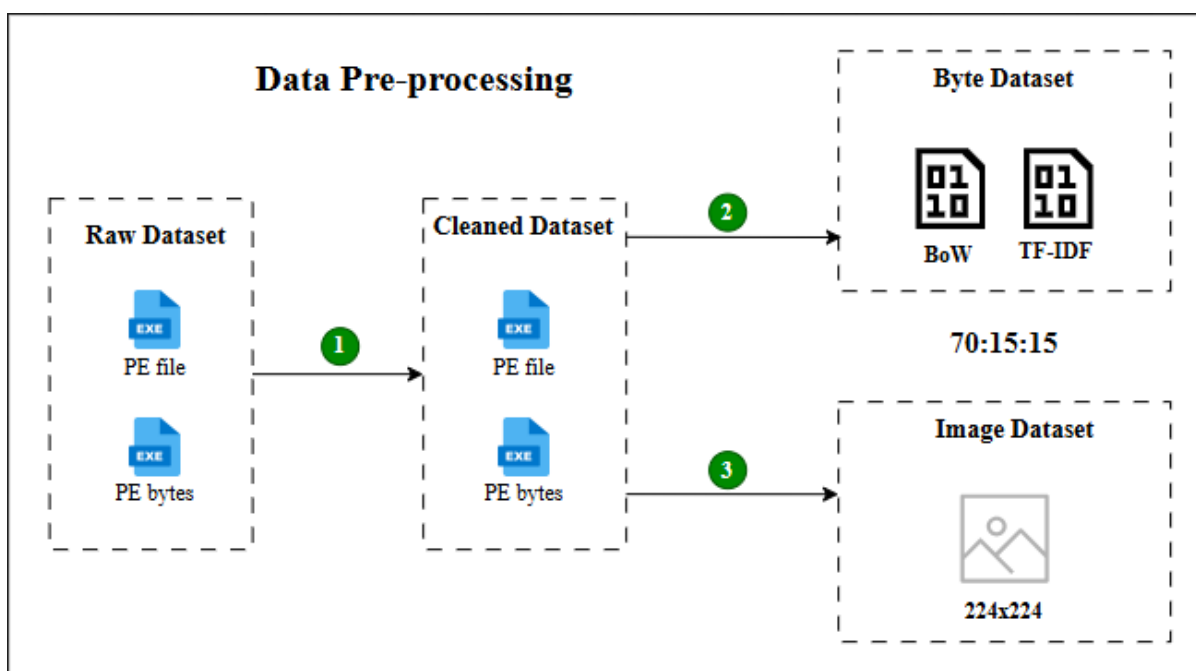
Một điểm nổi bật của bộ dữ liệu này là nó hoàn toàn dựa trên phân tích tĩnh, nghĩa là không cần thực thi mã độc để thu thập dữ liệu. Điều này giúp đảm bảo an toàn trong quá trình nghiên cứu, đồng thời phù hợp với các bài toán reverse engineering và trích xuất đặc trưng từ file PE. Ngoài ra, với kích thước lớn (lên đến hàng trăm GB), dataset

cũng tạo điều kiện cho việc nghiên cứu các phương pháp xử lý dữ liệu lớn và tối ưu hóa hiệu năng.

Tuy nhiên, bên cạnh những ưu điểm, bộ dữ liệu cũng tồn tại một số hạn chế. Do được công bố từ năm 2015, nó có thể không phản ánh đầy đủ các xu hướng malware hiện đại, đặc biệt là các kỹ thuật obfuscation và evasion mới. Hơn nữa, việc chỉ sử dụng dữ liệu tĩnh khiến dataset không thể biểu diễn được hành vi thực thi của mã độc, vốn là yếu tố quan trọng trong nhiều hệ thống phát hiện tiên tiến hiện nay.

3.1.2. Tiền xử lý dữ liệu

Để nâng cao hiệu quả của các mô hình học máy và học sâu trong bài toán phát hiện, phân loại mã độc, dữ liệu đầu vào cần được tiền xử lý và chuẩn hóa trước khi đưa vào huấn luyện. Quá trình tiền xử lý không chỉ giúp loại bỏ dữ liệu nhiễu và dữ liệu không hợp lệ mà còn hỗ trợ chuyển đổi dữ liệu mã độc sang các dạng biểu diễn phù hợp với từng mô hình khác nhau. Trong nghiên cứu này, dữ liệu được tiền xử lý theo hai hướng chính gồm biểu diễn dạng vector đặc trưng cho các mô hình học máy truyền thống và biểu diễn dạng hình ảnh cho các mô hình học sâu như ResNet50 và Vision Transformer. Quy trình tiền xử lý được xây dựng dựa trên mã nguồn thực nghiệm được triển khai trên bộ dữ liệu Microsoft Malware Classification Challenge 2015.



Hình 5: Quy trình tiền xử lý dữ liệu

Trước tiên ở bước (1), chương trình đọc tệp nhãn trainLabels.csv để xác định mã định danh và lớp tương ứng của từng mẫu mã độc. Các lớp mã độc được ánh xạ thành 9 họ khác nhau, bao gồm Ramnit, Lollipop, Kelihos_ver3, Vundo, Simda, Tracur, Kelihos_ver1, Obfuscator.ACY và Gatak. Sau đó, các tệp dữ liệu được giải nén theo từng lô từ file nén train.7z nhằm giảm tải bộ nhớ và tránh việc phải giải nén toàn bộ dữ liệu cùng lúc. Cách xử lý theo lô này phù hợp với đặc điểm của bộ dữ liệu có dung lượng lớn. Tiếp đến là làm sạch dữ liệu, chương trình loại bỏ một số tệp .bytes rỗng hoặc không chứa thông tin byte hợp lệ để tránh gây lỗi trong quá trình trích xuất đặc trưng và huấn luyện mô hình. Sau đó, nội dung của từng tệp .bytes được đọc dưới dạng chuỗi byte. Các địa chỉ bộ nhớ ở đầu dòng được loại bỏ bằng biểu thức chính quy, đồng thời các ký tự xuống dòng, ký tự không hợp lệ và dấu ? cũng được xóa bỏ. Kết quả của bước này là chuỗi mã hex sạch, chỉ giữ lại các giá trị byte cần thiết để phục vụ cho quá trình biểu diễn dữ liệu và trích xuất đặc trưng.

Tiếp theo ở bước (2), dữ liệu được trích xuất đặc trưng theo hướng Bag-of-Words trên byte. Chương trình xây dựng một bộ từ vựng gồm 256 giá trị byte từ 00 đến ff. Với mỗi mẫu mã độc, tần suất xuất hiện của từng byte được thống kê bằng CountVectorizer, tạo thành vector đặc trưng gồm kích thước tệp và 256 đặc trưng byte.

Bên cạnh biểu diễn đặc trưng dạng vector, dữ liệu mã độc còn được chuyển đổi sang dạng hình ảnh ở bước (3) để phục vụ các mô hình học sâu như ResNet50 và Vision Transformer. Cụ thể, chuỗi hex sau khi làm sạch được chuyển thành mảng số nguyên 8-bit, trong đó mỗi byte có giá trị từ 0 đến 255 tương ứng với cường độ điểm ảnh grayscale. Do độ dài của các tệp không giống nhau, chương trình tính kích thước ảnh vuông dựa trên căn bậc hai của số byte, sau đó thêm giá trị 0 vào cuối chuỗi nếu cần để tạo thành ma trận vuông. Ảnh grayscale thu được tiếp tục được resize về kích thước cố định 224×224 bằng phương pháp nội suy bilinear và lưu dưới định dạng .jpeg.

Cuối cùng, dữ liệu được chia thành ba tập huấn luyện, kiểm định và kiểm tra bằng phương pháp StratifiedShuffleSplit. Việc chia dữ liệu có phân tầng giúp đảm bảo tỷ lệ các lớp mã độc trong từng tập con gần tương đồng với phân bố ban đầu, hạn chế sai lệch do mất cân bằng dữ liệu. Theo quy trình trong mã nguồn, dữ liệu được chia thành 70% cho tập huấn luyện, 15% cho tập kiểm định và 15% cho tập kiểm tra. Các ảnh sau

khi tạo được lưu theo cấu trúc thư mục riêng cho từng tập và từng lớp mã độc, giúp thuận tiện cho quá trình huấn luyện, đánh giá và so sánh các mô hình.

3.2. Thiết kế mô hình

3.2.1. Thiết kế các mô hình Machine Learning

a) Random Forest

Trong nghiên cứu này, mô hình Random Forest được xây dựng nhằm đánh giá khả năng phân loại các họ mã độc trên bộ dữ liệu Microsoft Malware Classification Challenge 2015. Để nâng cao hiệu quả phân loại của mô hình Random Forest, nghiên cứu tiến hành tối ưu hóa siêu tham số bằng phương pháp Randomized Search Cross Validation. Quá trình tìm kiếm được thực hiện trên tập huấn luyện với 3-fold Cross Validation và sử dụng chỉ số F1-Score Macro làm tiêu chí đánh giá. Chỉ số này được lựa chọn nhằm đánh giá đồng đều hiệu năng trên tất cả các lớp mã độc, hạn chế ảnh hưởng của sự mất cân bằng dữ liệu giữa các lớp.

Bảng 7: Bảng thống kê kết quả của các bộ siêu tham số

#	mean_f1_macro	std_f1_macro	n_estimators	min samples split	min samples leaf	max_leaf_nodes	max_depth
1	0.313407	0.151254	400	10	2	300	None
2	0.333652	0.167677	300	2	1	300	None
3	0.32804	0.161853	200	2	2	100	None
4	0.323536	0.160565	100	5	2	100	20
5	0.316906	0.154398	100	10	2	300	None
6	0.316339	0.150556	400	2	2	100	20
7	0.316212	0.152392	400	10	1	200	None
8	0.315647	0.155972	100	2	4	200	None
9	0.314738	0.151448	400	5	4	300	None
10	0.334454	0.168642	400	2	2	300	20
11	0.309133	0.147194	300	5	4	300	20
12	0.295646	0.134489	100	10	1	300	10
13	0.28913	0.130898	400	5	4	300	10
14	0.286367	0.130033	200	2	4	150	10
15	0.285437	0.129276	200	10	1	150	10

Để tìm kiếm cấu hình phù hợp cho mô hình Random Forest, nghiên cứu sử dụng phương pháp Randomized Search kết hợp với Cross Validation và chỉ số đánh giá F1-Score Macro. Các siêu tham số được khảo sát bao gồm số lượng cây quyết định (n_estimators), số lượng mẫu tối thiểu để phân chia nút (min_samples_split), số lượng mẫu tối thiểu tại nút lá (min_samples_leaf), số lượng nút lá tối đa (max_leaf_nodes) và

độ sâu tối đa của cây (max_depth). Kết quả thực nghiệm cho thấy hiệu năng của mô hình chịu ảnh hưởng đáng kể bởi các tham số này. Trong số 15 cấu hình được đánh giá, bộ tham số gồm 400 cây quyết định, min_samples_split = 2, min_samples_leaf = 2, max_leaf_nodes = 300 và max_depth = 20 đạt kết quả tốt nhất với giá trị F1-Score Macro trung bình là 0,334454 và độ lệch chuẩn 0,168642. Các cấu hình có hiệu năng cao cũng tập trung chủ yếu ở số lượng cây từ 200 đến 400 cây và độ sâu cây lớn (20 hoặc không giới hạn), cho thấy việc tăng số lượng cây và cho phép cây học được nhiều đặc trưng hơn giúp cải thiện khả năng phân loại. Ngược lại, các cấu hình có độ sâu giới hạn ở mức 10 cho kết quả thấp hơn đáng kể, với F1-Score Macro chỉ dao động từ 0,285 đến 0,296.

Bảng 8: Kết quả của Random Forest

Metric Class	Precision	Recall	F1-Score	Support
Ramnit	97.05%	100.00%	98.50%	230
Lollipop	99.73%	100.00%	99.87%	371
Kelihos_ver3	100.00%	99.77%	99.89%	442
Vundo	97.26%	98.61%	97.93%	72
Simda	100.00%	83.33%	90.91%	6
Tracur	93.91%	96.43%	95.15%	112
Kelihos_ver1	100.00%	100.00%	100.00%	60
Obfuscator.ACY	97.71%	92.94%	95.27%	184
Gatak	99.34%	98.68%	99.01%	152
---	---	---	---	---
Accuracy			98.65%	1629
Macro Avg	98.33%	96.64%	97.39%	1629
Weighted Avg	98.66%	98.65%	98.64%	1629

Kết quả thực nghiệm cho thấy mô hình Random Forest đạt hiệu năng phân loại rất cao trên tập kiểm thử của bộ dữ liệu Microsoft Malware Classification Challenge 2015 với độ chính xác đạt 98,65%, F1-Score Macro đạt 97,39% và F1-Score Weighted đạt 98,64%. Mô hình nhận diện gần như hoàn hảo các họ mã độc Kelihos_ver1, Kelihos_ver3 và Lollipop với F1-Score lần lượt đạt 100,00%, 99,89% và 99,87%. Bên cạnh đó, các lớp Ramnit, Gatak, Vundo, Tracur và Obfuscator.ACY cũng đạt kết quả

rất khả quan với F1-Score đều trên 95%. Riêng lớp Simda đạt F1-Score thấp nhất là 90,91% do số lượng mẫu trong tập kiểm thử chỉ gồm 6 mẫu, dẫn đến kết quả đánh giá dễ bị ảnh hưởng bởi một số ít trường hợp phân loại sai. Giá trị F1-Score Macro cao cho thấy mô hình duy trì khả năng phân loại ổn định trên tất cả các lớp dữ liệu, bao gồm cả các lớp có số lượng mẫu nhỏ, đồng thời sự chênh lệch không đáng kể giữa F1-Score Macro và F1-Score Weighted cho thấy mô hình ít bị ảnh hưởng bởi hiện tượng mất cân bằng dữ liệu. Nhìn chung, kết quả đạt được chứng minh Random Forest là một phương pháp hiệu quả trong bài toán phân loại mã độc dựa trên đặc trưng byte-level, với khả năng nhận diện chính xác và tổng quát hóa tốt trên hầu hết các họ mã độc trong bộ dữ liệu nghiên cứu.

b) Support Vector Machine (SVM)

Trong nghiên cứu này, mô hình Support Vector Machine (SVM) được sử dụng để phân loại các họ mã độc dựa trên các đặc trưng được trích xuất từ dữ liệu dưới dạng Bag-of-Words (BoW). Dữ liệu sau khi tiền xử lý được chia thành tập huấn luyện, tập xác thực và tập kiểm thử để phục vụ cho quá trình xây dựng và đánh giá mô hình.

Để tìm kiếm cấu hình phù hợp, nghiên cứu tiến hành tối ưu siêu tham số bằng phương pháp Randomized Search kết hợp với Cross Validation. Các tham số được khảo sát bao gồm hệ số phạt C , tham số gamma, cơ chế shrinking và khả năng sinh xác suất dự đoán (probability). Chỉ số F1-Score Macro được sử dụng làm tiêu chí đánh giá nhằm đảm bảo hiệu năng phân loại đồng đều giữa các lớp mã độc.

Sau quá trình tối ưu hóa, mô hình SVM được lựa chọn với tham số $C = 50$. Tham số gamma được sử dụng ở chế độ scale, cho phép tự động điều chỉnh theo số lượng đặc trưng của dữ liệu. Đồng thời, tùy chọn probability được kích hoạt nhằm hỗ trợ tính toán xác suất dự đoán cho từng lớp mã độc, phục vụ quá trình đánh giá bằng các chỉ số như Log Loss và Confusion Matrix.

Bảng 9: Kết quả của Support Vector Machine

Metric Class	Precision	Recall	F1-Score	Support
Ramnit	84.50%	94.78%	89.34%	230
Lollipop	98.05%	94.88%	96.44%	371
Kelihos_ver3	99.55%	100.00%	99.77%	442
Vundo	79.55%	97.22%	87.50%	72
Simda	0.00%	0.00%	0.00%	6
Tracur	87.13%	78.57%	82.63%	112
Kelihos_ver1	90.48%	95.00%	92.68%	60
Obfuscator.ACY	93.14%	88.59%	90.81%	184
Gatak	95.75%	88.82%	92.15%	152
---	---	---	---	---
Accuracy			93.62%	1629
Macro Avg	80.90%	81.98%	81.26%	1629
Weighted Avg	93.57%	93.62%	93.47%	1629

Kết quả thực nghiệm cho thấy mô hình SVM đạt độ chính xác 93,62% trên tập kiểm thử, với F1-Score Macro đạt 81,26% và F1-Score Weighted đạt 93,47%. Mô hình thể hiện khả năng phân loại tốt đối với một số họ mã độc có số lượng mẫu lớn như Kelihos_ver3, Lollipop và Gatak, với F1-Score lần lượt đạt 99,77%, 96,44% và 92,15%. Bên cạnh đó, các lớp Ramnit, Vundo, Kelihos_ver1 và Obfuscator.ACY cũng đạt kết quả tương đối cao với F1-Score trên 87%. Tuy nhiên, hiệu năng của mô hình không đồng đều giữa các lớp dữ liệu. Đáng chú ý, lớp Simda không được nhận diện chính xác mẫu nào, dẫn đến Precision, Recall và F1-Score đều bằng 0%. Nguyên nhân chủ yếu là do số lượng mẫu của lớp này rất nhỏ trong tập dữ liệu, khiến mô hình khó học được các đặc trưng đại diện. Sự chênh lệch lớn giữa F1-Score Macro (81,26%) và F1-Score Weighted (93,47%) cho thấy hiệu năng của mô hình bị ảnh hưởng bởi hiện tượng mất cân bằng dữ liệu giữa các lớp. Mặc dù SVM đạt độ chính xác tương đối cao, kết quả thu được vẫn thấp hơn đáng kể so với mô hình Random Forest, đặc biệt ở khả năng nhận diện các lớp có số lượng mẫu ít. Điều này cho thấy SVM phù hợp với các lớp có số lượng dữ liệu lớn và ranh giới phân lớp rõ ràng, nhưng còn hạn chế trong việc xử lý các lớp thiểu số trong bài toán phân loại mã độc đa lớp.

c) Naive Bayes Multinomial

Quá trình xây dựng mô hình được thực hiện bằng thư viện Scikit-learn với lớp MultinomialNB. Khác với Random Forest và SVM, mô hình không yêu cầu quá trình tối ưu hóa siêu tham số phức tạp do số lượng tham số cần điều chỉnh tương đối ít. Dữ liệu sau khi tiền xử lý được chia thành tập huấn luyện, tập xác thực và tập kiểm thử, sau đó được sử dụng trực tiếp để huấn luyện mô hình. Trong quá trình thực nghiệm, Multinomial Naive Bayes sử dụng các giá trị đặc trưng dưới dạng tần suất xuất hiện byte hoặc trọng số TF-IDF để ước lượng xác suất thuộc về từng họ mã độc.

Bảng 10: Kết quả của Naive Bayes Multinomial

Class \ Metric	Precision	Recall	F1-Score	Support
Ramnit	58.76%	65.65%	62.01%	230
Lollipop	100.00%	60.92%	75.71%	371
Kelihos_ver3	89.11%	100.00%	94.24%	442
Vundo	57.14%	61.11%	59.06%	72
Simda	5.97%	66.67%	10.96%	6
Tracur	44.66%	41.07%	42.79%	112
Kelihos_ver1	66.67%	13.33%	22.22%	60
Obfuscator.ACY	83.33%	84.24%	83.78%	184
Gatak	56.10%	75.66%	64.43%	152
---	---	---	---	---
Accuracy			73.11%	1629
Macro Avg	62.42%	63.18%	57.25%	1629
Weighted Avg	77.97%	73.11%	73.46%	1629

Kết quả thực nghiệm cho thấy mô hình Multinomial Naive Bayes đạt độ chính xác 73,11% trên tập kiểm thử, với F1-Score Macro đạt 57,25% và F1-Score Weighted đạt 73,46%. So với Random Forest và SVM, hiệu năng của mô hình thấp hơn đáng kể trên hầu hết các chỉ số đánh giá. Mô hình đạt kết quả tốt đối với một số họ mã độc như Kelihos_ver3 và Obfuscator.ACY với F1-Score lần lượt đạt 94,24% và 83,78%, đồng thời lớp Lollipop đạt Precision 100,00% nhưng Recall chỉ đạt 60,92%, cho thấy mô hình có xu hướng dự đoán thận trọng đối với lớp này. Đối với các lớp Ramnit, Vundo, Tracur, Kelihos_ver1 và Gatak, giá trị F1-Score chỉ dao động từ 22,22% đến 64,43%,

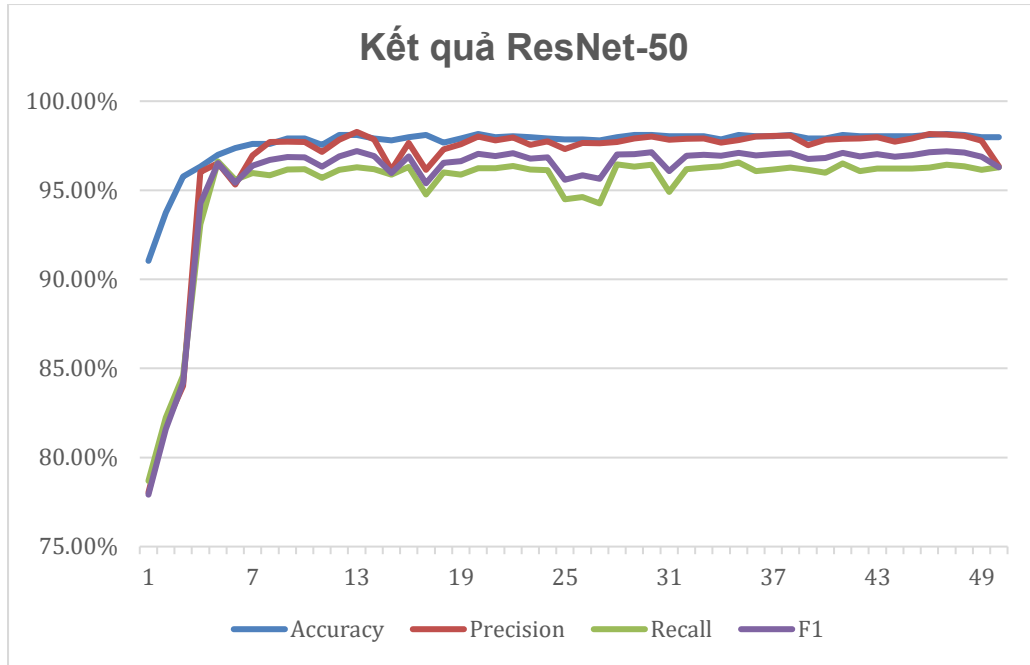
phản ánh khả năng phân biệt giữa các họ mã độc còn hạn chế. Đặc biệt, lớp Kelihos_ver1 chỉ đạt F1-Score 22,22% do Recall rất thấp (13,33%), trong khi lớp Simda mặc dù đạt Recall 66,67% nhưng Precision chỉ đạt 5,97%, dẫn đến F1-Score chỉ đạt 10,96%. Sự chênh lệch lớn giữa F1-Score Macro (57,25%) và F1-Score Weighted (73,46%) cho thấy mô hình bị ảnh hưởng đáng kể bởi hiện tượng mất cân bằng dữ liệu giữa các lớp. Kết quả này cho thấy Multinomial Naive Bayes có ưu điểm về tốc độ huấn luyện và độ đơn giản trong triển khai, tuy nhiên khả năng mô hình hóa mối quan hệ phức tạp giữa các đặc trưng còn hạn chế, dẫn đến hiệu năng phân loại thấp hơn đáng kể so với Random Forest và SVM trong bài toán phân loại mã độc dựa trên đặc trưng byte-level.

3.2.2. Thiết kế các mô hình Deep Learning

a) ResNet-50

Mô hình ResNet-50 được sử dụng để phân loại các họ mã độc sau khi chuyển đổi dữ liệu sang dạng ảnh. Thay vì huấn luyện mô hình từ đầu, nghiên cứu áp dụng phương pháp Transfer Learning bằng cách sử dụng mô hình ResNet-50 đã được huấn luyện trước trên tập dữ liệu ImageNet thông qua thư viện Transformers. Dữ liệu đầu vào được lưu trữ dưới dạng ảnh và được xử lý bằng bộ tiền xử lý (AutoImageProcessor) tương ứng với mô hình ResNet-50. Do dữ liệu ban đầu được biểu diễn dưới dạng ảnh xám, tất cả ảnh được chuyển đổi sang định dạng RGB gồm ba kênh màu để phù hợp với kiến trúc của mô hình. Sau đó, các ảnh được chuẩn hóa và chuyển đổi thành tensor trước khi đưa vào quá trình huấn luyện.

Để phù hợp với bài toán phân loại 9 họ mã độc, lớp phân loại cuối cùng của mô hình được thay thế bằng một lớp Fully Connected mới có số lượng đầu ra tương ứng với số lớp cần phân loại. Quá trình huấn luyện được thực hiện trên tập huấn luyện và đánh giá trên tập xác thực bằng thư viện Hugging Face Trainer. Các tham số huấn luyện được thiết lập gồm kích thước batch bằng 16, tốc độ học (learning rate) bằng 2×10^{-4} và số epoch tối đa là 50. Trong quá trình huấn luyện, mô hình được đánh giá sau mỗi epoch và tự động lưu lại trọng số tốt nhất dựa trên chỉ số F1-Score Macro của tập xác thực.



Hình 6: Kết quả của mô hình ResNet-50 sau 50 epoch

Trong quá trình huấn luyện 50 epoch, mô hình ResNet-50 cho thấy khả năng hội tụ nhanh và ổn định. Chỉ sau 5 epoch đầu tiên, độ chính xác trên tập xác thực đã đạt 96,99% cùng F1-Score 96,52%. Từ epoch 10 trở đi, các chỉ số đánh giá gần như ổn định với Accuracy dao động quanh mức 98% và F1-Score Macro đạt khoảng 97%. Đồng thời, giá trị Training Loss giảm mạnh từ 1,2608 xuống dưới 0,01 ở các epoch cuối, cho thấy mô hình đã học được các đặc trưng quan trọng của dữ liệu. Kết thúc quá trình huấn luyện, mô hình đạt Accuracy cao nhất là 98,16% và F1-Score Macro đạt 97,18% trên tập xác thực, chứng tỏ ResNet-50 có khả năng học và tổng quát hóa tốt đối với bài toán phân loại mã độc dựa trên biểu diễn ảnh.

Bảng 11: Kết quả cuối cùng của ResNet-50

Metric Class	Precision	Recall	F1-Score	Support
Ramnit	97.87%	100.00%	98.93%	230
Lollipop	98.65%	98.65%	98.65%	371
Kelihos_ver3	100.00%	100.00%	100.00%	442
Vundo	94.74%	100.00%	97.30%	72
Simda	71.43%	83.33%	76.92%	6
Tracur	98.23%	99.11%	98.67%	112
Kelihos_ver1	100.00%	98.33%	99.16%	60

Obfuscator.ACY	98.29%	93.48%	95.82%	184
Gatak	98.04%	98.68%	98.36%	152
---	---	---	---	---
Accuracy			98.59%	1629
Macro Avg	95.25%	96.92%	95.98%	1629
Weighted Avg	98.62%	98.59%	98.59%	1629

Kết quả thực nghiệm cho thấy mô hình ResNet-50 đạt hiệu năng rất cao trong bài toán phân loại mã độc với độ chính xác (Accuracy) đạt 98,59%, F1-Score Macro đạt 95,98% và F1-Score Weighted đạt 98,59%. Sau khoảng 1 giờ 10 phút huấn luyện, mô hình đã học được các đặc trưng trực quan hiệu quả từ dữ liệu ảnh được chuyển đổi từ tệp mã độc. Kết quả phân loại cho thấy hầu hết các họ mã độc đều được nhận diện với độ chính xác rất cao. Đặc biệt, lớp Kelihos_ver3 được phân loại hoàn toàn chính xác với Precision, Recall và F1-Score đều đạt 100%, trong khi các lớp Ramnit, Lollipop, Tracur, Kelihos_ver1 và Gatak đều đạt F1-Score trên 98%. Lớp Vundo cũng đạt kết quả rất tốt với F1-Score 97,30%. Đối với lớp Obfuscator.ACY, mô hình đạt Precision 98,29% và Recall 93,48%, tương ứng F1-Score 95,82%, cho thấy vẫn tồn tại một số trường hợp nhầm lẫn nhưng không đáng kể. Lớp Simda có số lượng mẫu ít nhất trong tập kiểm thử (6 mẫu) và đạt F1-Score 76,92%, thấp hơn các lớp còn lại, tuy nhiên Recall đạt 83,33% cho thấy mô hình vẫn nhận diện được phần lớn các mẫu thuộc lớp này. So với các mô hình Machine Learning truyền thống, ResNet-50 đạt hiệu năng tương đương Random Forest (Accuracy 98,65%) nhưng vượt trội hơn SVM (93,62%) và Multinomial Naive Bayes (73,11%). Đồng thời, mô hình duy trì sự cân bằng tốt giữa các lớp dữ liệu khi F1-Score Macro đạt gần 96%, chứng tỏ khả năng tổng quát hóa tốt ngay cả đối với các lớp có số lượng mẫu hạn chế. Kết quả này cho thấy ResNet-50 là một phương pháp hiệu quả trong bài toán phân loại mã độc dựa trên biểu diễn ảnh, có khả năng tự động trích xuất đặc trưng và đạt độ chính xác cao mà không cần thực hiện các bước lựa chọn đặc trưng thủ công như các phương pháp Machine Learning truyền thống.

b) Vision Transformer

Trong nghiên cứu này, mô hình Vision Transformer (ViT) được sử dụng để phân loại các họ mã độc sau khi chuyển đổi dữ liệu sang dạng ảnh. Nghiên cứu sử dụng mô hình ViT-Base-Patch16-224 đã được huấn luyện trước trên tập dữ liệu ImageNet và áp dụng

phương pháp Transfer Learning để tận dụng các đặc trưng thị giác đã được học trước đó. Dữ liệu đầu vào được xử lý thông qua ViTImageProcessor, trong đó các ảnh xám được chuyển đổi sang ảnh RGB ba kênh, chuẩn hóa và chia thành các patch kích thước 16×16 trước khi đưa vào mô hình.

Để phù hợp với bài toán phân loại 9 họ mã độc, lớp phân loại cuối cùng của mô hình được thay thế bằng một lớp Fully Connected mới có số lượng đầu ra tương ứng với số lớp cần nhận dạng. Trước khi huấn luyện chính thức, nghiên cứu tiến hành tối ưu hóa siêu tham số bằng thư viện Optuna. Các tham số được khảo sát bao gồm tốc độ học (learning rate), kích thước batch và số epoch huấn luyện. Quá trình tìm kiếm được thực hiện trên tập xác thực với chỉ số F1-Score Macro làm tiêu chí đánh giá nhằm lựa chọn cấu hình mang lại hiệu năng tốt nhất.

Bảng 12: Kết quả thử nghiệm các bộ siêu tham số của ViT

#	Eval F1 Macro	Duration	Learning Rate	Num Train Epochs	Per Device Train Batch Size
1	98.45%	0 days 00:09:06.480525	0.0001640	3	8
2	97.94%	0 days 00:15:08.258601	0.0000880	5	8
3	97.89%	0 days 00:11:08.497330	0.0001170	4	16
4	97.78%	0 days 00:15:21.504045	0.0001650	5	8
5	96.75%	0 days 00:15:08.962160	0.0002130	5	8

Theo *bảng 12*, Quá trình tối ưu hóa siêu tham số cho thấy cấu hình tốt nhất đạt F1-Score Macro 98,45% với thời gian huấn luyện khoảng 9 phút 06 giây trong giai đoạn tìm kiếm tham số. Bộ tham số tương ứng bao gồm learning rate bằng $1,64 \times 10^{-4}$, số epoch bằng 3 và kích thước batch bằng 8. Các cấu hình còn lại đạt F1-Score Macro từ 96,75% đến 97,94%, cho thấy mô hình ViT duy trì hiệu năng ổn định trên nhiều tổ hợp tham số khác nhau. Kết quả cũng cho thấy learning rate trong khoảng từ $8,8 \times 10^{-5}$ đến $1,65 \times 10^{-4}$ mang lại hiệu quả tốt hơn so với các giá trị lớn hơn như $2,13 \times 10^{-4}$. Ngoài ra, việc tăng số epoch từ 3 lên 5 không mang lại sự cải thiện đáng kể về hiệu năng trong giai đoạn tối ưu, trong khi làm tăng đáng kể thời gian huấn luyện.

Bảng 13: Kết quả của Vision Transformer

Metric Class	Precision	Recall	F1-Score	Support
Ramnit	98.71%	100.00%	99.35%	230
Lollipop	99.73%	98.92%	99.32%	371
Kelihos_ver3	99.77%	100.00%	99.89%	442
Vundo	97.26%	98.61%	97.93%	72
Simda	100.00%	83.33%	90.91%	6
Tracur	96.52%	99.11%	97.80%	112
Kelihos_ver1	100.00%	98.33%	99.16%	60
Obfuscator.ACY	98.88%	96.20%	97.52%	184
Gatak	98.70%	100.00%	99.35%	152
---	---	---	---	---
Accuracy			99.08%	1629
Macro Avg	98.84%	97.17%	97.91%	1629
Weighted Avg	99.09%	99.08%	99.08%	1629

Với bộ siêu tham số đã được chọn, mô hình Vision Transformer (ViT) đạt hiệu năng cao nhất trong số các mô hình được khảo sát với độ chính xác (Accuracy) đạt 99,08%, F1-Score Macro đạt 97,91% và F1-Score Weighted đạt 99,08%. Mô hình thể hiện khả năng phân loại xuất sắc trên hầu hết các họ mã độc, trong đó Kelihos_ver3 được nhận diện gần như hoàn hảo với F1-Score đạt 99,89%, trong khi các lớp Ramnit, Lollipop và Gatak đều đạt F1-Score trên 99%. Các lớp Vundo, Tracur, Kelihos_ver1 và Obfuscator.ACY cũng đạt kết quả rất cao với F1-Score dao động từ 97,52% đến 99,16%, cho thấy mô hình có khả năng học và phân biệt hiệu quả các đặc trưng của từng họ mã độc. Đối với lớp Simda, mặc dù số lượng mẫu trong tập kiểm thử rất nhỏ (6 mẫu), mô hình vẫn đạt F1-Score 90,91% với Recall 83,33%, chứng tỏ khả năng nhận diện tốt ngay cả đối với các lớp thiểu số. Giá trị F1-Score Macro đạt 97,91% cùng với sự chênh lệch không lớn so với F1-Score Weighted cho thấy mô hình duy trì hiệu năng ổn định trên toàn bộ các lớp dữ liệu và ít bị ảnh hưởng bởi hiện tượng mất cân bằng dữ liệu. So với các mô hình còn lại trong nghiên cứu, Vision Transformer đạt kết quả tốt hơn Random Forest (Accuracy 98,65%), ResNet-50 (98,59%), SVM (93,62%) và Multinomial Naive Bayes (73,11%), qua đó khẳng định hiệu quả của kiến trúc

Transformer trong việc khai thác các đặc trưng toàn cục của dữ liệu ảnh phục vụ bài toán phân loại mã độc.

c) Pyramid Vision Transformer (PVT)

Ngoài việc thử nghiệm với mô hình Vision Transformer (ViT), nghiên cứu cũng tiến hành đánh giá hiệu quả của mô hình Pyramid Vision Transformer (PVT) trong bài toán phân loại mã độc. PVT là một biến thể của Vision Transformer được thiết kế theo kiến trúc phân cấp nhiều tầng (pyramid architecture), cho phép trích xuất đặc trưng ở nhiều mức độ khác nhau tương tự như các mạng tích chập truyền thống. Nhờ cơ chế giảm dần kích thước đặc trưng qua từng tầng, PVT có khả năng khai thác cả thông tin cục bộ và toàn cục của ảnh đầu vào với chi phí tính toán thấp hơn so với Vision Transformer tiêu chuẩn.

Trong nghiên cứu này, mô hình PVT-Tiny-224 đã được huấn luyện trước trên tập dữ liệu ImageNet và được áp dụng theo phương pháp Transfer Learning. Dữ liệu ảnh mã độc được xử lý thông qua AutoImageProcessor, trong đó các ảnh xám được chuyển đổi sang định dạng RGB ba kênh và chuẩn hóa theo yêu cầu của mô hình. Lớp phân loại cuối cùng được thay thế bằng một lớp Fully Connected mới có số lượng đầu ra tương ứng với 9 họ mã độc.

Để tìm kiếm cấu hình phù hợp cho mô hình, nghiên cứu sử dụng thư viện Optuna nhằm tối ưu hóa các siêu tham số quan trọng như tốc độ học (learning rate), kích thước batch và số epoch huấn luyện. Quá trình tìm kiếm được thực hiện trên tập xác thực với chỉ số F1-Score Macro làm tiêu chí đánh giá. Sau khi xác định được bộ tham số tối ưu, mô hình được huấn luyện chính thức trong 50 epoch và đánh giá trên tập kiểm thử thông qua các chỉ số Accuracy, Precision, Recall, F1-Score, Confusion Matrix và Classification Report. Ngoài ra, nghiên cứu còn tiến hành đo lường độ phức tạp tính toán của mô hình thông qua số lượng phép toán dấu phẩy động (FLOPs) bằng thư viện fvcare và PyTorch Profiler. Kết quả thu được được sử dụng để so sánh với các mô hình ViT, ResNet-50 và các phương pháp Machine Learning truyền thống nhằm đánh giá hiệu quả của kiến trúc Transformer phân cấp trong bài toán phân loại mã độc.

Bảng 14: Kết quả của Pyramid Vision Transformer

Metric Class	Precision	Recall	F1-Score	Support
Ramnit	98.71%	99.57%	99.13%	230
Lollipop	99.73%	98.92%	99.32%	371
Kelihos_ver3	100.00%	99.77%	99.89%	442
Vundo	93.42%	98.61%	95.95%	72
Simda	83.33%	83.33%	83.33%	6
Tracur	99.10%	98.21%	98.65%	112
Kelihos_ver1	100.00%	98.33%	99.16%	60
Obfuscator.ACY	97.25%	96.20%	96.72%	184
Gatak	98.70%	100.00%	99.35%	152
---	---	---	---	---
Accuracy			98.90%	1629
Macro Avg	96.69%	96.99%	96.83%	1629
Weighted Avg	98.91%	98.90%	98.90%	1629

Pyramid Vision Transformer (PVT) đạt hiệu năng rất cao trên tập kiểm thử với độ chính xác (Accuracy) đạt 98,90%, F1-Score Macro đạt 96,83% và F1-Score Weighted đạt 98,90%. Mô hình nhận diện hiệu quả hầu hết các họ mã độc trong bộ dữ liệu, trong đó các lớp Kelihos_ver3, Lollipop, Ramnit, Kelihos_ver1 và Gatak đều đạt F1-Score trên 99%. Đối với các lớp Tracur và Obfuscator.ACY, mô hình cũng cho kết quả khả quan với F1-Score lần lượt đạt 98,65% và 96,72%. Lớp Vundo đạt F1-Score 95,95%, cho thấy khả năng phân loại chính xác và ổn định mặc dù số lượng mẫu không lớn. Riêng lớp Simda có số lượng mẫu rất hạn chế trong tập kiểm thử nhưng vẫn đạt Precision, Recall và F1-Score ở mức 83,33%, chứng tỏ mô hình có khả năng nhận diện tương đối tốt đối với các lớp thiểu số. Bên cạnh đó, giá trị F1-Score Macro đạt 96,83% cho thấy hiệu năng phân loại được duy trì đồng đều trên các lớp dữ liệu khác nhau, trong khi F1-Score Weighted đạt 98,90% phản ánh khả năng dự đoán chính xác trên toàn bộ tập kiểm thử. Kết quả này cho thấy PVT có khả năng học được các đặc trưng quan trọng từ dữ liệu ảnh mã độc và đạt hiệu quả phân loại cao trên nhiều họ mã độc khác nhau.

3.2.3. Thiết kế kỹ thuật Ensemble Learning

Sau khi lựa chọn được các mô hình có hiệu năng tốt nhất, nghiên cứu tiến hành xây dựng mô hình Ensemble Learning nhằm khai thác ưu điểm của nhiều phương pháp biểu diễn dữ liệu và thuật toán học máy khác nhau. Cụ thể, mô hình tổ hợp kết hợp kết quả dự đoán từ Random Forest (sử dụng đặc trưng Bag-of-Words đã được lựa chọn đặc trưng) và Vision Transformer (sử dụng ảnh trực quan hóa từ file malware). Đối với mỗi mẫu dữ liệu, hai mô hình tạo ra xác suất dự đoán cho từng lớp malware. Các xác suất này sau đó được tổng hợp bằng nhiều chiến lược kết hợp khác nhau, bao gồm Sum Rule, Weighted Sum Rule, Product Rule, Max Rule, Min Rule, Median Rule và Mean Rule. Trong đó, Sum Rule cộng trực tiếp các xác suất dự đoán của các mô hình, Weighted Sum Rule sử dụng trọng số động dựa trên mức độ đồng thuận giữa các bộ phân loại, Product Rule nhân các xác suất tương ứng, trong khi Max Rule, Min Rule và Median Rule lần lượt lựa chọn giá trị lớn nhất, nhỏ nhất hoặc trung vị của các xác suất dự đoán. Cuối cùng, lớp có điểm số cao nhất sau quá trình kết hợp sẽ được chọn làm kết quả phân loại cuối cùng. Cách tiếp cận này giúp tận dụng đồng thời thông tin từ đặc trưng tĩnh dạng văn bản và đặc trưng trực quan của malware, từ đó nâng cao khả năng nhận diện và giảm thiểu sai số của từng mô hình riêng lẻ.

a) Sum rule

Sum Rule là phương pháp ensemble được sử dụng để kết hợp xác suất dự đoán từ các mô hình thành viên, bao gồm Random Forest và Vision Transformer (ViT). Đối với mỗi mẫu mã độc, các mô hình sẽ xuất ra xác suất thuộc về từng lớp. Xác suất của cùng một lớp được cộng lại để tạo thành điểm số tổng hợp:

$$Score_j = \sum_{i=1}^R P_i(\omega_j|x)$$

Trong đó:

- $(P_i(\omega_j|x))$ là xác suất dự đoán của mô hình thứ (i) đối với lớp (ω_j)
- (R) là số lượng mô hình tham gia ensemble.

Lớp có tổng xác suất lớn nhất sẽ được chọn làm kết quả phân loại cuối cùng. Phương pháp này cho phép khai thác thông tin bổ sung từ các mô hình khác nhau, từ đó cải thiện hiệu quả phát hiện và phân loại mã độc so với việc sử dụng từng mô hình riêng lẻ.

Bảng 15: Kết quả của Sum Rule

Metric Class	Precision	Recall	F1-Score	Support
Ramnit	97.86%	99.57%	98.71%	230
Lollipop	100.00%	98.65%	99.32%	371
Kelihos_ver3	100.00%	99.77%	99.89%	442
Vundo	94.74%	100.00%	97.30%	72
Simda	100.00%	66.67%	80.00%	6
Tracur	100.00%	96.43%	98.18%	112
Kelihos_ver1	100.00%	100.00%	100.00%	60
Obfuscator.ACY	95.19%	96.74%	95.96%	184
Gatak	99.35%	100.00%	99.67%	152
---	---	---	---	---
Accuracy			98.83%	1629
Macro Avg	98.57%	95.31%	96.56%	1629
Weighted Avg	98.86%	98.83%	98.83%	1629

Phương pháp Sum Rule đạt độ chính xác 98,83% trên tập kiểm thử gồm 1.629 mẫu, với Precision, Recall và F1-score trung bình có trọng số lần lượt là 98,86%, 98,83% và 98,83%. Hầu hết các họ mã độc đều được phân loại rất chính xác với F1-score trên 95%, trong đó Kelihos_ver1 đạt kết quả hoàn hảo (100%), còn Ramnit, Lollipop, Kelihos_ver3 và Gatak đều đạt trên 98%. Mặc dù lớp Simda chỉ đạt F1-score 80,00% do số lượng mẫu rất ít, kết quả tổng thể vẫn cho thấy Sum Rule khai thác hiệu quả thông tin từ các mô hình thành viên, góp phần nâng cao hiệu suất phân loại mã độc.

b) Weighted Sum Rule

Weighted Sum Rule là phiên bản mở rộng của Sum Rule, trong đó xác suất dự đoán của từng mô hình được nhân với một trọng số tương ứng trước khi thực hiện phép cộng. Trong nghiên cứu này, trọng số được tính động dựa trên mức độ khác biệt giữa xác suất dự đoán của từng mô hình và giá trị xác suất trung bình của lớp tương ứng. Các mô hình có dự đoán gần với xu hướng chung sẽ nhận trọng số lớn hơn, trong khi các mô hình có dự đoán lệch nhiều sẽ bị giảm ảnh hưởng. Điểm số của mỗi lớp được tính bằng tổng có

trọng số của các xác suất dự đoán, và lớp có điểm số cao nhất sẽ được chọn làm kết quả phân loại cuối cùng.

Bảng 16: Kết quả của Weighted Sum Rule

Metric Class	Precision	Recall	F1-Score	Support
Ramnit	97.45%	99.57%	98.50%	230
Lollipop	100.00%	98.65%	99.32%	371
Kelihos_ver3	100.00%	99.77%	99.89%	442
Vundo	94.74%	100.00%	97.30%	72
Simda	100.00%	66.67%	80.00%	6
Tracur	100.00%	95.54%	97.72%	112
Kelihos_ver1	100.00%	100.00%	100.00%	60
Obfuscator.ACY	95.19%	96.74%	95.96%	184
Gatak	99.35%	100.00%	99.67%	152
---	---	---	---	---
Accuracy			98.77%	1629
Macro Avg	98.52%	95.22%	96.48%	1629
Weighted Avg	98.80%	98.77%	98.77%	1629

Phương pháp Weighted Sum Rule đạt độ chính xác 98,77% với Precision, Recall và F1-score trung bình có trọng số lần lượt là 98,80%, 98,77% và 98,77%. Hầu hết các họ mã độc đều đạt kết quả phân loại rất cao với F1-score trên 95%, trong đó Kelihos_ver1 đạt 100%, còn Lollipop, Kelihos_ver3 và Gatak đều đạt trên 99%. Tuy nhiên, hiệu suất tổng thể của Weighted Sum Rule thấp hơn nhẹ so với Sum Rule thông thường, cho thấy việc áp dụng trọng số động chưa mang lại cải thiện đáng kể trong trường hợp này, mặc dù vẫn duy trì khả năng phân loại ổn định trên hầu hết các lớp mã độc.

c) Product Rule

Tiếp theo, Product Rule là phương pháp ensemble kết hợp kết quả dự đoán bằng cách nhân xác suất của cùng một lớp từ các mô hình thành viên. Trong nghiên cứu này, xác suất dự đoán của Random Forest và Vision Transformer (ViT) đối với mỗi lớp được nhân với nhau để tạo thành điểm số tổng hợp. Lớp có tích xác suất lớn nhất sẽ được chọn làm kết quả phân loại cuối cùng. So với Sum Rule, Product Rule nhấn mạnh sự

đồng thuận giữa các mô hình, vì một lớp chỉ đạt điểm số cao khi được tất cả các mô hình cùng dự đoán với xác suất lớn. Điều này giúp tăng độ tin cậy của quyết định phân loại nhưng cũng khiến phương pháp nhạy cảm hơn với các dự đoán có xác suất thấp từ một trong các mô hình thành viên.

Bảng 17: Kết quả của Product Rule

Metric Class	Precision	Recall	F1-Score	Support
Ramnit	97.86%	99.57%	98.71%	230
Lollipop	100.00%	98.92%	99.46%	371
Kelihos_ver3	100.00%	99.77%	99.89%	442
Vundo	95.95%	98.61%	97.26%	72
Simda	100.00%	83.33%	90.91%	6
Tracur	99.09%	97.32%	98.20%	112
Kelihos_ver1	100.00%	100.00%	100.00%	60
Obfuscator.ACY	96.22%	96.74%	96.48%	184
Gatak	99.35%	100.00%	99.67%	152
---	---	---	---	---
Accuracy			98.96%	1629
Macro Avg	98.72%	97.14%	97.84%	1629
Weighted Avg	98.97%	98.96%	98.96%	1629

Phương pháp Product Rule đạt độ chính xác 98,96% trên tập kiểm thử gồm 1.629 mẫu, cao hơn cả Sum Rule và Weighted Sum Rule. Precision, Recall và F1-score trung bình có trọng số lần lượt đạt 98,97%, 98,96% và 98,96%. Hầu hết các lớp mã độc đều được phân loại rất chính xác với F1-score trên 96%, trong đó Kelihos_ver1 đạt kết quả hoàn hảo 100%, còn Lollipop, Kelihos_ver3 và Gatak đều đạt trên 99%. Đặc biệt, lớp Simda đạt F1-score 90,91%, cao hơn đáng kể so với các phương pháp trước. Kết quả này cho thấy Product Rule khai thác hiệu quả sự đồng thuận giữa các mô hình thành viên, từ đó nâng cao hiệu suất phân loại tổng thể.

d) Max Rule

Max Rule là phương pháp ensemble lựa chọn xác suất lớn nhất của từng lớp từ các mô hình thành viên để thực hiện phân loại. Trong nghiên cứu này, đối với mỗi lớp (ω_j),

điểm số được xác định bằng giá trị xác suất lớn nhất mà các mô hình thành viên dự đoán:

$$Score_j = \max_{i=1,\dots,R} P_i(\omega_j|x)$$

Trong đó: $(P_i(\omega_j|x))$ là xác suất dự đoán của mô hình thứ (i) đối với lớp (ω_j) , và (R) là số lượng mô hình tham gia ensemble.

Kết quả dự đoán cuối cùng được xác định bởi:

$$\hat{\omega} = \arg \max_j (Score_j)$$

Khác với Sum Rule hay Product Rule, Max Rule chỉ xem xét giá trị xác suất cao nhất của mỗi lớp thay vì kết hợp toàn bộ thông tin từ các mô hình. Do đó, phương pháp này ưu tiên quyết định của mô hình có độ tin cậy cao nhất đối với từng lớp và có thể đạt hiệu quả tốt khi một trong các mô hình thành viên đưa ra dự đoán vượt trội so với các mô hình còn lại.

Bảng 18: Kết quả của Max Rule

Metric Class	Precision	Recall	F1-Score	Support
Ramnit	97.45%	99.57%	98.50%	230
Lollipop	100.00%	98.65%	99.32%	371
Kelihos_ver3	100.00%	99.77%	99.89%	442
Vundo	93.51%	100.00%	96.64%	72
Simda	100.00%	66.67%	80.00%	6
Tracur	100.00%	94.64%	97.25%	112
Kelihos_ver1	100.00%	100.00%	100.00%	60
Obfuscator.ACY	95.19%	96.74%	95.96%	184
Gatak	99.35%	100.00%	99.67%	152
---	---	---	---	---
Accuracy			98.71%	1629
Macro Avg	98.39%	95.12%	96.36%	1629
Weighted Avg	98.75%	98.71%	98.70%	1629

Max Rule đạt độ chính xác 98,71% trên tập kiểm thử gồm 1.629 mẫu, với Precision, Recall và F1-score trung bình có trọng số lần lượt đạt 98,75%, 98,71% và 98,70%. Hầu

hết các lớp mã độc đều được phân loại chính xác với F1-score trên 95%, trong đó Kelihos_ver1 đạt kết quả hoàn hảo 100%, còn Lollipop, Kelihos_ver3 và Gatak đều đạt trên 99%. Tuy nhiên, hiệu suất tổng thể của Max Rule thấp hơn so với Sum Rule và Product Rule, cho thấy việc chỉ sử dụng xác suất lớn nhất từ các mô hình thành viên chưa khai thác đầy đủ thông tin bổ sung giữa các mô hình trong quá trình phân loại mã độc.

e) Min Rule

Min Rule là phương pháp ensemble sử dụng xác suất nhỏ nhất của từng lớp từ các mô hình thành viên để đưa ra quyết định phân loại. Đối với mỗi lớp (ω_j) , điểm số được xác định theo công thức:

$$Score_j = \min_{i=1,\dots,R} P_i(\omega_j|x)$$

Trong đó $(P_i(\omega_j|x))$ là xác suất dự đoán của mô hình thứ (i) đối với lớp (ω_j) và (R) là số lượng mô hình tham gia ensemble.

Kết quả phân loại cuối cùng được xác định bởi:

$$\hat{\omega} = \arg \max_j (Score_j)$$

Nếu so với Max Rule, Min Rule chỉ xem xét mức độ đồng thuận thấp nhất giữa các mô hình. Một lớp chỉ nhận được điểm số cao khi tất cả các mô hình đều đánh giá lớp đó với xác suất tương đối lớn. Do đó, phương pháp này có xu hướng thận trọng hơn trong quá trình ra quyết định và giảm ảnh hưởng của các dự đoán quá tự tin từ một mô hình riêng lẻ.

Bảng 19: Kết quả của Min Rule

Metric Class	Precision	Recall	F1-Score	Support
Ramnit	97.86%	99.57%	98.71%	230
Lollipop	100.00%	98.92%	99.46%	371
Kelihos_ver3	100.00%	99.77%	99.89%	442
Vundo	95.95%	98.61%	97.26%	72
Simda	100.00%	83.33%	90.91%	6
Tracur	98.20%	97.32%	97.76%	112
Kelihos_ver1	100.00%	100.00%	100.00%	60
Obfuscator.ACY	96.74%	96.74%	96.74%	184
Gatak	99.35%	100.00%	99.67%	152
---	---	---	---	---
Accuracy			98.96%	1629
Macro Avg	98.68%	97.14%	97.82%	1629
Weighted Avg	98.97%	98.96%	98.96%	1629

Kết quả cho thấy Min Rule đạt độ chính xác 98,96% trên tập kiểm thử gồm 1.629 mẫu, với Precision, Recall và F1-score trung bình có trọng số đều đạt 98,96%. Hầu hết các lớp mã độc được phân loại rất chính xác với F1-score trên 97%, trong đó Kelihos_ver1 đạt kết quả hoàn hảo 100%, còn Lollipop, Kelihos_ver3 và Gatak đều đạt trên 99%. Đặc biệt, lớp Simda đạt F1-score 90,91%, cao hơn đáng kể so với Sum Rule và Max Rule. Kết quả này cho thấy Min Rule khai thác hiệu quả sự đồng thuận giữa các mô hình thành viên và đạt hiệu suất tương đương Product Rule, đồng thời là một trong những phương pháp ensemble cho kết quả tốt nhất trong nghiên cứu.

f) Median Rule

Còn Median Rule là phương pháp ensemble sử dụng giá trị trung vị (median) của các xác suất dự đoán từ các mô hình thành viên để tính điểm cho từng lớp. Đối với mỗi lớp (ω_j), điểm số được xác định theo công thức:

$$Score_j = \text{median} \left(P_1(\omega_j|x), P_2(\omega_j|x), \dots, P_R(\omega_j|x) \right)$$

Trong đó $(P_i(\omega_j|x))$ là xác suất dự đoán của mô hình thứ (i) đối với lớp (ω_j) , và (R) là số lượng mô hình tham gia ensemble.

Và kết quả phân loại cuối cùng được xác định bởi:

$$\hat{\omega} = \arg \max_j (Score_j)$$

Median Rule có ưu điểm là giảm ảnh hưởng của các giá trị xác suất bất thường hoặc quá chênh lệch từ một mô hình riêng lẻ. Nhờ sử dụng giá trị trung vị thay vì giá trị trung bình, phương pháp này tạo ra quyết định ổn định hơn trong trường hợp các mô hình thành viên đưa ra các dự đoán không đồng nhất.

Bảng 20: Kết quả của Median Rule

Class \ Metric	Precision	Recall	F1-Score	Support
Ramnit	97.45%	99.57%	98.50%	230
Lollipop	100.00%	98.65%	99.32%	371
Kelihos_ver3	100.00%	99.77%	99.89%	442
Vundo	94.74%	100.00%	97.30%	72
Simda	100.00%	66.67%	80.00%	6
Tracur	100.00%	95.54%	97.72%	112
Kelihos_ver1	100.00%	100.00%	100.00%	60
Obfuscator.ACY	95.19%	96.74%	95.96%	184
Gatak	99.35%	100.00%	99.67%	152
---	---	---	---	---
Accuracy			98.77%	1629
Macro Avg	98.52%	95.22%	96.48%	1629
Weighted Avg	98.80%	98.77%	98.77%	1629

Phương pháp Median Rule đạt độ chính xác 98,77% trên tập kiểm thử gồm 1.629 mẫu, với Precision, Recall và F1-score trung bình có trọng số lần lượt đạt 98,80%, 98,77% và 98,77%. Hầu hết các lớp mã độc đều được phân loại chính xác với F1-score trên 95%, trong đó Kelihos_ver1 đạt kết quả hoàn hảo 100%, còn Lollipop, Kelihos_ver3 và Gatak đều đạt trên 99%. Tuy nhiên, hiệu suất tổng thể của Median Rule thấp hơn so với Product Rule và Min Rule, đồng thời tương đương với Weighted Sum Rule. Điều này cho thấy việc sử dụng giá trị trung vị giúp duy trì sự ổn định của hệ thống ensemble, nhưng chưa khai thác tối đa thông tin từ các mô hình thành viên để cải thiện độ chính xác phân loại.

g) Mean Rule

Cuối cùng, Mean Rule là phương pháp ensemble kết hợp kết quả dự đoán bằng cách tính giá trị trung bình xác suất của các mô hình thành viên đối với từng lớp. Đối với mỗi lớp (ω_j), điểm số được xác định theo công thức:

$$Score_j = \frac{1}{R} \sum_{i=1}^R P_i(\omega_j|x)$$

Trong đó ($P_i(\omega_j|x)$) là xác suất dự đoán của mô hình thứ (i) đối với lớp (ω_j), và (R) là số lượng mô hình tham gia ensemble.

Kết quả phân loại cuối cùng được xác định bởi:

$$\hat{\omega} = \arg \max_j (Score_j)$$

Mean Rule là một dạng soft voting phổ biến, trong đó tất cả các mô hình thành viên được xem có mức độ đóng góp như nhau vào quyết định cuối cùng. Phương pháp này giúp tận dụng thông tin từ nhiều mô hình khác nhau, đồng thời giảm ảnh hưởng của các dự đoán sai lệch từ từng mô hình riêng lẻ, từ đó nâng cao độ ổn định và khả năng tổng quát hóa của hệ thống phân loại mã độc.

Bảng 21: Kết quả của Mean Rule

Metric Class	Precision	Recall	F1-Score	Support
Ramnit	97.45%	99.57%	98.50%	230
Lollipop	100.00%	98.65%	99.32%	371
Kelihos_ver3	100.00%	99.77%	99.89%	442
Vundo	94.74%	100.00%	97.30%	72
Simda	100.00%	66.67%	80.00%	6
Tracur	100.00%	95.54%	97.72%	112
Kelihos_ver1	100.00%	100.00%	100.00%	60
Obfuscator.ACY	95.19%	96.74%	95.96%	184
Gatak	99.35%	100.00%	99.67%	152
---	---	---	---	---
Accuracy			98.77%	1629
Macro Avg	98.52%	95.22%	96.48%	1629

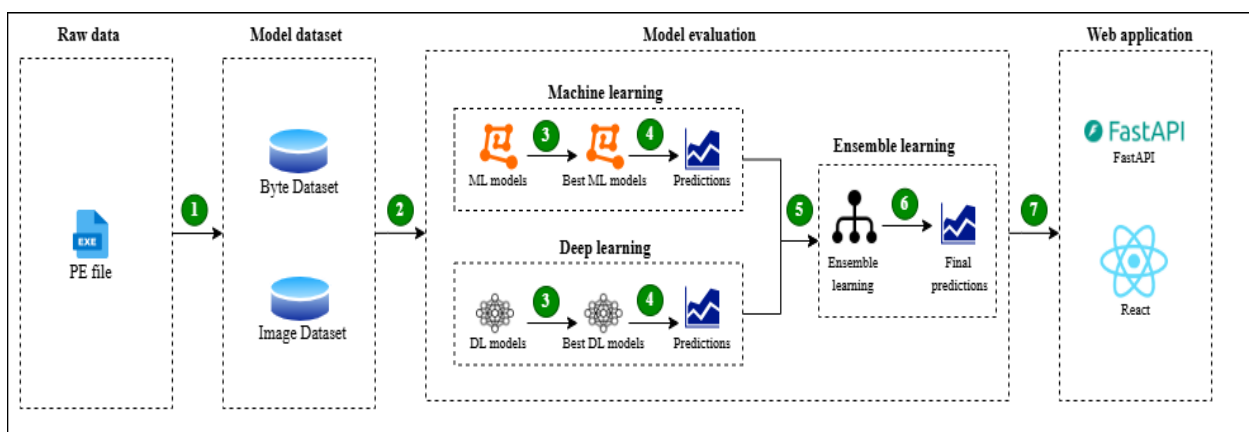
Weighted Avg	98.80%	98.77%	98.77%	1629
---------------------	--------	--------	--------	------

Phương pháp Mean Rule đạt độ chính xác 98,77% trên tập kiểm thử gồm 1.629 mẫu, với Precision, Recall và F1-score trung bình có trọng số lần lượt đạt 98,80%, 98,77% và 98,77%. Hầu hết các họ mã độc đều được phân loại rất chính xác với F1-score trên 95%, trong đó Kelihos_ver1 đạt kết quả hoàn hảo 100%, còn Lollipop, Kelihos_ver3 và Gatak đều đạt trên 99%. Tuy nhiên, hiệu suất tổng thể của Mean Rule thấp hơn so với Product Rule và Min Rule, đồng thời tương đương với Weighted Sum Rule và Median Rule. Kết quả này cho thấy việc lấy trung bình xác suất từ các mô hình thành viên giúp duy trì khả năng phân loại ổn định, nhưng chưa tạo ra sự cải thiện đáng kể về hiệu suất so với các phương pháp ensemble dựa trên mức độ đồng thuận như Product Rule và Min Rule.

3.3. Xây dựng hệ thống phát hiện và phân loại mã độc

3.3.1. Sơ đồ chức năng hệ thống

Để làm rõ quy trình xử lý dữ liệu và cơ chế hoạt động của hệ thống phát hiện, phân loại mã độc được đề xuất, sơ đồ chức năng tổng thể của hệ thống được xây dựng nhằm mô tả các thành phần chính cũng như luồng xử lý từ đầu vào đến đầu ra. Sơ đồ không chỉ thể hiện mối quan hệ giữa các giai đoạn tiền xử lý dữ liệu, huấn luyện mô hình và kết hợp dự đoán mà còn minh họa cách hệ thống được triển khai thành một ứng dụng thực tế. Thông qua sơ đồ này, có thể thấy được cách dữ liệu được chuyển đổi qua nhiều dạng biểu diễn khác nhau, được xử lý bởi các mô hình học máy và học sâu, sau đó kết hợp bằng kỹ thuật Ensemble Learning để tạo ra kết quả phân loại cuối cùng.



Hình 7: Sơ đồ chức năng hệ thống phát hiện mã độc

Hình 7 mô tả tổng quan kiến trúc và quy trình hoạt động của hệ thống phát hiện và phân loại mã độc được đề xuất. Hệ thống được xây dựng theo hướng kết hợp giữa các mô hình học máy, học sâu và kỹ thuật Ensemble Learning nhằm tận dụng ưu điểm của nhiều phương pháp khác nhau để nâng cao hiệu quả phân loại.

Quy trình xử lý bắt đầu từ việc thu thập các tệp thực thi định dạng PE (Portable Executable) ở bước (1). Sau khi được tiếp nhận, dữ liệu sẽ trải qua giai đoạn tiền xử lý để chuyển đổi sang hai dạng biểu diễn khác nhau gồm dữ liệu byte và dữ liệu ảnh. Dữ liệu byte được sử dụng để trích xuất các đặc trưng thống kê phục vụ cho các mô hình học máy, trong khi dữ liệu ảnh được sử dụng làm đầu vào cho các mô hình học sâu nhằm khai thác các đặc trưng trực quan của mã độc.

Tiếp theo ở bước (2), các tập dữ liệu được đưa vào quá trình huấn luyện và đánh giá mô hình. Đối với nhánh học máy, nhiều mô hình được xây dựng và đánh giá để lựa chọn mô hình có hiệu suất tốt nhất ở bước (3). Tương tự, ở nhánh học sâu, các kiến trúc mạng nơ-ron khác nhau được thử nghiệm và lựa chọn dựa trên các chỉ số đánh giá như Accuracy, Precision, Recall và F1-score. Các mô hình tối ưu sau đó được sử dụng để thực hiện dự đoán trên dữ liệu đầu vào và tạo ra các xác suất phân loại tương ứng cho từng họ mã độc.

Trong bước (5) và (6), thay vì sử dụng kết quả từ một mô hình đơn lẻ, hệ thống áp dụng các phương pháp Ensemble Learning để kết hợp đầu ra từ các mô hình thành viên. Các kỹ thuật như Sum Rule, Weighted Sum Rule, Product Rule, Max Rule, Min Rule và Median Rule được sử dụng nhằm tận dụng tính bổ sung giữa các mô hình, giảm ảnh hưởng của các dự đoán sai lệch và cải thiện độ chính xác tổng thể. Kết quả sau khi kết hợp sẽ tạo thành dự đoán cuối cùng của hệ thống.

Bên cạnh bài toán phân loại đa lớp trên bộ dữ liệu Microsoft Malware Classification Challenge 2015, nghiên cứu còn mở rộng dữ liệu bằng cách thu thập thêm 25.464 tệp lành tính từ hệ điều hành Windows và 65.536 mẫu mã độc từ VirusShare nhằm xây dựng bộ phân loại nhị phân giữa tệp lành tính và mã độc. Việc mở rộng tập dữ liệu giúp hệ thống có khả năng hoạt động trên các tình huống thực tế thay vì chỉ giới hạn trong môi trường nghiên cứu.

Cuối cùng, mô hình được triển khai thành một ứng dụng web hoàn chỉnh ở bước (7). Phần backend được xây dựng bằng FastAPI để xử lý yêu cầu từ người dùng, thực hiện

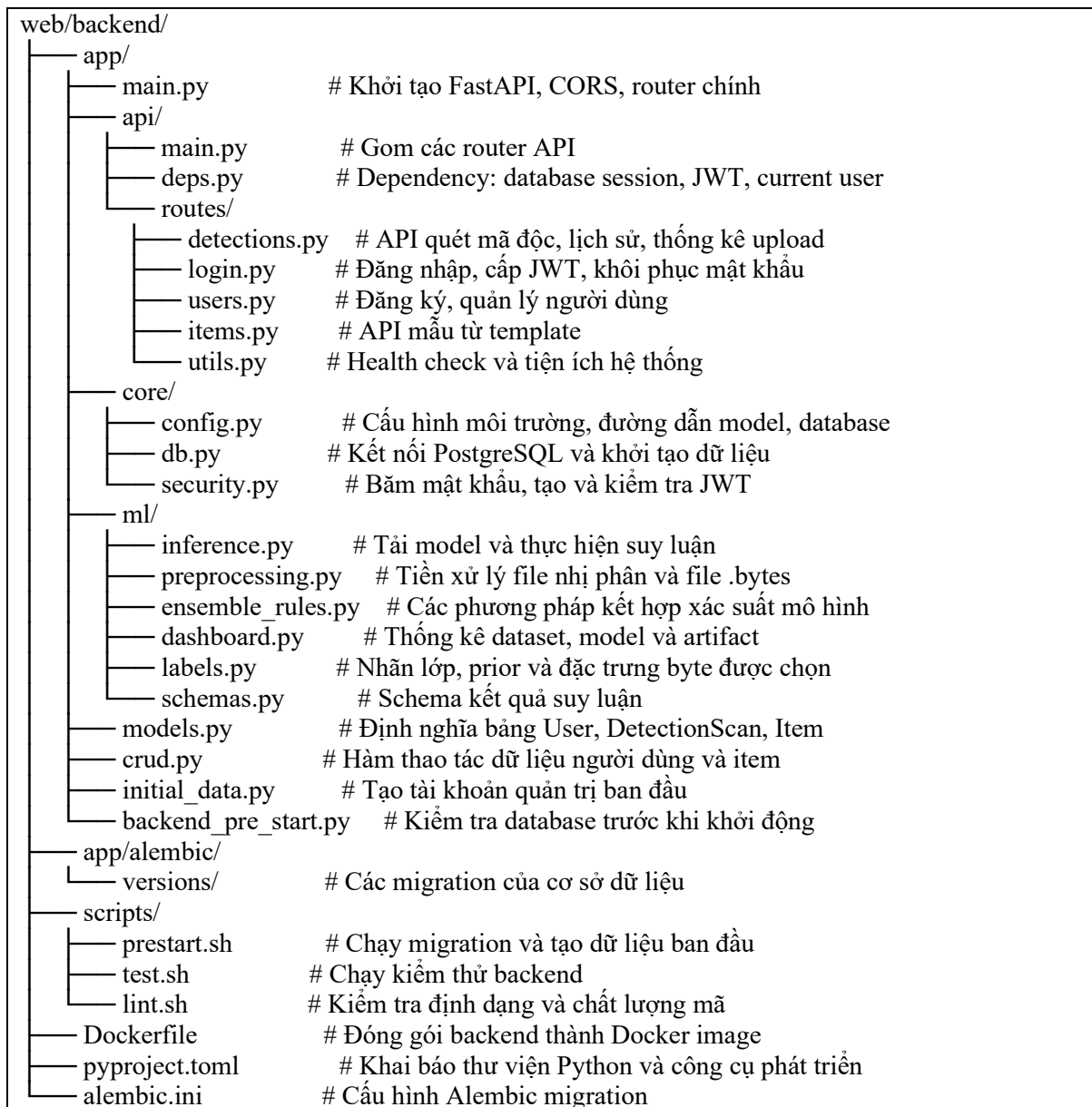
tiền xử lý dữ liệu và suy luận mô hình. Trong khi đó, React được sử dụng để phát triển giao diện người dùng, cung cấp môi trường tương tác trực quan cho phép tải lên tệp thực thi, theo dõi quá trình phân tích và nhận kết quả phân loại mã độc. Cách tiếp cận này giúp hệ thống không chỉ đạt hiệu suất phân loại cao mà còn có khả năng triển khai thực tế trong các bài toán hỗ trợ phân tích và nhận diện mã độc.

3.3.2. Triển khai

a. Backend module

Backend của hệ thống được xây dựng bằng Python và framework FastAPI, cung cấp các RESTful API phục vụ giao tiếp với frontend, quản lý người dùng, lưu lịch sử quét và thực hiện suy luận trên tệp được tải lên. Dữ liệu được lưu trữ trong PostgreSQL, trong khi SQLAlchemy được sử dụng làm lớp ánh xạ đối tượng – quan hệ. Cấu trúc backend được chia thành các nhóm chính gồm: API, xử lý dữ liệu, suy luận mô hình, truy cập cơ sở dữ liệu và cấu hình hệ thống.

Và cơ sở dữ liệu mã độc được cải thiện bằng cách thu thập thêm 25.464 tệp lành tính (benign) từ hệ điều hành Windows và 65.536 mẫu mã độc từ nền tảng VirusShare. Việc bổ sung dữ liệu từ các nguồn thực tế giúp tăng tính đa dạng của tập dữ liệu, giảm sự phụ thuộc vào một bộ dữ liệu duy nhất và nâng cao khả năng tổng quát hóa của mô hình khi triển khai trong môi trường thực tế. Tập dữ liệu mở rộng này được sử dụng để xây dựng bộ phân loại nhị phân giữa tệp lành tính và mã độc, góp phần nâng cao khả năng ứng dụng của hệ thống trong các tình huống sử dụng thực tế.



Hình 8: Cấu trúc thư mục backend hệ thống

Cấu trúc backend được tổ chức theo hướng tách biệt trách nhiệm giữa các tầng xử lý. Tầng api chịu trách nhiệm tiếp nhận yêu cầu HTTP và trả về phản hồi cho frontend. Tầng core quản lý các thành phần nền tảng như cấu hình hệ thống, kết nối cơ sở dữ liệu và bảo mật. Tầng ml chứa toàn bộ logic liên quan đến tiền xử lý dữ liệu, tải mô hình, suy luận và kết hợp kết quả từ nhiều mô hình học máy. Các bảng dữ liệu và schema lưu trữ được định nghĩa trong models.py, trong khi Alembic được sử dụng để quản lý thay đổi cấu trúc cơ sở dữ liệu. Cách tổ chức này giúp backend dễ bảo trì, dễ mở rộng và thuận tiện khi thay đổi hoặc bổ sung mô hình phát hiện mã độc mới.

Luồng xử lý chính bắt đầu tại API POST /api/v1/detections/scan. API tiếp nhận tệp nhị phân thông thường hoặc tệp .bytes của bộ dữ liệu Microsoft Malware Classification Challenge. Kích thước tệp được giới hạn ở mức 50 MB nhằm kiểm soát tài nguyên máy chủ. Sau khi nhận tệp, hệ thống tính giá trị băm SHA-256 để định danh nội dung và kiểm tra kết quả đã tồn tại trong cơ sở dữ liệu hay chưa. Nếu tìm thấy kết quả tương ứng, backend trả về dữ liệu đã lưu thay vì thực hiện lại toàn bộ quá trình suy luận. Cơ chế này giúp giảm thời gian phản hồi và hạn chế việc sử dụng tài nguyên cho các tệp được quét nhiều lần.

Nếu kết quả của tệp chưa tồn tại trong bộ nhớ đệm, hệ thống sẽ chuyển tệp đến module tiền xử lý để xây dựng dữ liệu đầu vào cho các mô hình phân loại. Đối với bộ dữ liệu Microsoft Malware Classification Challenge 2015 ở định dạng .bytes, hệ thống loại bỏ địa chỉ bộ nhớ, các ký tự không hợp lệ và các ký hiệu đặc biệt, sau đó chuyển các giá trị hexadecimal hợp lệ thành mảng byte.

Đối với các tệp thực thi Windows (PE), hệ thống trước tiên kiểm tra tính hợp lệ của định dạng PE thông qua chữ ký MZ và PE. Sau đó, chỉ dữ liệu trong các section của tệp PE được trích xuất làm dữ liệu đầu vào thay vì sử dụng toàn bộ tệp, giúp giảm ảnh hưởng của các vùng dữ liệu không cần thiết như header. Trong trường hợp không thể đọc được bảng section, hệ thống sẽ tự động sử dụng cơ chế dự phòng (fallback) để trích xuất phần thân của tệp. Với các tệp nhị phân thông thường, dữ liệu được đọc trực tiếp thành mảng số nguyên không dấu 8 bit.

Từ cùng một mảng byte thu được sau tiền xử lý, hệ thống xây dựng đồng thời hai dạng dữ liệu đầu vào phục vụ cho các mô hình khác nhau:

- Vector đặc trưng byte: Hệ thống thống kê tần suất xuất hiện của toàn bộ 256 giá trị byte (00–FF), sau đó lựa chọn 20 giá trị byte đã được xác định trước để tạo thành vector đặc trưng đầu vào cho mô hình Random Forest. Cách tiếp cận này giúp giảm số chiều dữ liệu, đồng thời vẫn giữ được những đặc trưng có khả năng phân biệt cao giữa các họ mã độc.
- Ảnh biểu diễn mã độc: Mảng byte được sắp xếp tuần tự thành ma trận vuông hai chiều. Nếu số lượng byte chưa đủ để tạo thành ma trận hoàn chỉnh, hệ thống sẽ tự động bổ sung các giá trị 0 vào cuối mảng. Ma trận sau đó được chuyển thành ảnh grayscale, thay đổi kích thước về 224×224 pixel bằng phép Bilinear

Interpolation và chuyển sang ảnh ba kênh RGB. Ảnh thu được được sử dụng làm đầu vào cho các mô hình học sâu như Vision Transformer (ViT) và Pyramid Vision Transformer (PVT).

Sau bước tiền xử lý, dữ liệu được đưa qua hai bộ phát hiện hoạt động song song. Bộ phát hiện thứ nhất thực hiện phân loại họ mã độc thành chín lớp gồm Ramnit, Lollipop, Kelihos_ver3, Vundo, Simda, Tracur, Kelihos_ver1, Obfuscator.ACY và Gatak. Bộ phát hiện này kết hợp mô hình Random Forest và mô hình phân loại ảnh PVT. Bộ phát hiện thứ hai thực hiện phân loại nhị phân giữa tệp lành tính và mã độc, sử dụng ba mô hình thành phần gồm Random Forest, PVT và Vision Transformer.

Mỗi mô hình thành phần trả về một vector xác suất trên các lớp. Các vector xác suất được chuẩn hóa và đưa vào module ensemble để tạo kết quả cuối cùng. Hệ thống hỗ trợ nhiều phương pháp kết hợp như tổng có trọng số, soft voting, majority voting, trung bình nhân, trung bình điều hòa và trung bình logit. Trong phiên bản triển khai hiện tại, phương pháp mặc định là weighted sum. Lớp có xác suất lớn nhất được chọn làm nhãn dự đoán, đồng thời hệ thống trả về độ tin cậy, xác suất của từng lớp và kết quả riêng của từng mô hình thành phần. Việc trả về đầy đủ các kết quả trung gian hỗ trợ quá trình phân tích và so sánh giữa các mô hình.

Kết quả quét được lưu trong bảng DetectionScan, bao gồm tên tệp, giá trị SHA-256, kích thước tệp, kết quả phân loại nhị phân, họ mã độc dự đoán, độ tin cậy, thời gian xử lý và toàn bộ nội dung kết quả dưới dạng JSON. Nếu người dùng đã đăng nhập, bản ghi được liên kết với tài khoản thông qua trường owner_id. Hệ thống vẫn cho phép người dùng chưa đăng nhập thực hiện quét nhanh, nhưng các chức năng xem lịch sử và thống kê cá nhân yêu cầu xác thực.

Backend cung cấp các API chính như sau:

Bảng 22: Các API chính của backend

Phương thức	Đường dẫn	Chức năng
POST	/api/v1/detections/scan	Tải tệp và thực hiện phát hiện mã độc
GET	/api/v1/detections/history	Lấy lịch sử quét của người dùng
GET	/api/v1/detections/upload-stats	Thống kê số lượng tệp đã quét

GET	/api/v1/detections/dashboard	Cung cấp thông tin mô hình và bộ dữ liệu
POST	/api/v1/login/access-token	Đăng nhập và cấp access token
POST	/api/v1/users/signup	Đăng ký tài khoản
GET	/api/v1/utls/health-check/	Kiểm tra trạng thái hoạt động của backend

Cơ chế xác thực được triển khai theo chuẩn OAuth2 kết hợp với JSON Web Token. Mật khẩu không được lưu trực tiếp mà được băm bằng Argon2 hoặc Bcrypt. Token được ký bằng thuật toán HS256 và được kiểm tra tại các API yêu cầu đăng nhập. Backend cũng áp dụng CORS để giới hạn các nguồn frontend được phép gửi yêu cầu.

Toàn bộ module được đóng gói bằng Docker. Khi khởi động, dịch vụ prestart chờ PostgreSQL sẵn sàng, thực hiện migration cơ sở dữ liệu bằng Alembic và tạo tài khoản quản trị ban đầu. Sau đó, FastAPI được chạy với bốn worker trong môi trường production. Các mô hình đã huấn luyện và dữ liệu tham chiếu được gắn vào container dưới dạng thư mục chỉ đọc. Kiến trúc này giúp tách biệt mã nguồn, mô hình và cơ sở dữ liệu, đồng thời thuận tiện cho việc triển khai và mở rộng hệ thống.

b. Frontend module

Về phía Frontend của hệ thống được xây dựng bằng React, TypeScript và công cụ đóng gói Vite. Giao diện sử dụng Tailwind CSS kết hợp với các thành phần Radix UI, giúp bảo đảm tính nhất quán, khả năng tái sử dụng và tương thích với nhiều kích thước màn hình. Cấu trúc frontend được chia thành các nhóm chính gồm trang giao diện, thành phần dùng chung, lớp giao tiếp API, hook quản lý trạng thái và cấu hình định tuyến.

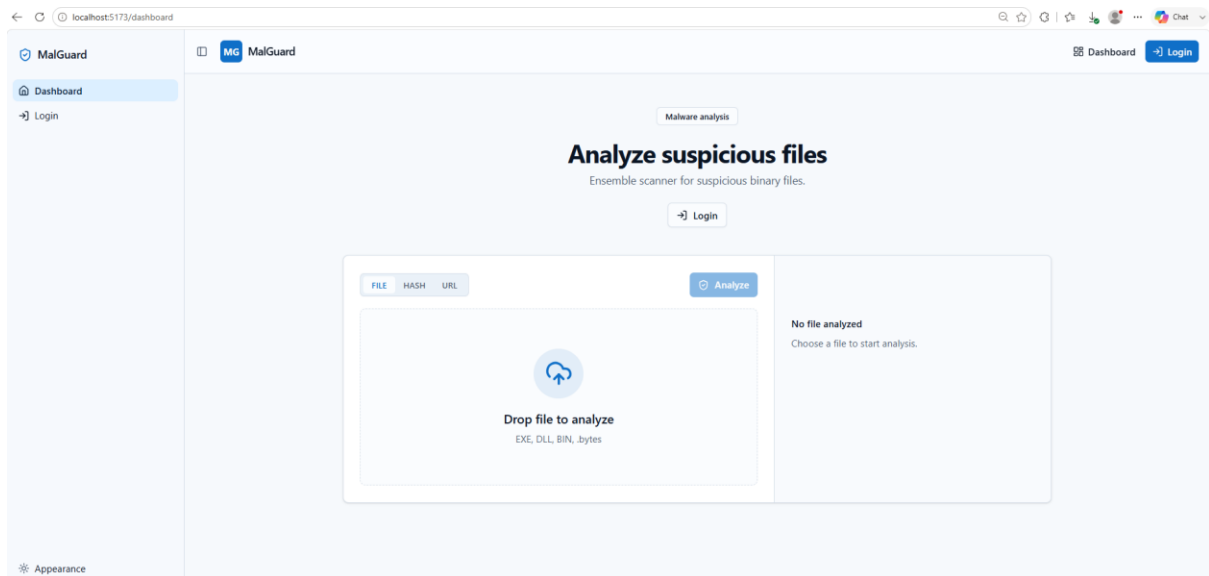
web/frontend/	
src/	
main.tsx	# Điểm khởi tạo React app, router, query client
routeTree.gen.ts	# Cây định tuyến sinh tự động bởi TanStack Router
routes/	
_layout.tsx	# Layout chính, sidebar, header, bảo vệ route
login.tsx	# Giao diện đăng nhập
signup.tsx	# Giao diện đăng ký
_layout/	
index.tsx	# Dashboard chính và quét nhanh tệp
dashboard.tsx	# Route dashboard
detections.tsx	# Giao diện quét mã độc, kết quả, lịch sử, thống kê
client/	
detections.ts	# Service gọi API quét tệp, lịch sử, thống kê upload
dashboard.ts	# Service gọi API thống kê dataset và mô hình

core/	# Hàm request, xử lý lỗi, cấu hình HTTP client
index.ts	# Client sinh từ OpenAPI của backend
components/	
ui/	# Các thành phần giao diện dùng chung
Sidebar/	# Sidebar, menu điều hướng, thông tin người dùng
Common/	# Footer, theme, thành phần chung
Admin/	# Thành phần quản lý người dùng
Items/	# Thành phần mẫu quản lý item
hooks/	
useAuth.ts	# Đăng nhập, đăng ký, đăng xuất, kiểm tra token
useCustomToast.ts	# Hiển thị thông báo lỗi và trạng thái
config/	
project.ts	# Tên và thông tin hiển thị của hệ thống
lib/	
utils.ts	# Hàm tiện ích cho className và giao diện
utils.ts	# Xử lý lỗi API và tiện ích chung
index.css	# Tailwind CSS và style toàn cục
public/	# Tài nguyên tĩnh
tests/	# Kiểm thử end-to-end bằng Playwright
package.json	# Khai báo thư viện và script frontend
vite.config.ts	# Cấu hình Vite, React, Tailwind, router
tsconfig.json	# Cấu hình TypeScript
Dockerfile	# Đóng gói frontend thành Docker image

Hình 9: Cấu trúc thư mục frontend hệ thống

Cấu trúc frontend ở *hình 9* được tổ chức theo mô hình tách biệt giữa trang giao diện, thành phần tái sử dụng, lớp giao tiếp API và hook quản lý trạng thái. Thư mục routes định nghĩa các trang chính của hệ thống, trong đó detections.tsx là nơi triển khai giao diện tải tệp, gửi yêu cầu phân tích, hiển thị kết quả dự đoán, lịch sử quét và thống kê upload. Thư mục client đóng vai trò trung gian giữa frontend và backend, giúp chuẩn hóa các lời gọi API và kiểu dữ liệu trả về. Các thành phần giao diện dùng chung được đặt trong components, giúp giảm lặp mã và giữ giao diện nhất quán. Phần xác thực người dùng được quản lý trong hooks/useAuth.ts, kết hợp với localStorage để lưu JWT và tự động gắn token vào các yêu cầu gửi đến backend.

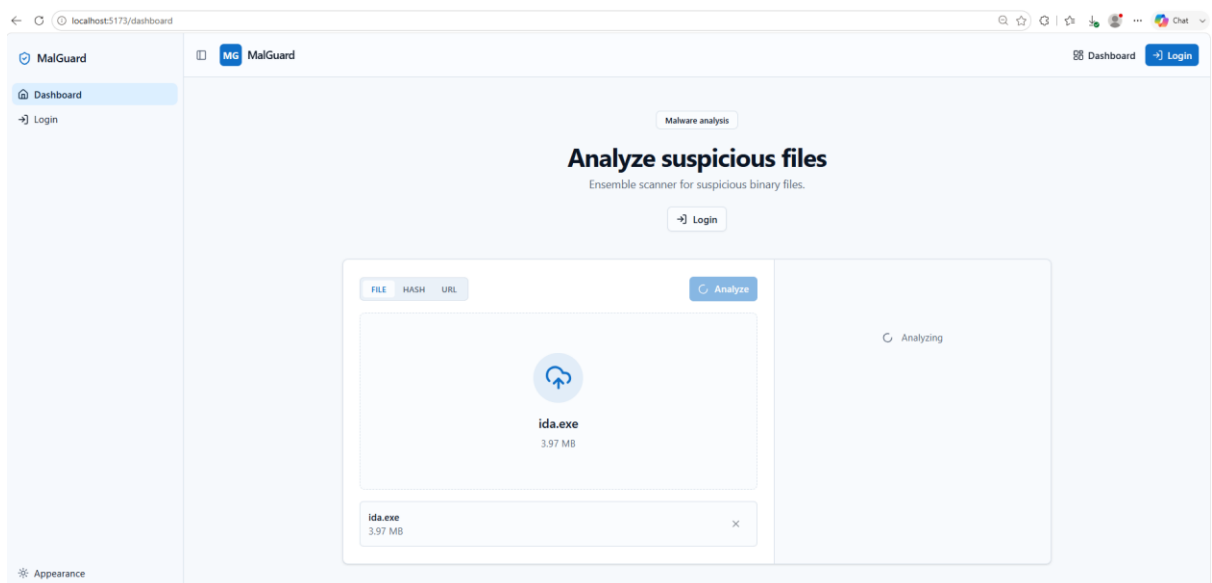
Việc điều hướng giữa các trang được quản lý bằng TanStack Router. Hệ thống cung cấp trang tổng quan công khai cho phép người dùng quét nhanh tệp mà không cần đăng nhập. Các chức năng nâng cao như xem báo cáo đầy đủ, lịch sử quét và thống kê cá nhân chỉ được truy cập sau khi người dùng xác thực thành công. Trước khi tải một trang được bảo vệ, frontend kiểm tra sự tồn tại của access token và tự động chuyển người dùng đến trang đăng nhập nếu chưa xác thực.



Hình 10: Giao diện website để tải và phân tích tệp

Chức năng quan trọng nhất của frontend là giao diện tải lên và phân tích tệp ở hình 10. Người dùng có thể chọn tệp từ thiết bị hoặc kéo thả tệp trực tiếp vào vùng tải lên. Giao diện hỗ trợ các tệp thực thi và dữ liệu nhị phân như EXE, DLL, BIN và .bytes. Sau khi tệp được lựa chọn, frontend hiển thị tên và kích thước tệp để người dùng kiểm tra trước khi gửi yêu cầu phân tích.

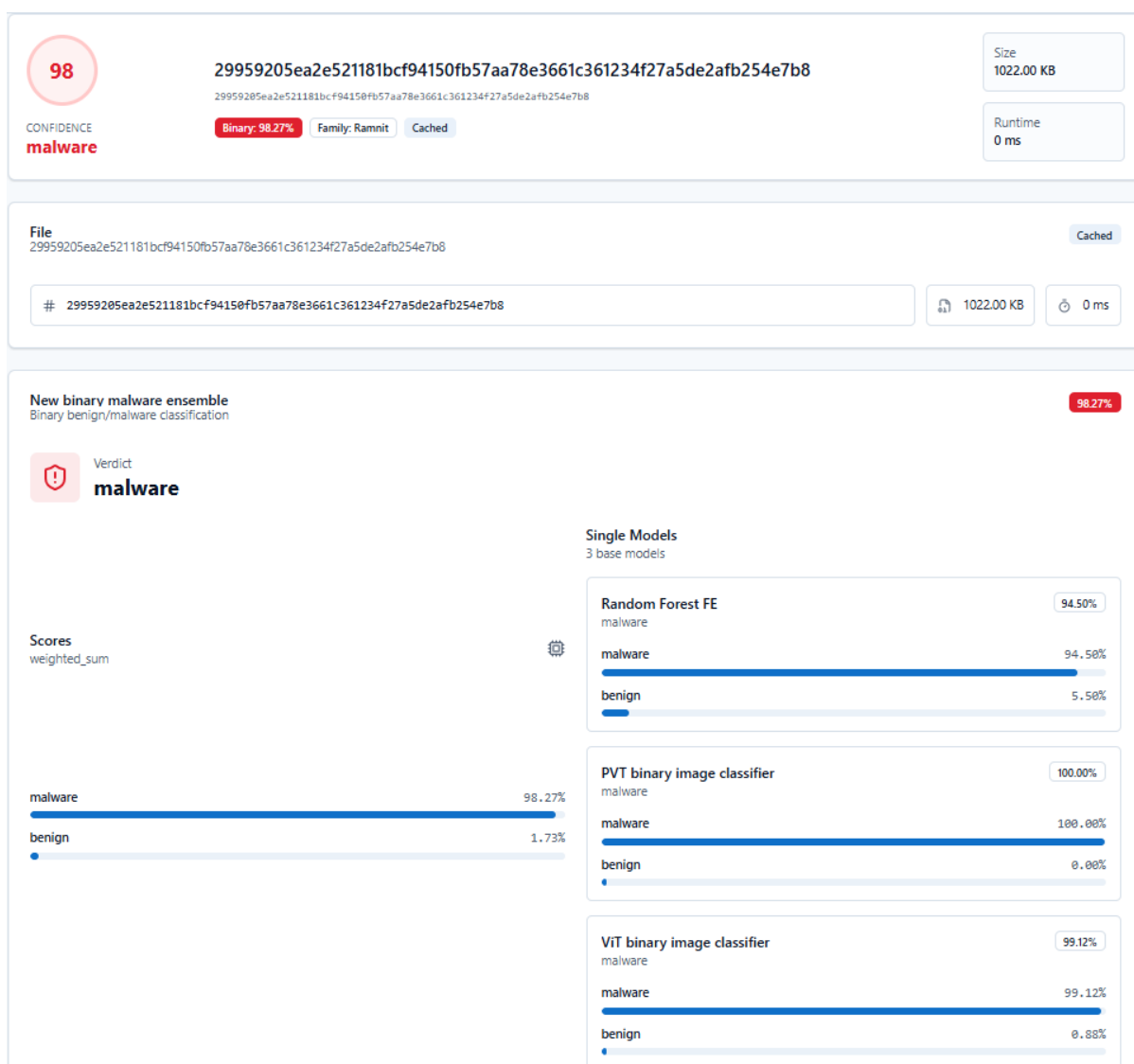
Khi người dùng nhấn nút phân tích, tệp được đóng gói dưới dạng multipart/form-data và gửi đến API: POST /api/v1/detections/scan



Hình 11: Giao diện website khi đang phân tích tệp

Trong thời gian backend thực hiện tiền xử lý và suy luận mô hình, giao diện hiển thị trạng thái đang phân tích và vô hiệu hóa nút gửi nhằm tránh tạo nhiều yêu cầu trùng lặp

(hình 11). Nếu xảy ra lỗi như tệp không hợp lệ, vượt quá giới hạn dung lượng hoặc mô hình không thể suy luận, frontend tiếp nhận mã lỗi từ backend và hiển thị thông báo cho người dùng.



Hình 12: Giao diện website khi phát hiện mã độc

Kết quả trả về được trực quan hóa thành nhiều nhóm thông tin (hình 12). Phần tổng quan hiển thị nhãn phân loại nhị phân là benign hoặc malware, độ tin cậy, họ mã độc dự đoán, kích thước tệp, mã băm SHA-256 và thời gian xử lý. Màu xanh được sử dụng cho tệp lành tính, trong khi màu đỏ biểu diễn tệp bị phát hiện là mã độc. Nếu kết quả được lấy từ bộ nhớ đệm, giao diện hiển thị thêm nhãn “Cached” để người dùng nhận biết.

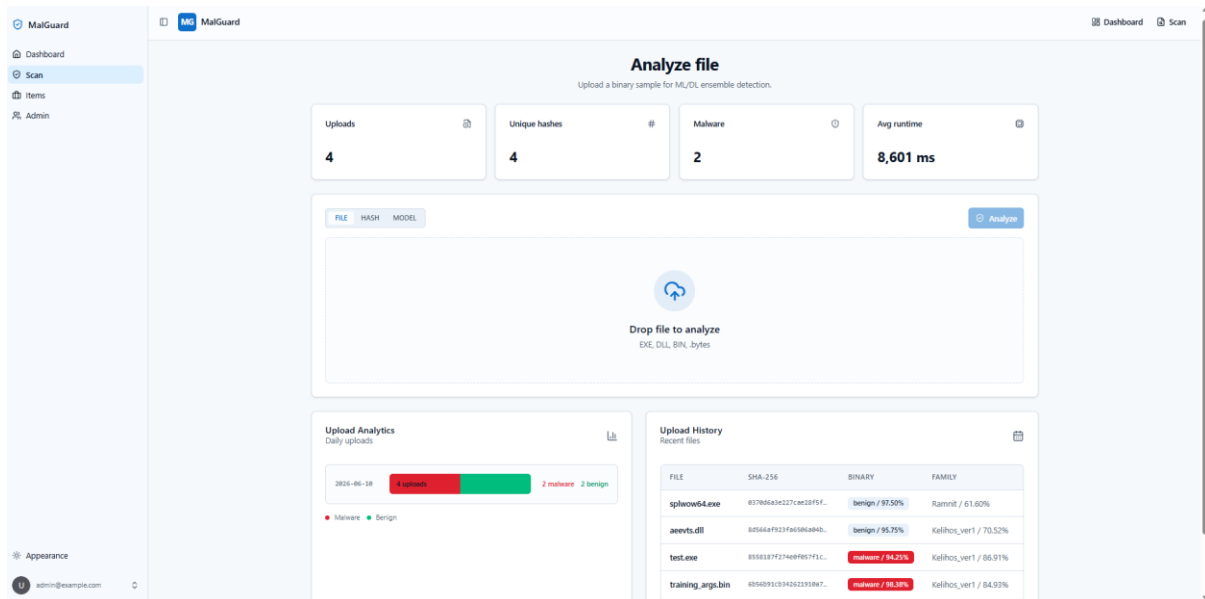
Phần báo cáo chi tiết trình bày kết quả của hai bộ phát hiện:

- Bộ phân loại nhị phân giữa tệp lành tính và mã độc.

- Bộ phân loại họ mã độc gồm chín lớp.

Đối với mỗi bộ phát hiện, frontend hiển thị nhãn dự đoán cuối cùng, độ tin cậy và phân phối xác suất của các lớp dưới dạng thanh tiến trình. Kết quả riêng của từng mô hình Random Forest, PVT và ViT cũng được trình bày để hỗ trợ so sánh. Ngoài ra, giao diện cung cấp bảng kết quả của các phương pháp ensemble như weighted sum, soft voting, majority voting, geometric mean và harmonic mean. Qua đó, người dùng có thể quan sát sự khác biệt giữa các chiến lược kết hợp mô hình.

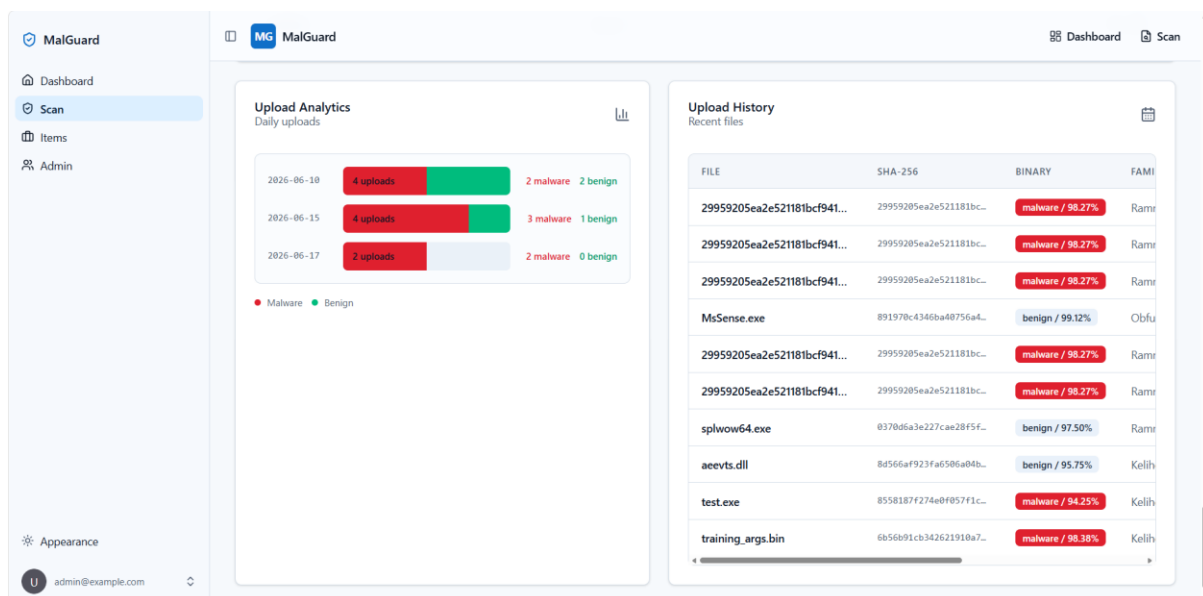
Frontend sử dụng TanStack Query để quản lý dữ liệu bất đồng bộ. Các thao tác lấy dữ liệu được tổ chức thành query, trong khi quá trình gửi tệp được quản lý dưới dạng mutation. Sau khi quét thành công, frontend tự động làm mới dữ liệu lịch sử và thống kê mà không cần tải lại toàn bộ trang.



Hình 13: Giao diện website thống kê cá nhân

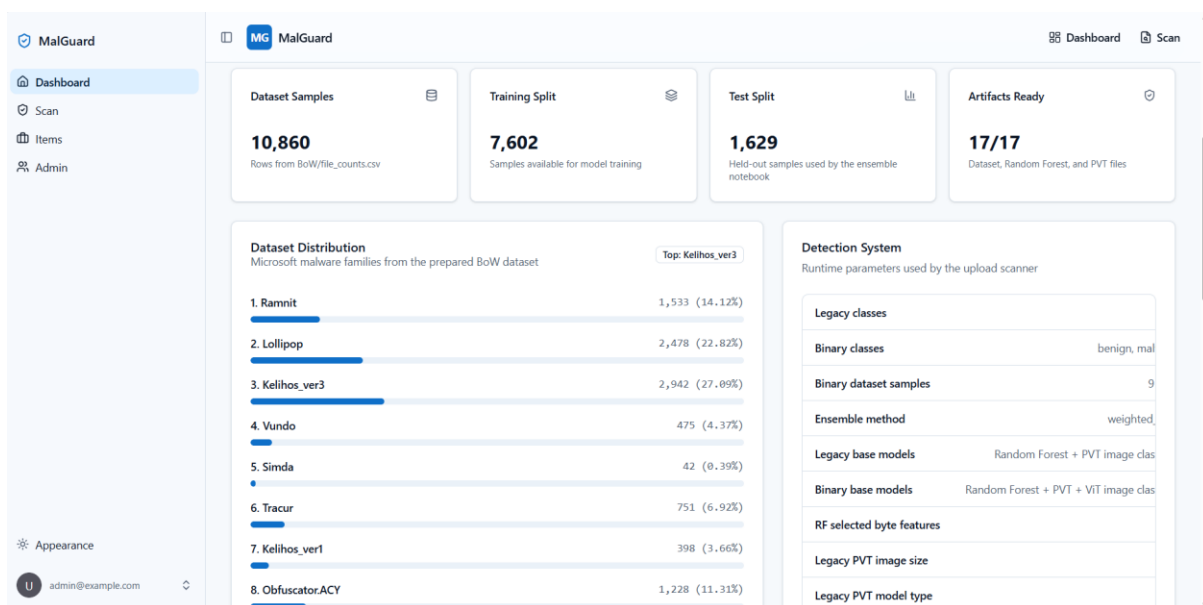
Trang thống kê cá nhân cung cấp các thông tin (hình 13):

- Tổng số lượt tải tệp.
- Số lượng tệp duy nhất theo SHA-256.
- Số tệp được phát hiện là mã độc và lành tính.
- Thời gian xử lý trung bình.
- Xu hướng tải tệp theo ngày.
- Danh sách các tệp được quét gần đây.



Hình 14: Giao diện lịch sử quét

Lịch sử quét được trình bày dưới dạng bảng ở hình 14, bao gồm tên tệp, mã SHA-256, kết quả phân loại nhị phân, họ mã độc, kích thước và thời điểm quét. Biểu đồ xu hướng sử dụng các thanh màu để phân biệt số lượng tệp mã độc và tệp lành tính theo từng ngày.



Hình 15: Giao diện dashboard

Trang dashboard còn hiển thị thông tin về bộ dữ liệu và trạng thái của hệ thống học máy, bao gồm số lượng mẫu, phân bố các họ mã độc, số lượng mẫu trong tập huấn luyện và kiểm thử, phương pháp ensemble đang sử dụng và trạng thái tồn tại của các tệp mô hình. Những thông tin này giúp kiểm tra nhanh khả năng sẵn sàng của hệ thống.

Lớp giao tiếp API được xây dựng dựa trên lược đồ OpenAPI của backend. Các kiểu dữ liệu phản hồi được khai báo bằng TypeScript, giúp phát hiện lỗi kiểu dữ liệu trong quá trình phát triển. Địa chỉ backend được cấu hình thông qua biến môi trường `VITE_API_URL`, nhờ đó frontend có thể kết nối với các môi trường phát triển, kiểm thử hoặc production mà không cần thay đổi mã nguồn.

CHƯƠNG 4: THỰC NGHIỆM VÀ ĐÁNH GIÁ

4.1. Môi trường thực nghiệm

Các thí nghiệm trong nghiên cứu được thực hiện trên hai môi trường khác nhau. Quá trình huấn luyện và đánh giá mô hình được thực hiện trên nền tảng Kaggle Notebook nhằm tận dụng tài nguyên tính toán hiệu năng cao, đặc biệt là GPU NVIDIA Tesla P100. Sau khi hoàn thành huấn luyện, các mô hình được triển khai thành hệ thống phát hiện và phân loại mã độc trên máy tính cá nhân để đánh giá khả năng vận hành trong điều kiện thực tế.

Bảng 23: Cấu hình môi trường huấn luyện

Thành phần	Cấu hình
Nền tảng	Kaggle Notebook
Hệ điều hành	Linux (môi trường Kaggle)
CPU	Intel Xeon đa nhân
GPU	NVIDIA Tesla P100
Bộ nhớ GPU	16 GB HBM2
RAM hệ thống	Khoảng 13 GB
Ngôn ngữ lập trình	Python 3
Thư viện học máy	Scikit-learn
Thư viện học sâu	PyTorch, TensorFlow
Thư viện Transformer	Hugging Face Transformers
Mục đích sử dụng	Huấn luyện, tinh chỉnh và đánh giá các mô hình Random Forest, ViT và PVT

Đối với quá trình huấn luyện và đánh giá mô hình học máy, học sâu, nghiên cứu sử dụng nền tảng Kaggle Notebook. Môi trường này cung cấp GPU NVIDIA Tesla P100 với 16 GB bộ nhớ đồ họa, hỗ trợ tăng tốc đáng kể cho quá trình huấn luyện các mô hình học sâu như Vision Transformer (ViT) và Pyramid Vision Transformer (PVT). Ngoài ra, Kaggle còn cung cấp CPU đa nhân Intel Xeon, bộ nhớ RAM khoảng 13 GB và môi trường Python được cài đặt sẵn các thư viện phổ biến như TensorFlow, PyTorch, Scikit-learn và Transformers, tạo điều kiện thuận lợi cho việc xây dựng, huấn luyện và đánh giá các mô hình trong nghiên cứu.

Bảng 24: Cấu hình môi trường triển khai

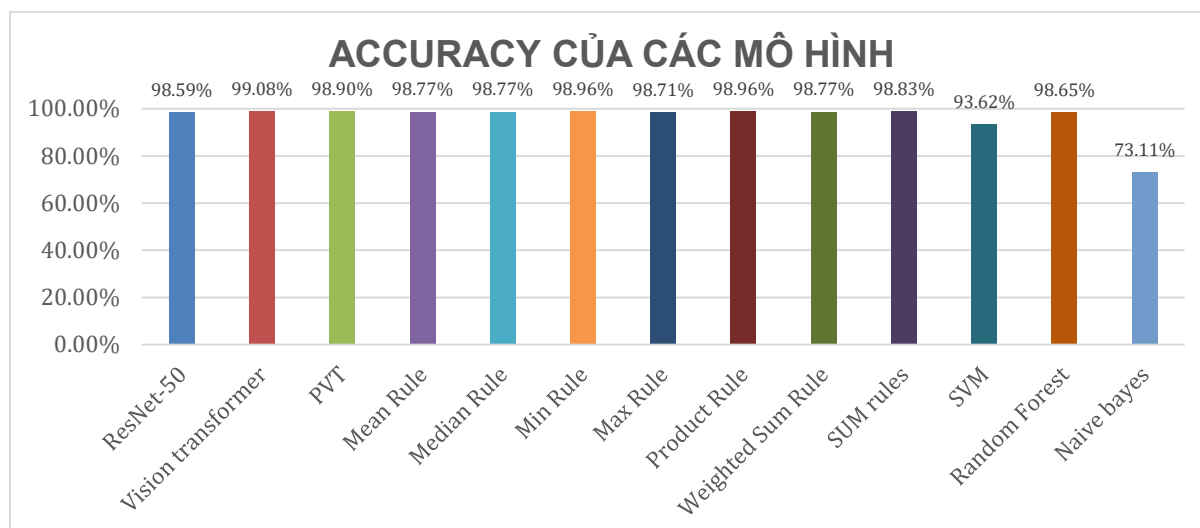
Thành phần	Cấu hình
Thiết bị	MSI GF63 Thin 10SC
Hệ điều hành	Microsoft Windows 11 Enterprise (Build 26200)
CPU	Intel Core i7-10750H @ 2.60 GHz
Số nhân / luồng	6 nhân, 12 luồng
GPU	NVIDIA GeForce GTX 1650 Max-Q Design
Bộ nhớ RAM	16 GB
Kiến trúc hệ thống	x64-based PC
Backend	FastAPI (Python)
Frontend	React + TypeScript
Cơ sở dữ liệu	PostgreSQL
Công cụ đóng gói	Docker
Mục đích sử dụng	Triển khai và vận hành hệ thống phát hiện, phân loại mã độc thông qua giao diện web

Sau khi hoàn thành quá trình huấn luyện, các mô hình được triển khai thành hệ thống phát hiện và phân loại mã độc trên máy tính cá nhân. Môi trường triển khai sử dụng hệ điều hành Microsoft Windows 11 Enterprise (phiên bản 10.0.26200 Build 26200) trên máy tính xách tay MSI GF63 Thin 10SC. Thiết bị được trang bị bộ vi xử lý Intel Core i7-10750H với 6 nhân và 12 luồng xử lý, cùng 16 GB RAM. Backend của hệ thống được phát triển bằng Python và FastAPI, trong khi frontend được xây dựng bằng React và TypeScript. Việc triển khai trên môi trường phần cứng phổ thông nhằm đánh giá khả năng ứng dụng thực tế của hệ thống cũng như thời gian phản hồi khi thực hiện phân tích và phân loại các tệp thực thi.

4.2. Kết quả

Sau khi hoàn thành quá trình huấn luyện và xây dựng mô hình, các thí nghiệm được tiến hành trên tập kiểm thử của bộ dữ liệu Microsoft Malware Classification Challenge 2015 nhằm đánh giá hiệu quả của các mô hình học máy, học sâu và các phương pháp Ensemble Learning được đề xuất. Các mô hình được đánh giá dựa trên các chỉ số Accuracy, Precision, Recall và F1-score. Bên cạnh đó, nghiên cứu cũng thực hiện so

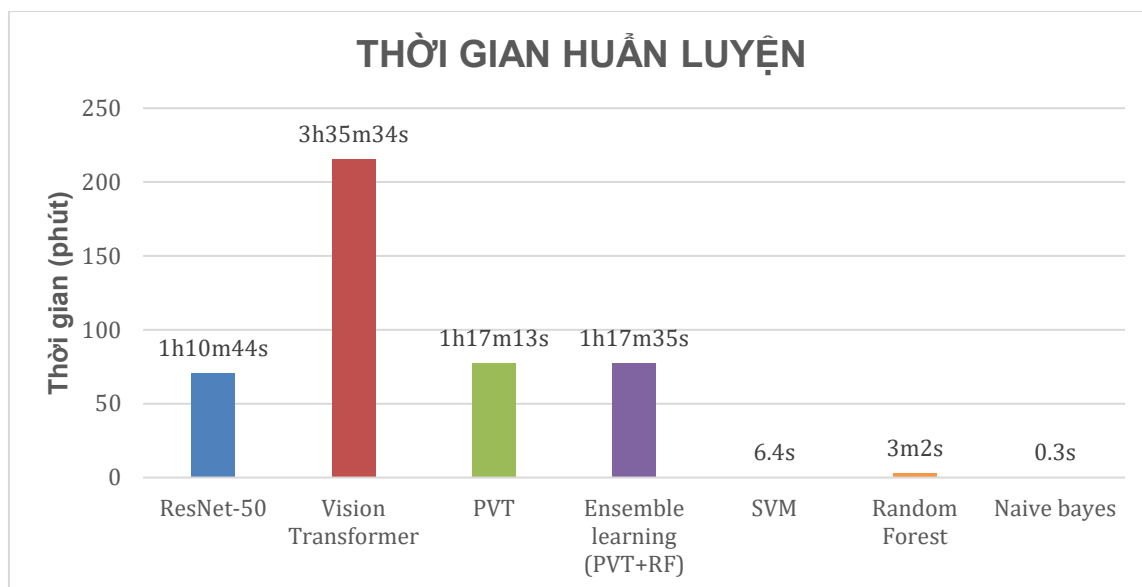
sánh kết quả đạt được với một số công trình liên quan nhằm đánh giá mức độ hiệu quả của phương pháp đề xuất.



Hình 16: Kết quả của các mô hình

Hình 16 trình bày kết quả đánh giá của các mô hình học máy, học sâu và các phương pháp Ensemble Learning. Kết quả cho thấy các mô hình học sâu đạt hiệu suất vượt trội so với các mô hình học máy truyền thống. Trong đó, Vision Transformer đạt kết quả cao nhất với Accuracy 99,08%, tiếp theo là PVT với Accuracy 98,90% và ResNet-50 với Accuracy 98,59%. Đối với nhóm học máy, Random Forest đạt Accuracy 98,65%, cao hơn đáng kể so với SVM (93,62%) và Naive Bayes (73.11%).

Các phương pháp Ensemble Learning đều cho kết quả ổn định với Accuracy trên 98,7%. Trong đó, Product Rule và Min Rule đạt hiệu suất tốt nhất với Accuracy 98,96%, Precision 98,97%, Recall 98,96% và F1-score 98,96%. Sum Rule đạt Accuracy 98,83%, trong khi Mean Rule, Median Rule và Weighted Sum Rule cùng đạt Accuracy 98,77%. Kết quả này cho thấy việc kết hợp các mô hình giúp cải thiện độ ổn định của hệ thống và khai thác hiệu quả thông tin bổ sung giữa các mô hình thành phần.



Hình 17: Thời gian huấn luyện của các mô hình

Tiếp theo, *hình 17* trình bày thời gian huấn luyện của các mô hình được sử dụng trong nghiên cứu. Kết quả cho thấy các mô hình học máy truyền thống như Naive Bayes, SVM và Random Forest có thời gian huấn luyện ngắn hơn đáng kể so với các mô hình học sâu. Ngược lại, các mô hình học sâu như ResNet-50, Vision Transformer và PVT yêu cầu thời gian huấn luyện dài hơn do số lượng tham số lớn và quá trình tối ưu phức tạp. Tuy nhiên, sự gia tăng chi phí tính toán này được bù đắp bằng hiệu suất phân loại vượt trội.

Bảng 25: Các nghiên cứu sử dụng tập dữ liệu BIG 2015

#	Tác giả	Mô hình	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)
1	S. Abijah Roseline [14]	Mixed forest model kết hợp các thuật toán: Random Forest, Completely Random Forests và XGBoost.	97.2	97.61	96.79	97.20
2	Sanjeev Kumar [24]	ResNet-50	94.85	95	94	95
3	Sachith Seneviratne [27]	Vision Transformer	97.33	95.41	92.27	93.49
4	Geeta Gayatri Behera [34]	Ensemble Learning (VGG-16 + ResNet50)	99.15	99	99	99
5	Nghiên cứu này	Ensemble Learning (PVT + RF)	98.77	98.80	98.77	98.77

Để đánh giá khách quan hơn hiệu quả của phương pháp đề xuất, *bảng 25* thực hiện so sánh kết quả với một số nghiên cứu tiêu biểu trên cùng bộ dữ liệu Microsoft Malware

Classification Challenge 2015. Kết quả cho thấy phương pháp đề xuất đạt Accuracy 98,96%, vượt trội hơn các nghiên cứu sử dụng Mixed Forest Model (97,20%), LightGBM (65%), Vision Transformer (97,33%) và ResNet-50 (98,77%). Mặc dù chưa vượt qua nghiên cứu của Geeta Gayatri Behera với Accuracy 99,15%, mô hình đề xuất vẫn đạt các chỉ số Precision, Recall và F1-score ở mức rất cao. Điều này chứng tỏ việc kết hợp đặc trưng ảnh được học bởi PVT với đặc trưng thống kê byte được khai thác bởi Random Forest thông qua các phương pháp Ensemble Learning đã góp phần nâng cao hiệu quả phát hiện và phân loại mã độc.

Nhìn chung, các kết quả thực nghiệm cho thấy phương pháp đề xuất không chỉ đạt hiệu suất phân loại cao mà còn duy trì được tính ổn định trên tập dữ liệu kiểm thử. Sự kết hợp giữa mô hình học sâu và học máy truyền thống giúp tận dụng ưu điểm của từng phương pháp, trong khi các kỹ thuật Ensemble Learning đóng vai trò quan trọng trong việc cải thiện khả năng tổng quát hóa và giảm sai số của hệ thống.

4.3. Đánh giá

Kết quả thực nghiệm cho thấy các mô hình học sâu có khả năng phân loại mã độc hiệu quả hơn so với các mô hình học máy truyền thống. Trong số các mô hình được khảo sát, Vision Transformer đạt độ chính xác cao nhất với 99,08%, chứng tỏ khả năng khai thác đặc trưng từ ảnh biểu diễn mã độc một cách hiệu quả. Bên cạnh đó, PVT cũng đạt kết quả rất cao với Accuracy 98,90%, trong khi Random Forest vẫn duy trì hiệu suất tốt trên tập đặc trưng byte với Accuracy 98,65%. Điều này cho thấy cả hai hướng tiếp cận dựa trên đặc trưng ảnh và đặc trưng thống kê byte đều có khả năng nhận diện đặc trưng của các họ mã độc trong bộ dữ liệu Microsoft Malware Classification Challenge 2015.

Đối với các phương pháp Ensemble Learning, kết quả cho thấy việc kết hợp các mô hình thành viên giúp duy trì độ chính xác ở mức cao và tăng tính ổn định của hệ thống. Trong số các phương pháp được đánh giá, Product Rule và Min Rule đạt kết quả tốt nhất với Accuracy 98,96%, cao hơn các phương pháp Mean Rule, Median Rule, Weighted Sum Rule và Max Rule. Điều này cho thấy việc khai thác mức độ đồng thuận giữa các mô hình thành viên đóng vai trò quan trọng trong quá trình ra quyết định. Tuy nhiên, kết quả cũng cho thấy các phương pháp ensemble chưa vượt qua được Vision Transformer đơn lẻ. Nguyên nhân có thể xuất phát từ việc ViT đã đạt hiệu suất rất cao,

trong khi Random Forest có độ chính xác thấp hơn nên việc kết hợp chưa tạo ra mức cải thiện đáng kể.

Một điểm đáng chú ý là các mô hình học sâu yêu cầu thời gian huấn luyện và tài nguyên tính toán lớn hơn so với các mô hình học máy truyền thống. Mặc dù vậy, sau khi hoàn thành quá trình huấn luyện, thời gian suy luận của hệ thống vẫn đáp ứng tốt yêu cầu triển khai thực tế. Việc xây dựng hệ thống dưới dạng ứng dụng web cho phép người dùng tải tệp và nhận kết quả phân tích một cách trực quan, góp phần nâng cao khả năng ứng dụng của nghiên cứu.

Từ các kết quả đạt được có thể nhận thấy rằng phương pháp kết hợp giữa đặc trưng ảnh và đặc trưng byte mang lại hiệu quả phân loại cao, đồng thời giúp hệ thống tận dụng được ưu điểm của nhiều mô hình khác nhau. Kết quả so sánh với các nghiên cứu trước đây cũng cho thấy phương pháp đề xuất đạt hiệu suất cạnh tranh trên bộ dữ liệu Microsoft Malware Classification Challenge 2015. Điều này chứng minh tính khả thi của hướng tiếp cận sử dụng học sâu kết hợp Ensemble Learning trong bài toán phát hiện và phân loại mã độc.

PHẦN BA: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

1. Kết quả nghiên cứu của đề tài

Trong khuôn khổ đề tài, nghiên cứu đã hoàn thành việc xây dựng một hệ thống phát hiện và phân loại mã độc dựa trên sự kết hợp giữa các mô hình học máy, học sâu và kỹ thuật Ensemble Learning. Trên cơ sở nghiên cứu lý thuyết về mã độc, phân tích tĩnh, học máy và học sâu, đề tài đã xây dựng quy trình xử lý dữ liệu hoàn chỉnh từ giai đoạn thu thập, tiền xử lý, huấn luyện mô hình, đánh giá kết quả đến triển khai ứng dụng thực tế.

Về mặt dữ liệu, đề tài đã sử dụng bộ dữ liệu Microsoft Malware Classification Challenge Dataset (2015) để xây dựng bài toán phân loại đa lớp gồm 9 họ mã độc. Đồng thời, nhằm nâng cao khả năng ứng dụng trong thực tế, nghiên cứu đã mở rộng bộ dữ liệu bằng cách thu thập thêm 25.464 tệp lành tính từ hệ điều hành Windows và 65.536 mẫu mã độc từ VirusShare để xây dựng bộ phân loại nhị phân giữa tệp lành tính và mã độc. Việc kết hợp dữ liệu nghiên cứu và dữ liệu thực tế giúp tăng tính đa dạng của tập dữ liệu và nâng cao khả năng tổng quát hóa của mô hình.

Về mặt mô hình, đề tài đã xây dựng và đánh giá nhiều phương pháp khác nhau bao gồm Naive Bayes, SVM, Random Forest, ResNet-50, Vision Transformer (ViT) và Pyramid Vision Transformer (PVT). Kết quả thực nghiệm cho thấy các mô hình học sâu đạt hiệu suất vượt trội so với các mô hình học máy truyền thống, trong đó Vision Transformer đạt độ chính xác cao nhất với Accuracy 99,08%, trong khi PVT và Random Forest lần lượt đạt Accuracy 98,90% và 98,65%.

Bên cạnh đó, đề tài đã nghiên cứu và triển khai nhiều phương pháp Ensemble Learning như Sum Rule, Weighted Sum Rule, Product Rule, Max Rule, Min Rule, Mean Rule và Median Rule nhằm kết hợp kết quả dự đoán từ các mô hình thành viên. Kết quả thực nghiệm cho thấy Product Rule và Min Rule đạt hiệu suất tốt nhất với Accuracy 98,96%, Precision 98,97%, Recall 98,96% và F1-score 98,96%. Điều này cho thấy việc kết hợp nhiều mô hình giúp nâng cao tính ổn định và khả năng tổng quát hóa của hệ thống trong quá trình phân loại mã độc.

Ngoài việc đánh giá trên tập dữ liệu nghiên cứu, đề tài còn tiến hành so sánh kết quả với các công trình liên quan trên cùng bộ dữ liệu Microsoft Malware Classification Challenge 2015. Kết quả cho thấy phương pháp đề xuất đạt hiệu suất cạnh tranh và vượt

trội hơn nhiều nghiên cứu sử dụng các mô hình học máy hoặc học sâu đơn lẻ, qua đó khẳng định tính hiệu quả của hướng tiếp cận kết hợp giữa học sâu và Ensemble Learning.

Một kết quả quan trọng khác của đề tài là việc xây dựng và triển khai thành công hệ thống phát hiện, phân loại mã độc dưới dạng ứng dụng web hoàn chỉnh. Hệ thống được phát triển với backend sử dụng FastAPI, frontend sử dụng React và TypeScript, kết hợp cơ sở dữ liệu PostgreSQL để quản lý người dùng và lưu trữ lịch sử quét. Ứng dụng cho phép người dùng tải lên tệp thực thi, thực hiện phân tích tự động, hiển thị kết quả phân loại mã độc, xác suất dự đoán của từng lớp và thông tin chi tiết về quá trình suy luận của mô hình. Kết quả triển khai cho thấy hệ thống hoạt động ổn định, đáp ứng tốt yêu cầu phân tích và hỗ trợ người dùng trong quá trình nhận diện mã độc.

Nhìn chung, đề tài đã hoàn thành các mục tiêu đặt ra, từ nghiên cứu cơ sở lý thuyết, xây dựng và đánh giá mô hình đến triển khai thành công hệ thống thực tế. Các kết quả đạt được cho thấy tính khả thi của việc ứng dụng các kỹ thuật học sâu kết hợp Ensemble Learning trong bài toán phát hiện và phân loại mã độc, đồng thời tạo nền tảng cho các nghiên cứu và ứng dụng mở rộng trong tương lai.

2. Hạn chế của sản phẩm

Mặc dù đã đạt được những kết quả khả quan trong việc phát hiện và phân loại mã độc, sản phẩm vẫn còn một số hạn chế nhất định về dữ liệu, tài nguyên tính toán và phạm vi triển khai có thể kể đến như:

Thứ nhất, nguồn tài nguyên phần cứng phục vụ nghiên cứu còn hạn chế so với các hệ thống thương mại hoặc các trung tâm nghiên cứu chuyên về an toàn thông tin. Quá trình huấn luyện mô hình chủ yếu được thực hiện trên nền tảng Kaggle với GPU NVIDIA Tesla P100, trong khi việc triển khai hệ thống được thực hiện trên máy tính cá nhân. Do giới hạn về tài nguyên tính toán và dung lượng lưu trữ, nghiên cứu chưa thể thu thập và huấn luyện trên các tập dữ liệu có quy mô hàng chục triệu mẫu như các hệ thống phát hiện mã độc thực tế của các hãng bảo mật lớn. Điều này có thể ảnh hưởng đến khả năng bao phủ đối với các biến thể mã độc mới hoặc các mẫu hiếm gặp trong môi trường thực tế.

Thứ hai, việc thu thập dữ liệu mã độc gặp nhiều khó khăn do các vấn đề liên quan đến an toàn, bảo mật và pháp lý. Phần lớn các nguồn dữ liệu mã độc uy tín đều áp dụng

các chính sách kiểm soát truy cập nghiêm ngặt nhằm hạn chế nguy cơ phát tán mã độc ra cộng đồng. Do đó, việc tiếp cận và thu thập số lượng lớn mẫu mã độc phục vụ nghiên cứu không phải lúc nào cũng thuận lợi. Ngoài ra, nhiều cơ sở dữ liệu chia sẻ mã độc chỉ cung cấp tệp mẫu mà không đi kèm thông tin phân loại chi tiết về họ mã độc hoặc hành vi của chúng, dẫn đến khó khăn trong quá trình gán nhãn và xây dựng tập dữ liệu chất lượng cao.

Thứ ba, hệ thống hiện tại chủ yếu dựa trên phương pháp phân tích tĩnh, khai thác đặc trưng từ dữ liệu byte và ảnh được tạo từ tệp thực thi. Cách tiếp cận này giúp giảm chi phí xử lý và thời gian phân tích nhưng có thể gặp hạn chế đối với các mẫu mã độc sử dụng kỹ thuật mã hóa, đóng gói, làm rối mã hoặc các kỹ thuật né tránh phân tích hiện đại. Việc chưa kết hợp các đặc trưng động từ hành vi thực thi của chương trình có thể làm giảm khả năng phát hiện một số loại mã độc phức tạp.

Cuối cùng, mặc dù hệ thống đã được triển khai dưới dạng ứng dụng web và có khả năng hoạt động trên dữ liệu thực tế, phạm vi thử nghiệm vẫn chủ yếu tập trung trên các tập dữ liệu nghiên cứu và tập dữ liệu mở rộng được thu thập trong quá trình thực hiện đề tài. Do đó, hiệu quả của hệ thống trong các môi trường doanh nghiệp hoặc trên các tập dữ liệu có quy mô rất lớn cần được tiếp tục đánh giá và kiểm chứng trong các nghiên cứu tiếp theo.

3. Hướng phát triển

Kết quả đạt được từ đề tài cho thấy tiềm năng của việc ứng dụng học sâu và Ensemble Learning trong bài toán phát hiện, phân loại mã độc. Tuy nhiên, để nâng cao hơn nữa hiệu quả và khả năng ứng dụng thực tế của hệ thống, một số hướng phát triển có thể được xem xét trong tương lai.

Hướng phát triển quan trọng nhất là mở rộng tập dữ liệu huấn luyện. Trong nghiên cứu này, việc thu thập dữ liệu mã độc còn gặp nhiều khó khăn do hạn chế về nguồn dữ liệu công khai, các vấn đề liên quan đến pháp lý cũng như chính sách chia sẻ của các kho lưu trữ mã độc. Trong tương lai, nếu có điều kiện tiếp cận được số lượng lớn hơn các mẫu mã độc và tệp lành tính, đặc biệt là các mẫu được phân loại rõ ràng theo từng họ hoặc từng nhóm hành vi, chất lượng của mô hình có thể được cải thiện đáng kể. Tập dữ liệu đa dạng hơn cũng giúp hệ thống thích ứng tốt hơn với các biến thể mã độc mới xuất hiện trong thực tế.

Ngoài ra, nghiên cứu hiện tại chủ yếu sử dụng các đặc trưng thu được từ phân tích tĩnh. Do đó, việc kết hợp thêm các đặc trưng hành vi từ phân tích động có thể giúp nâng cao khả năng phát hiện đối với những mã độc sử dụng kỹ thuật đóng gói, mã hóa hoặc làm rối mã để né tránh quá trình phân tích. Việc kết hợp đồng thời nhiều nguồn đặc trưng cũng là một hướng tiềm năng để cải thiện độ chính xác của hệ thống.

Về mặt mô hình, có thể tiếp tục thử nghiệm các kiến trúc học sâu mới hoặc các phương pháp kết hợp mô hình khác nhằm tìm kiếm giải pháp có hiệu suất cao hơn. Đồng thời, việc tối ưu hóa mô hình để giảm thời gian suy luận và tài nguyên sử dụng cũng là yếu tố cần thiết khi triển khai trên các hệ thống thực tế.

Đối với ứng dụng web, hệ thống có thể được mở rộng theo hướng hỗ trợ quét thời gian thực, quản lý tập trung nhiều người dùng, lưu trữ và phân tích lịch sử quét, cũng như tự động cập nhật mô hình khi xuất hiện các mẫu mã độc mới. Điều này sẽ giúp hệ thống tiệm cận hơn với các giải pháp phát hiện mã độc đang được sử dụng trong thực tế.

Tổng kết lại, với việc tiếp tục mở rộng dữ liệu, cải thiện mô hình và hoàn thiện hệ thống triển khai, giải pháp được đề xuất có thể trở thành nền tảng cho các nghiên cứu và ứng dụng thực tiễn trong lĩnh vực phát hiện và phân loại mã độc.

TÀI LIỆU THAM KHẢO

- [1] D. Gibert, C. Mateu, and J. Planes, “The rise of machine learning for detection and classification of malware: Research developments, trends and challenges,” Mar. 01, 2020, *Academic Press*. doi: 10.1016/j.jnca.2019.102526.
- [2] A. Jain, Z. Zhou, and U. Guin, “Survey of recent developments for hardware Trojan detection,” in *Proceedings - IEEE International Symposium on Circuits and Systems*, Institute of Electrical and Electronics Engineers Inc., 2021. doi: 10.1109/ISCAS51556.2021.9401143.
- [3] Z. Li, A. L. G. Rios, and L. Trajkovic, “Detecting Internet Worms, Ransomware, and Blackouts Using Recurrent Neural Networks,” in *Conference Proceedings - IEEE International Conference on Systems, Man and Cybernetics*, Institute of Electrical and Electronics Engineers Inc., Oct. 2020, pp. 2165–2172. doi: 10.1109/SMC42975.2020.9283472.
- [4] S. Kim, J. Park, K. Lee, I. You, and K. Yim, “A Brief Survey on Rootkit Techniques in Malicious Codes.” [Online]. Available: <http://lisa.sch.ac>.
- [5] *2020 6th International Conference on Web Research (ICWR)*. IEEE, 2020.
- [6] L. Maffia, D. Nisi, P. Kotzias, G. Lagorio, S. Aonzo, and D. Balzarotti, “Longitudinal Study of the Prevalence of Malware Evasive Techniques,” Dec. 2021, [Online]. Available: <http://arxiv.org/abs/2112.11289>
- [7] F. Biondi, T. Given-Wilson, A. Legay, C. Puodzius, and J. Quilbeuf, “Tutorial: an Overview of Malware Detection and Evasion Techniques.”
- [8] M. Miraoui and M. Ben Belgacem, “Binary and multiclass malware classification of windows portable executable using classic machine learning and deep learning,” *Front. Comput. Sci.*, vol. 7, 2025, doi: 10.3389/fcomp.2025.1539519.
- [9] C. K. Yuk and C. J. Seo, “Static Analysis and Machine Learning-based Malware Detection System using PE Header Feature Values,” *International Journal of Innovative Research and Scientific Studies*, vol. 5, no. 4, pp. 281–288, 2022, doi: 10.53894/ijirss.v5i4.690.
- [10] O. Or-Meir, N. Nissim, Y. Elovici, and L. Rokach, “Dynamic malware analysis in the modern era—A state of the art survey,” *ACM Comput. Surv.*, vol. 52, no. 5, Sep. 2020, doi: 10.1145/3329786.
- [11] L. Binxiang, Z. Gang, and S. Ruoying, “A deep reinforcement learning malware detection method based on PE feature distribution,” in *Proceedings - 2019 6th International Conference on Information Science and Control Engineering, ICISCE 2019*, Institute of Electrical and Electronics Engineers Inc., Dec. 2019, pp. 23–27. doi: 10.1109/ICISCE48695.2019.00014.
- [12] T. Oladipupo, “Types of Machine Learning Algorithms,” in *New Advances in Machine Learning*, InTech, 2010. doi: 10.5772/9385.
- [13] F. C. Carlo Garcia and F. P. Muga II, “Random Forest for Malware Classification.”

- [14] S. A. Roseline, S. Geetha, S. Kadry, and Y. Nam, "Intelligent Vision-Based Malware Detection and Classification Using Deep Random Forest Paradigm," *IEEE Access*, vol. 8, pp. 206303–206324, 2020, doi: 10.1109/ACCESS.2020.3036491.
- [15] S. Yoo, S. Kim, S. Kim, and B. B. Kang, "AI-HydRa: Advanced hybrid approach using random forest and deep learning for malware classification," *Inf. Sci. (N. Y.)*, vol. 546, pp. 420–435, Feb. 2021, doi: 10.1016/j.ins.2020.08.082.
- [16] M. Esnaashari and N. Moradi, "Predicting Vulnerability to Malware Using Machine Learning Models: A Study on Microsoft Windows Machines," Jan. 2025, [Online]. Available: <http://arxiv.org/abs/2501.02493>
- [17] U. Baldangombo, N. Jambaljav, and S.-J. Horng, "A STATIC MALWARE DETECTION SYSTEM USING DATA MINING METHODS."
- [18] M. Goyal and R. Kumar, "Machine Learning for Malware Detection on Balanced and Imbalanced Datasets," in *2020 International Conference on Decision Aid Sciences and Application, DASA 2020*, Institute of Electrical and Electronics Engineers Inc., Nov. 2020, pp. 867–871. doi: 10.1109/DASA51403.2020.9317206.
- [19] M. Kruczkowski and E. Niewiadomska-Szynkiewicz, "Support vector machine for malware analysis and classification," in *Proceedings - 2014 IEEE/WIC/ACM International Joint Conference on Web Intelligence and Intelligent Agent Technology - Workshops, WI-IAT 2014*, Institute of Electrical and Electronics Engineers Inc., Oct. 2014, pp. 415–420. doi: 10.1109/WI-IAT.2014.127.
- [20] *2016 International Conference on Advanced Communication Control and Computing Technologies*. IEEE, 2016.
- [21] W. K. Wong, F. H. Juwono, and C. Apriono, "Vision-Based Malware Detection: A Transfer Learning Approach Using Optimal ECOC-SVM Configuration," *IEEE Access*, vol. 9, pp. 159262–159270, 2021, doi: 10.1109/ACCESS.2021.3131713.
- [22] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," Dec. 2015, [Online]. Available: <http://arxiv.org/abs/1512.03385>
- [23] R. Chaganti, V. Ravi, and T. D. Pham, "Image-based malware representation approach with EfficientNet convolutional neural networks for effective malware classification," *Journal of Information Security and Applications*, vol. 69, Sep. 2022, doi: 10.1016/j.jisa.2022.103306.
- [24] S. Kumar, Y. Sapru, J. Kumar, and S. Sapru, "Image analysis and fine-tuned ResNet50 model for effective malware detection," in *Procedia Computer Science*, Elsevier B.V., 2025, pp. 1098–1106. doi: 10.1016/j.procs.2025.05.019.
- [25] R. U. Khan, X. Zhang, and R. Kumar, "Analysis of ResNet and GoogleNet models for malware detection," *Journal of Computer Virology and Hacking*

- Techniques*, vol. 15, no. 1, pp. 29–37, Mar. 2019, doi: 10.1007/s11416-018-0324-z.
- [26] A. Dosovitskiy *et al.*, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” Jun. 2021, [Online]. Available: <http://arxiv.org/abs/2010.11929>
 - [27] S. Seneviratne, R. Shariffdeen, S. Rasnayaka, and N. Kasthuriarachchi, “Self-Supervised Vision Transformers for Malware Detection,” *IEEE Access*, vol. 10, pp. 103121–103135, 2022, doi: 10.1109/ACCESS.2022.3206445.
 - [28] I. W. Jeong, H. J. Lee, G. N. Kim, and S. H. Choi, “MalFormer: A Novel Vision Transformer Model for Robust Malware Analysis,” *IEEE Access*, vol. 13, pp. 122671–122683, 2025, doi: 10.1109/ACCESS.2025.3588232.
 - [29] S. Bavishi and S. Modi, “Accelerating Malware Classification: A Vision Transformer Solution,” Sep. 2024, [Online]. Available: <http://arxiv.org/abs/2409.19461>
 - [30] S. Aurangzeb, H. Anwar, M. A. Naeem, and M. Aleem, “BigRC-EML: big-data based ransomware classification using ensemble machine learning,” *Cluster Comput.*, vol. 25, no. 5, pp. 3405–3422, Oct. 2022, doi: 10.1007/s10586-022-03569-4.
 - [31] J. Chen, C. Yuan, J. Li, D. Tian, R. Ma, and X. Jia, “ELAMD: An ensemble learning framework for adversarial malware defense,” *Journal of Information Security and Applications*, vol. 75, Jun. 2023, doi: 10.1016/j.jisa.2023.103508.
 - [32] B. Hammi, J. Hachem, A. Rachini, and R. Khatoun, “Malware Detection Through Windows System Call Analysis.” [Online]. Available: www.statista.com/statistics/494947/ransomware-attempts-per-year-
 - [33] Y. Zhang, Q. Huang, X. Ma, Z. Yang, and J. Jiang, “Using Multi-features and Ensemble Learning Method for Imbalanced Malware Classification,” 2016, doi: 10.1109/TrustCom/BigDataSE/ISPA.2016.161.
 - [34] G. G. Behera, J. Pradhan, and A. K. Mishra, “A Hybrid Deep Learning Malware Detection Model with the Fusion of VGG-16 and ResNet-50 Architectures,” in *Procedia Computer Science*, Elsevier B.V., 2025, pp. 1659–1668. doi: 10.1016/j.procs.2025.04.397.

PHỤ LỤC

Mã nguồn:

https://github.com/truongvip1/KTLN_MalwareDetection_EnsembleLearning